University of Toronto

CSC263

Data Structures and Analysis

Lecture Notes

Fall 2026

Instructor: Sam Toueg

Cheryl Shi

Contents

Lecture 1: Worst Case Time Complexity **7**

1 Steps and Input Size **7**

2 Worst Case Time Complexity **7**

3 Proving Bounds on $T(n)$ **7**

4 Two Issues $\Rightarrow$ Asymptotic Notation **8**

5 Big O (Upper Bound) **8**

6 Big Ω (Lower Bound) **8**

7 Big Θ (Tight Bound) **9**

8 Worked Example: Bubble Sort is $\Theta(n^2)$ **9**

Lecture 2: Priority Queues and Max-Heaps **11**

1 ADT vs. Data Structure **11**

2 The Priority Queue ADT **11**

3 Naive Implementations: Why We Need Something Better **11**

4 Complete Binary Tree (CBT) **11**
4.1 Height . . . 12

5 Max-Heap **12**

6 Array Representation **13**

7 The Three Operations **14**
7.1 Max(A) . . . 14
7.2 Insert(A, x): percolate up . . . 14
7.3 Extract_Max(A): drip down . . . 15

8 Complexity Analysis **16**

9 HeapSort **17**

10 Past Exam: k-Set ADT **17**

Quick Reference: Lecture 2 **18**

Lecture 3: Min Binomial Heaps **19**

1 Mergeable Priority Queue ADT **19**

2 Binomial Trees **19**
2.1 Recursive Definition of B_k . . . 19

2.2 Children Sequence of B_k . . . 20
2.3 Properties of B_k . . . 21
2.4 Why "Binomial"? . . . 21

3 Binomial Forest F_n 21
3.1 Connection to Binary Representation . . . 21
3.2 What is $\alpha(n)$? . . . 22
3.3 Properties of F_n . . . 22

4 Min Binomial Heap: The Full Definition 22

5 Building a Binomial Heap: Worked Example 23

6 Memory Representation 24
6.1 Three Pointers Per Node . . . 24
6.2 Visual vs. Memory Representation . . . 24
6.3 HEAD Pointer and Root List . . . 24

7 Two Key Lemmas 25
7.1 Lemma 1: Merging Two B_k Trees . . . 25
7.2 Lemma 2: Deleting the Root of B_k . . . 26

8 Operations 27
8.1 Union(T, Q): Merging Two Binomial Heaps . . . 27
8.2 Insert(T, x): Adding a Single Element . . . 28
8.3 Min(T): Finding the Minimum . . . 29
8.4 Extract_Min(T): Remove the Minimum . . . 29

9 Complexity Summary 30

10 Practice Problems 30

11 Exam Relevance 31

Quick Reference: Lecture 3 32

Lecture 4: Binomial Heaps (Part 2) + BST 32

1 Remaining Binomial Heap Operations 32
1.1 Decrease_Key(T, x, k): Make a Key Smaller . . . 32
1.2 Delete(T, x): Remove a Specific Node . . . 33
1.3 Increase_Key(T, x, k): Make a Key Larger . . . 34
1.4 Complete Binomial Heap Operations Summary . . . 34

2 Cost of k Successive Inserts 35
2.1 Key Insight: Each Insert = Adding 1 in Binary . . . 35
2.2 Worked Example: 5 Successive Inserts into $|T| = 27$ . . . 35
2.3 Exact Formula via Edge Counting . . . 36

3 The Dictionary ADT 38
3.1 First Attempt: Linked List . . . 38

4 Binary Search Trees (BST) 38
4.1 The BST Property . . . 38

4.2 In-Order Traversal . . . 39
4.3 Left Rotation . . . 40
4.4 BST Operations . . . 41
4.5 BST Delete(T, x): Three Cases . . . 42
4.6 The Problem: Worst-Case Height . . . 45

5 Past Exam Questions **46**

Quick Reference: Lecture 4 **47**

Lecture 5: Balanced BSTs and AVL Trees **48**

1 Recall: the Problem with BSTs **48**

2 Height and Balance Factor **48**
2.1 Height of a Node . . . 48
2.2 Balance Factor . . . 48

3 AVL Trees **48**
3.1 Definition (Adelson-Velsky and Landis) . . . 49
3.2 Is This an AVL Tree? . . . 49
3.3 Properties . . . 49

4 AVL Operations **50**

5 Insert **50**
5.1 General Idea . . . 51
5.2 Worked Example 1: No Rebalancing . . . 51
5.3 Worked Example 2: Single Rotation . . . 52
5.4 Worked Example 3: Double Rotation . . . 52
5.5 The Four Rotation Cases . . . 53
5.6 Insert Algorithm Sketch . . . 54

Quick Reference: Lecture 5 **54**

Lecture 6: AVL Insert Correctness and Cost **55**

1 What This Lecture Adds **55**

2 Why the Insert Algorithm is Correct **55**
2.1 The Skipped Step: BF$(B) \neq 0$ . . . 55
2.2 Case 1(a): Single Left Rotation . . . 55
2.3 Case 1(b): Double Right-Left Rotation . . . 56

3 Cost, Node Layout, and Algorithm **57**
3.1 Rotation Cost and Total Insert Cost . . . 57
3.2 Contents of an AVL Node . . . 57
3.3 Final Insert Algorithm . . . 57

4 Exam-Style Practice **58**

Quick Reference: Lecture 6 **58**

Lecture 7: Augmenting Data Structures **59**

1 Augmenting a Data Structure **59**

2 Dynamic Order Statistics **59**
2.1 Example . . . 59
2.2 Naive Attempt: Store Each Node's Rank . . . 59

3 Size-Augmented AVL Tree **60**
3.1 The `size` Field . . . 60
3.2 Select: the k-th Smallest . . . 60
3.3 Rank of a Node . . . 62
3.4 Maintaining `size` on Insert and Delete . . . 62

4 Summary and Practice **63**

Quick Reference: Lecture 7 **63**

Lecture 8: Hash Tables **64**

5 Recall: the Dictionary ADT **64**

6 If the Universe U is Small: Direct-Access Table **64**

7 When the Universe is Huge **64**

8 Hash Tables **65**

9 Resolving Collisions: Chaining **65**

10 Dictionary Operations with Chaining **65**

11 Analysis: SUHA and the Load Factor **66**

12 Expected Cost of Search (under SUHA) **66**

13 Choosing a Hash Function **67**

14 Past Exam Questions **67**

Quick Reference: Lecture 8 **68**

Lecture 9: Bloom Filters **69**

15 What is a Bloom Filter? **69**

16 Structure **69**

17 Worked Example: BF$[0 \ldots 7]$ with $t = 2$ **70**

18 Probability of a False Positive **70**

19 Choosing t: the Optimal Number of Hash Functions **71**

20 Past Exam Question **72**

Quick Reference: Lecture 9 **72**

Lecture 10: Randomized Quicksort **74**

21 The Algorithm **74**

22 Worst-Case Number of Comparisons **74**

23 Expected Number of Comparisons: Indicator Variables **75**

24 The Key Lemma **75**

25 Putting it Together: $E(C) = O(n \log n)$ **76**

26 Past Exam Question **76**

Quick Reference: Lecture 10 **77**

Tutorial: Augmented Data Structures & Interval Trees **79**

27 The Problem: Overlapping Intervals **79**

28 Designing an Augmented Data Structure (the Method) **79**

29 Maintaining the Augmentation $\max[x]$ Efficiently **80**

30 Interval-Search(x) **80**

31 Why it is Correct **81**

32 Past Exam Questions **82**

Quick Reference: Interval Trees **83**

Lecture 11: Disjoint Sets (Union–Find), Part 1 **84**

1 The Disjoint Sets ADT **84**

2 Implementation 1: Linked Lists **84**

3 Implementation 2: Augmented Linked Lists **85**
3.1 Weighted Union (WU) on augmented lists 85

4 Implementation 3: Disjoint-Set Forest **85**

5 Heuristic 1: Weighted Union (WU) by Size **86**

6 Past Exam Questions **87**

Quick Reference: Lecture 11 **88**

Lecture 12: Disjoint Sets (Union–Find), Part 2 **89**

1 Where We Are **89**

2 Heuristic 2: Path Compression (PC) **89**

3 A Quick Detour: the Tower 2^{*n} and $\log^* n$ **89**

4 Time Complexity with WU and PC **90**

5 Analysis of the Disjoint-Set Forest: a Timeline **90**

6 Past Exam Questions **91**

Quick Reference: Lecture 12 **92**

Lecture 1: Worst Case Time Complexity

1 Steps and Input Size

We count **basic operations**, each taking constant time:

Arithmetic	$+, -, *, /$
Comparisons	$>, <, =$
Data moves	load, store, copy

Input size n depends on the problem: number of array elements (sorting), number of vertices + edges (graphs), or number of bits (primality).

2 Worst Case Time Complexity

Worst case time complexity $T(n)$

Let A be an algorithm and $t(x) =$ steps A takes on input x.

$$T(n) = \max\{\, t(x) \mid x \text{ is an input of size } n \,\}$$

$T(n) =$ the **maximum** steps over *all* inputs of size n.

★ **Seen on a past exam** (Midterm, Fall 2014, Q1a)

```
FindLast(A, x):
    i ← A.length − 1
    while i ≥ 0 and A[i] ≠ x:
        i ← i − 1
    return i
```

Q. What is the worst case complexity of `FindLast`? (Count array accesses only.)

A. $T(n) = n$.
Worst case: $x \notin A$, so we scan every element exactly once $\Rightarrow$ n accesses. Cannot exceed n, so $T(n) = n$.

3 Proving Bounds on $T(n)$

The key: unpack what max means.

The max trick

$$\max(S) \le c \iff \forall e \in S : e \le c$$
$$\max(S) \ge c \iff \exists e \in S : e \ge c$$

Apply to $T(n) = \max\{t(x)\}$:

$T(n) \leq 7n^3 \iff$ for **every** input of size n, A takes **at most** $7n^3$ steps

$T(n) \geq 3n^2 \iff$ for **some** input of size n, A takes **at least** $3n^2$ steps

Example: proving $T(n) \geq 3n$ for FindLast

We need **some** input of size n with $t(x) \geq 3n$. Hmm, FindLast does at most n accesses, so $T(n) \geq 3n$ is **false**. But $T(n) \geq n$ is true: choose $x \notin A$, then $t(x) = n \geq n$. ✓

4 Two Issues ⇒ Asymptotic Notation

$T(n) \leq 7n^3$ vs. $T(n) \leq 100n^3$? The *shape* is n^3 either way. We want to ignore the constant.

The bound might fail for $n = 1, 2, 3$ but hold for all $n \geq 10$. We only need it for **sufficiently large** n.

Fix both ⇒ **asymptotic notation**: O, Ω, Θ.

5 Big O (Upper Bound)

Big O (formal)

$T(n)$ is $O(g(n)) \iff \exists\, c > 0,\ \exists\, n_0 > 0$ such that $\forall\, n \geq n_0$:

$$T(n) \leq c \cdot g(n)$$

For **every** input of size n (large enough), the algorithm takes **at most** $c \cdot g(n)$ steps.

Example: show $T(n) = 5n^2 + 3n$ is $O(n^2)$

Need c, n_0 such that $5n^2 + 3n \leq c \cdot n^2$ for all $n \geq n_0$.
For $n \geq 1$: $5n^2 + 3n \leq 5n^2 + 3n^2 = 8n^2$.
So $c = 8$, $n_0 = 1$. ✓

6 Big Ω (Lower Bound)

Big Ω (formal)

$T(n)$ is $\Omega(g(n)) \iff \exists\, c > 0,\ \exists\, n_0 > 0$ such that $\forall\, n \geq n_0$:

$$T(n) \geq c \cdot g(n)$$

For **some** input of size n (large enough), the algorithm takes **at least** $c \cdot g(n)$ steps.

O vs Ω in one line

O = **every** input, **at most**. Ω = **some** input, **at least**.

Example: show $T(n) = 5n^2 + 3n$ is $\Omega(n^2)$

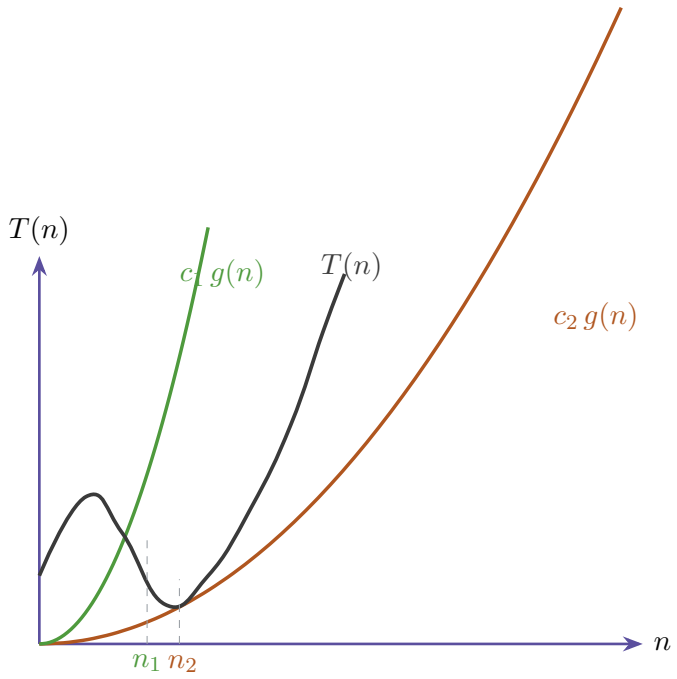
Need c, n_0 such that $5n^2 + 3n \geq c \cdot n^2$ for all $n \geq n_0$.
For $n \geq 1$: $5n^2 + 3n \geq 5n^2$.
So $c = 5$, $n_0 = 1$. ✓

7 Big Θ (Tight Bound)

Big Θ (formal)

$$T(n) \text{ is } \Theta(g(n)) \iff T(n) \text{ is } O(g(n)) \textbf{ and } T(n) \text{ is } \Omega(g(n))$$

Same $g(n)$ works as both upper bound and lower bound.



Past n_2, $T(n)$ is trapped between the $c_2 g(n)$ floor and the $c_1 g(n)$ ceiling. That is Θ.

8 Worked Example: Bubble Sort is $\Theta(n^2)$

BubbleSort(A)

```
for i = 1 to A.length − 1:
    for j = A.length downto i + 1:
        if A[j] < A[j−1]:  swap A[j] and A[j−1]
```

Bubble Sort: $T(n) = \Theta(n^2)$

$O(n^2)$: For **every** input, the nested loops do $\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$ comparisons. So $T(n) \leq c_1 n^2$.
$\Omega(n^2)$: For **some** input (reverse-sorted: $[n, n-1, \ldots, 1]$), every comparison triggers a swap, doing exactly $\frac{n(n-1)}{2}$ comparisons. So $T(n) \geq c_2 n^2$.
Same shape $\Rightarrow \Theta(n^2)$.

Practice: is $3n^2 + 5n \log n$ in $\Theta(n^2)$?

$O(n^2)$: $3n^2 + 5n \log n \leq 3n^2 + 5n^2 = 8n^2$ for $n \geq 1$. ✓
$\Omega(n^2)$: $3n^2 + 5n \log n \geq 3n^2$ for $n \geq 1$. ✓
Yes, $3n^2 + 5n \log n = \Theta(n^2)$.

Quick Reference: Lecture 1

$T(n)$	$\max\{t(x) \mid x \text{ of size } n\}$	
$O(g(n))$	$T(n) \leq c\,g(n)$ eventually	**every** input, **at most**
$\Omega(g(n))$	$T(n) \geq c\,g(n)$ eventually	**some** input, **at least**
$\Theta(g(n))$	both O and Ω with same g (tight)	
Bubble sort	$\Theta(n^2)$	

9 Midterm-Style Process Problems

Predicted: prove a bound with an explicit worst-case input

```
Scan(A[1..n]):
    i ← 1
    while i ≤ n and A[i] ≥ 0:
      i ← i + 1
    return i
```

Let $T(n)$ count **array accesses**. Prove $T(n) = \Theta(n)$: show both the O upper bound and the Ω lower bound, and for the lower bound give an **explicit** input.

O **(upper bound).** The loop body runs at most once per index $i = 1, \ldots, n$, and each iteration does **at most** one access $A[i]$. After index n the test $i \leq n$ fails with no further access. So for **every** input, $T(n) \leq n$, giving $T(n) = O(n)$.
Ω **(lower bound).** We must exhibit **some** input of size n that forces $\geq cn$ accesses. Take $A = [0, 0, \ldots, 0]$ (all nonnegative). Then $A[i] \geq 0$ holds for every i, so the loop never exits early and performs **at least** n accesses. Hence $T(n) = \Omega(n)$.
Both bounds meet, so $T(n) = \Theta(n)$.

Graders reject a $\Omega(n)$ argument that does not **name a concrete input**. Saying "in the worst case it runs n times" is not a proof. You must point to one array, here all zeros, and show *it* forces the work.

Lecture 2: Priority Queues and Max-Heaps

1 ADT vs. Data Structure

Two different things

Abstract data type (ADT): specifies an object + operations. Says *what*, not *how*.
Data structure: one specific implementation of an ADT (memory layout + code for each operation).
Multiple data structures can implement the same ADT, with different costs.

2 The Priority Queue ADT

Priority Queue ADT

Object: a set S of elements, each with a comparable key (priority).
Operations:

- **Insert**(S, x): add element x into S.
- **Max**(S): return element with highest priority (no removal).
- **Extract_Max**(S): return **and remove** element with highest priority.

Application: OS job scheduling. Jobs arrive → **Insert**. Processor free → **Extract_Max** to run the highest-priority job.

3 Naive Implementations: Why We Need Something Better

Worst case	Insert	Extract_Max
Unordered linked list	$\Theta(1)$	$\Theta(n)$
Ordered linked list	$\Theta(n)$	$\Theta(1)$

Unordered: Insert at front is $\Theta(1)$, but Extract_Max scans the whole list $\Rightarrow \Theta(n)$.

Ordered: Max is at the end $\Rightarrow$ Extract_Max is $\Theta(1)$, but inserting in sorted position $\Rightarrow \Theta(n)$.

Goal

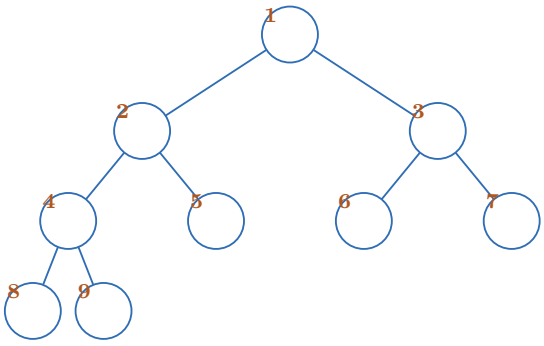
Both operations in $O(\log n)$. A **max-heap** achieves this.

4 Complete Binary Tree (CBT)

Complete Binary Tree

A binary tree where:

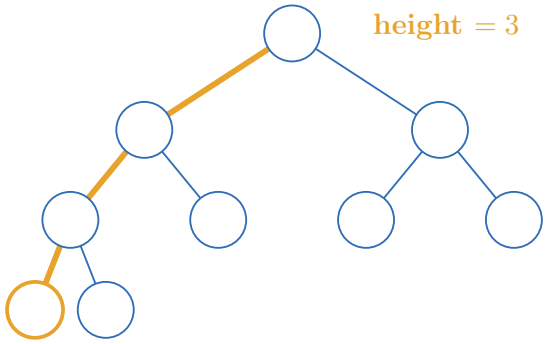
- every level except possibly the last is completely full,
- the last level is filled from the left with no gaps.



4.1 Height

Height of a tree

Number of edges on the longest root-to-leaf path.



Key fact

A CBT with n nodes has height $\lfloor \log_2 n \rfloor$.
Each full level doubles the node count $(1, 2, 4, 8, \ldots)$, so $\log_2 n$ levels suffice for n nodes.

5 Max-Heap

Max-Heap = CBT + max-heap property

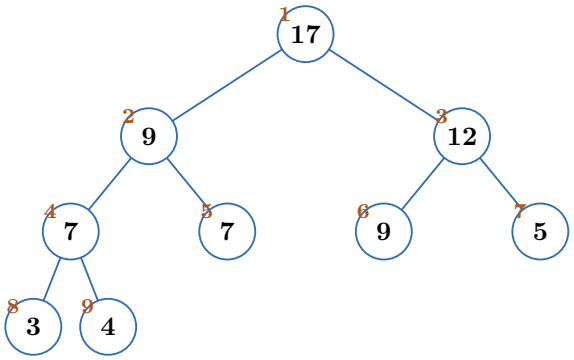
Elements stored in a CBT such that:

$$\text{priority(node)} \geq \text{priority(each child)}$$

This is a **local** rule (parent vs. own children only). It chains upward, so the **global max is always at the root**.

Example

$S = \{3, 4, 5, 7, 7, 9, 9, 12, 17\}$. One valid max-heap:

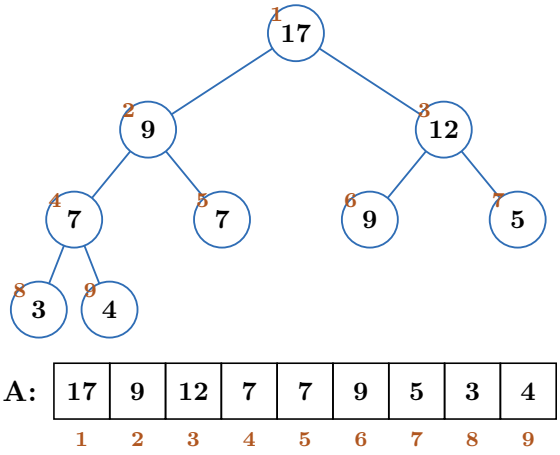


Check: $17 \geq 9, 12$. $\quad 9 \geq 7, 7$. $\quad 12 \geq 9, 5$. ✓

Swapping children of the root ($9 \leftrightarrow 12$) gives a different valid heap $\Rightarrow$ max-heaps are not unique. Only the root (global max) is guaranteed.

6 Array Representation

Store the tree **level by level, left to right** in array `A[1..n]`. No pointers needed.



A:	17	9	12	7	7	9	5	3	4
	1	2	3	4	5	6	7	8	9

Navigation formulas

For node at index i:

- **Left child:** $2i$ **Right child:** $2i+1$ **Parent:** $\lfloor i/2 \rfloor$

`A.heapsize` tracks the number of elements currently in the heap.

Verify: node at $i = 2$ holds 9

Left child: $A[2 \cdot 2] = A[4] = 7$ ✓ Right child: $A[2 \cdot 2 + 1] = A[5] = 7$ ✓
Parent: $A[\lfloor 2/2 \rfloor] = A[1] = 17$ ✓
Each level is $\sim 2\times$ the width of the one above, so going down doubles the index, going up halves it.

7 The Three Operations

Invariant for every operation

1) Maintain the CBT shape. 2) Maintain the max-heap property.

7.1 Max(A)

Max(A)

```
return A[1]
```

The root always holds the max. $\Rightarrow \Theta(1)$.

7.2 Insert(A, x): percolate up

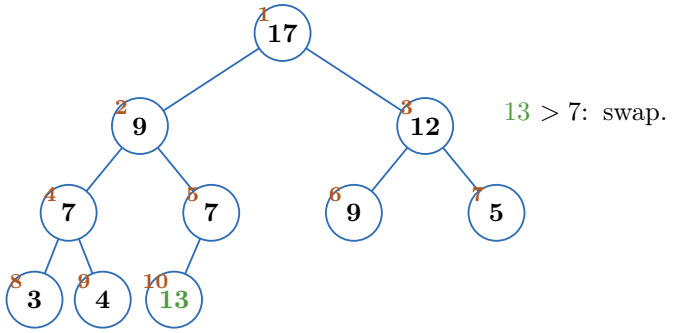
Insert(A, x)

```
1. A.heapsize++; A[A.heapsize] = x          (place at next open leaf)
2. while x is not root and priority(x) > priority(parent(x)):
       swap x with parent(x)                (percolate up)
```

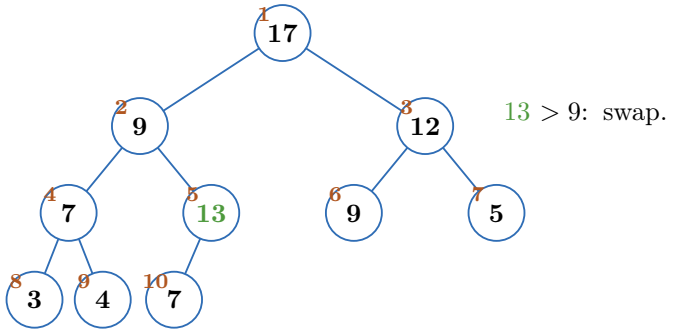
Worked example: Insert(A, 13)

Start: $A = [17, 9, 12, 7, 7, 9, 5, 3, 4]$. Insert 13 at index 10.

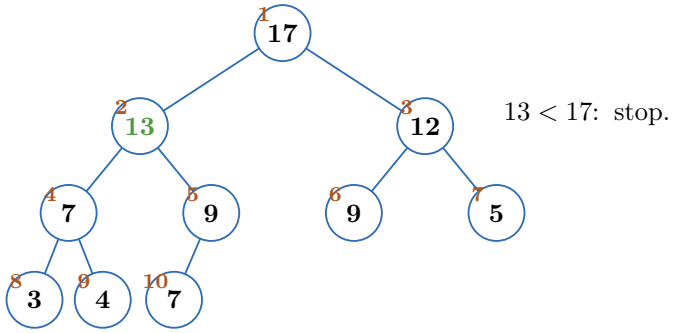
Step 1. Place 13 at $A[10]$. Property broken: $13 > 7$ (parent at $i = 5$).



Step 2a. Swap $13 \leftrightarrow 7$. Now 13 at $i = 5$. Compare: $13 > 9$ (parent at $i = 2$).



Step 2b. Swap $13 \leftrightarrow 9$. Now 13 at $i = 2$. Compare: $13 < 17$ (root). Stop.



Result: $A = [17, 13, 12, 7, 9, 9, 5, 3, 4, 7]$, `heapsize`$= 10$.

7.3 Extract_Max(A): drip down

Extract_Max(A)

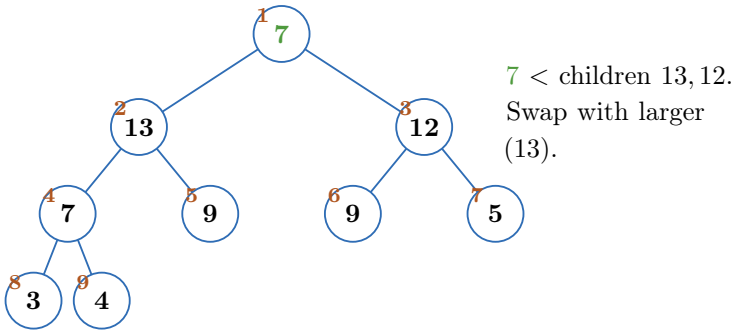
```
1. max = A[1]                                          (save the root)
2. A[1] = A[A.heapsize]; A.heapsize--                  (move last leaf to root)
3. while x has a child with higher priority:
       swap x with its largest child                   (drip down)
4. return max
```

Important: always swap with the *larger* child (otherwise we create a new violation). Node at index i is a leaf when $i > \lfloor \texttt{heapsize}/2 \rfloor$.

Worked example: Extract_Max(A)

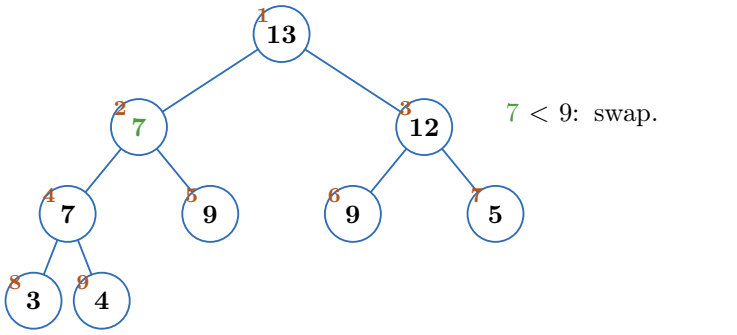
Start: $A = [17, 13, 12, 7, 9, 9, 5, 3, 4, 7]$, heapsize$= 10$.

Steps 1 & 2. Save 17. Move $A[10] = 7$ to root. Shrink heapsize to 9.



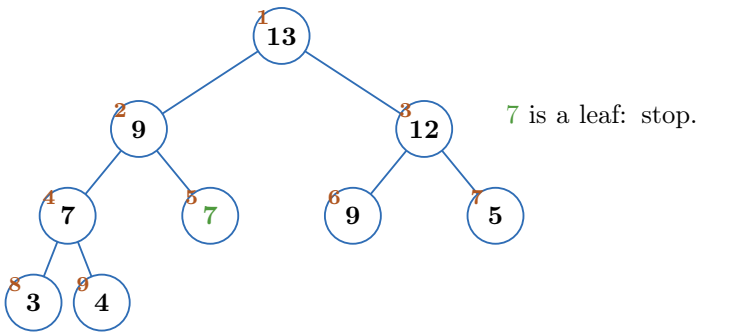
7 < children 13, 12. Swap with larger (13).

Step 3a. Swap $7 \leftrightarrow 13$. Now 7 at $i = 2$. Children: $7, 9$. Larger is $9 > 7$.



7 < 9: swap.

Step 3b. Swap $7 \leftrightarrow 9$. Now 7 at $i = 5$. Children at $i = 10 >$ heapsize$= 9$, so 7 is a leaf. Stop.



7 is a leaf: stop.

Return 17. Heap: $[13, 9, 12, 7, 7, 9, 5, 3, 4]$.

8 Complexity Analysis

Each operation walks one root-to-leaf (or leaf-to-root) path. Path length $\leq$ height $= \lfloor \log_2 n \rfloor$. Constant work per level.

Heap operation costs

Max(A)	$\Theta(1)$	read root
Insert(A, x)	$\Theta(\log n)$	percolate up $\leq \lfloor \log_2 n \rfloor$ levels
Extract_Max(A)	$\Theta(\log n)$	drip down $\leq \lfloor \log_2 n \rfloor$ levels

Insert $= \Theta(\log n)$:

- $O(\log n)$: At most $\lfloor \log_2 n \rfloor$ swaps (one per level). **every** input, **at most** $c_1 \log n$.
- $\Omega(\log n)$: Insert a value larger than the current root $\Rightarrow$ percolates all the way to the top. **some** input, **at least** $c_2 \log n$.

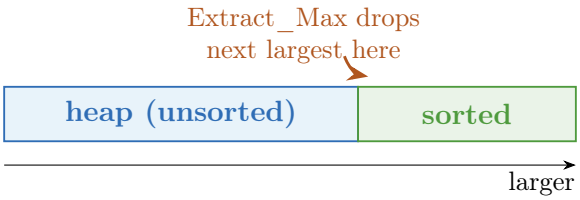
Same argument for Extract_Max (drip down with the smallest value $\Rightarrow$ sinks to a leaf).

9 HeapSort

HeapSort

1. Build a max-heap from A. ($\Theta(n)$ time)
2. Call **Extract_Max** n times. (each $\Theta(\log n)$)

Total: $n \times \Theta(\log n) = \Theta(n \log n)$



Each extraction moves the max to the front of the sorted region. The heap shrinks by one, the sorted region grows by one. After n extractions, the array is sorted in place.

10 Past Exam: k-Set ADT

★ **Seen on a past exam** (Midterm, Fall 2014, Q2)

The *rank* of an element = its position in sorted order (from 1). For fixed $k \in \mathbb{Z}^+$:

- **Object:** collection C of rationals (duplicates allowed).
- **Insert**(C, x): add x to C.
- k-**Select**(C): return element at rank k, or `nil` if $|C| < k$.

Q. Implement with data structures from class. k-**Select** must be $O(1)$.

Data structure: **max-heap** H_1 (stores the k smallest elements) + **min-heap** H_2 (stores the rest).
The k-th smallest element = the largest among the k smallest = root of H_1.

k-**Select**(C)**:**
`if` $|H_1| < k$`:` `return nil`
`else:` `return` $H_1[1]$ $\Rightarrow O(1)$

Insert(C, x)**:**
`if` $|H_1| < k$ `or` $x < H_1[1]$`:` `insert` x `into` H_1
`else:` `insert` x `into` H_2
`if` $|H_1| > k$`:` `insert Extract-Max(`H_1`) into` H_2
Constant number of heap ops, each $O(\log n) \Rightarrow O(\log n)$.

Correctness: H_1 always holds exactly the k smallest. New small values enter H_1; overflow gets pushed to H_2. Large values go straight to H_2.

11 Midterm-Style Process Problems

Predicted: Insert into a max-heap, show every sift-up

Insert 15 into the max-heap $A = [16, 14, 10, 8, 7, 9, 3]$. Show the array after **each** swap.

Append 15 at the next free slot (index 8), then **sift up** while it beats its parent (parent of i is $\lfloor i/2 \rfloor$).

	1	2	3	4	5	6	7	8
A:	**16**	**14**	**10**	**8**	**7**	**9**	**3**	**15**

Index 8, parent index 4 is 8: $15 > 8$, swap.

	1	2	3	4	5	6	7	8
A:	**16**	**14**	**10**	**15**	**7**	**9**	**3**	**8**

Index 4, parent index 2 is 14: $15 > 14$, swap.

	1	2	3	4	5	6	7	8
A:	**16**	**15**	**10**	**14**	**7**	**9**	**3**	**8**

Index 2, parent index 1 is 16: $15 < 16$, stop. Final heap:

	1	2	3	4	5	6	7	8
A:	**16**	**15**	**10**	**14**	**7**	**9**	**3**	**8**

Sift-up touches one node per level, so Insert is $O(\log n)$.

Predicted: Extract-Max, show every sift-down

From the heap $A = [16, 15, 10, 14, 7, 9, 3, 8]$ (size 8), perform EXTRACT-MAX. Show the array after each swap.

Save the root 16 (this is the returned max). Move the **last** element 8 to the root, drop the size to 7, then **sift down**, always swapping with the *larger* child (children of i are $2i$ and $2i+1$).

	1	2	3	4	5	6	7
A:	**8**	**15**	**10**	**14**	**7**	**9**	**3**

Index 1, children 15 (idx 2) and 10 (idx 3): larger is $15 > 8$, swap.

	1	2	3	4	5	6	7
A:	**15**	**8**	**10**	**14**	**7**	**9**	**3**

Index 2, children 14 (idx 4) and 7 (idx 5): larger is $14 > 8$, swap.

	1	2	3	4	5	6	7
A:	**15**	**14**	**10**	**8**	**7**	**9**	**3**

Index 4, children would be idx $8, 9$ which are past size 7: no children, stop. Returns 16; final heap:

	1	2	3	4	5	6	7
A:	**15**	**14**	**10**	**8**	**7**	**9**	**3**

Sift-down also walks one path, so EXTRACT-MAX is $O(\log n)$.

Quick Reference: Lecture 2

Max-heap property	priority(node) $\geq$ priority(children); max at root `A[1]`
Array navigation	left $= 2i$, right $= 2i+1$, parent $= \lfloor i/2 \rfloor$
Height of CBT	$\lfloor \log_2 n \rfloor$
Max	read `A[1]` $\to \Theta(1)$
Insert	place at end, percolate up $\to \Theta(\log n)$
Extract_Max	move last to root, drip down $\to \Theta(\log n)$
HeapSort	build heap + n extractions $\to \Theta(n \log n)$

Lecture 3: Min Binomial Heaps

1 Mergeable Priority Queue ADT

Last lecture: Priority Queue ADT (Insert, Max, Extract_Max). Now we flip to **min** (smallest key = highest priority) and add one more operation:

Mergeable Priority Queue ADT

Object: a set S of elements, each with a comparable key.
Operations:

- **Insert**(S, x): add element x to S.
- **Min**(S): return the element with the smallest key.
- **Extract_Min**(S): remove and return the element with the smallest key.
- **Union**(S_1, S_2): merge two priority queues into a single one.

Operation	Min Heap (array)	Min Binomial Heap
`Insert`	$O(\log n)$	$O(\log n)$
`Min`	$O(1)$	$O(\log n)$
`Extract_Min`	$O(\log n)$	$O(\log n)$
`Union`	$O(n)$	$O(\log n)$

A binary min heap stores elements in an array. To merge two heaps you basically dump them together and rebuild. That costs $O(n)$. Binomial heaps are specifically **designed** so Union runs in $O(\log n)$.

2 Binomial Trees

2.1 Recursive Definition of B_k

Binomial Tree B_k

Base case: B_0 = a single node.

Recursive case: B_k = take two copies of B_{k-1}. Make one root the **leftmost child** of the other root.

Equivalently: the root of B_k has children $B_{k-1}, B_{k-2}, \ldots, B_1, B_0$ (left to right).

Breaking Down the Recursive Definition

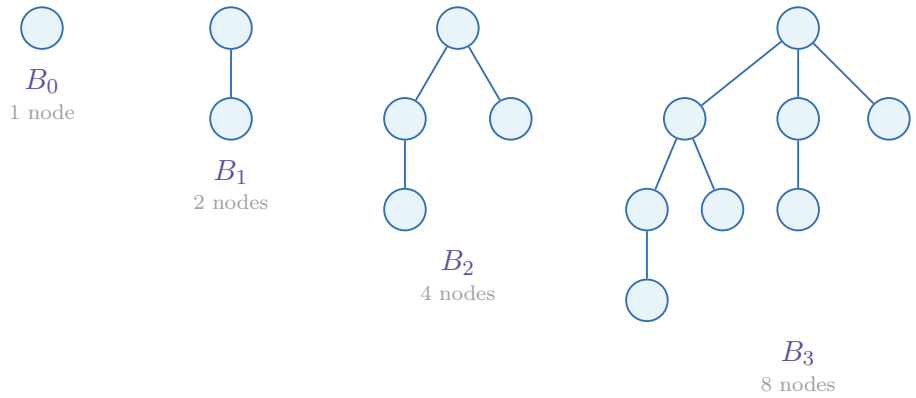
This is a common confusion point, so let's be explicit.
To build B_k:

1. Start with one copy of B_{k-1}. Call its root r.

2. Take a **second** copy of B_{k-1}.
3. Make the root of copy 2 the **new overall root**, and attach r's entire tree as a subtree under it.

So B_k = **two B_{k-1}'s glued together**, one becoming a child of the other.

Building Up From B_0



2.2 Children Sequence of B_k

Children of the Root of B_k

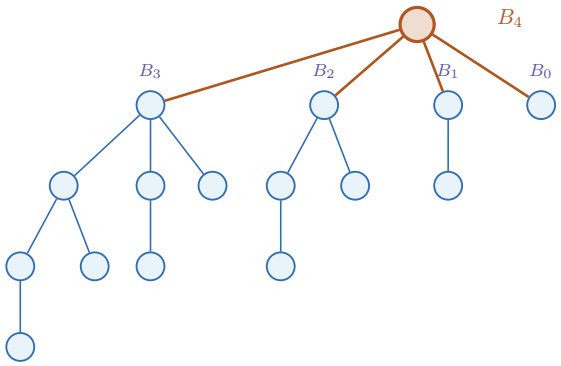
The root of B_k has exactly k children. Reading **left to right**:

$$\underbrace{B_{k-1}}_{\text{leftmost (biggest)}}, B_{k-2}, \ldots, B_1, \underbrace{B_0}_{\text{rightmost (single node)}}$$

The leftmost child is the largest subtree (B_{k-1}), and the rightmost child is the smallest (B_0). This ordering matters a lot for the memory representation later.

Visualizing B_4: Children of the Root

The root of B_4 has children B_3, B_2, B_1, B_0 (left to right, largest subtree to smallest). Total: $2^4 = 16$ nodes, height 4.



2.3 Properties of B_k

Key Properties of B_k

Height of B_k	$= k$
Number of nodes in B_k	$= 2^k$
Nodes at depth d in B_k	$= \binom{k}{d}$
Degree of root of B_k	$= k$

2.4 Why "Binomial"?

Why the Name "Binomial"

The number of nodes at depth d in B_k is $\binom{k}{d}$, the **binomial coefficient** (same "n choose k" from combinatorics).
Quick check for B_3 (8 nodes total):

$$\begin{aligned} \text{Depth 0:} \quad & \binom{3}{0} = 1 \quad \text{(the root)} \\ \text{Depth 1:} \quad & \binom{3}{1} = 3 \\ \text{Depth 2:} \quad & \binom{3}{2} = 3 \\ \text{Depth 3:} \quad & \binom{3}{3} = 1 \end{aligned}$$

Total: $1 + 3 + 3 + 1 = 8$ ✓

3 Binomial Forest F_n

Binomial Forest F_n

A **Binomial Forest** F_n is a sequence of binomial trees $\langle B_{k_1}, B_{k_2}, \ldots, B_{k_m} \rangle$ where:

- The k_i values are **strictly decreasing**: $k_1 > k_2 > \cdots > k_m$
- The total number of nodes across all trees $= n$

3.1 Connection to Binary Representation

Binary Representation ⇔ Which Trees Are Present

Write n in binary. B_i is present in F_n **if and only if** bit i (counting from the right, starting at 0) is **1**.
This is one of the most important ideas in this lecture. The entire structure of a binomial heap is dictated by the binary representation of n.

Example: F_7 and F_9

F_7: $7 = \langle 111 \rangle_2$
All three bits are 1, so $F_7 = \langle B_2, B_1, B_0 \rangle$.

Sizes: $4 + 2 + 1 = 7$ ✓

F_9: $9 = \langle 1001 \rangle_2$
Bits 0 and 3 are 1, so $F_9 = \langle B_3, B_0 \rangle$.
Sizes: $8 + 1 = 9$ ✓

3.2 What is $\alpha(n)$?

$\alpha(n)$ (popcount)

$\alpha(n)$ = the number of **1's** in the binary representation of n.
Other names: **popcount**, **Hamming weight**, **bit population count**.

Computing $\alpha(n)$

$$\begin{aligned} \alpha(7) &= \alpha(\langle 111 \rangle_2) &&= 3 \\ \alpha(9) &= \alpha(\langle 1001 \rangle_2) &&= 2 \\ \alpha(16) &= \alpha(\langle 10000 \rangle_2) &&= 1 \\ \alpha(15) &= \alpha(\langle 1111 \rangle_2) &&= 4 \end{aligned}$$

3.3 Properties of F_n

Properties of F_n

Number of trees in F_n	$= \alpha(n)$	(one per 1 bit)
Max number of trees	$\leq \lfloor \log_2 n \rfloor + 1$	
Number of edges in F_n	$= n - \alpha(n)$	

Verifying Edge Count for F_7

$F_7 = \langle B_2, B_1, B_0 \rangle$.
Edges in B_2: 3, edges in B_1: 1, edges in B_0: 0. Total = 4.
Formula: $n - \alpha(n) = 7 - 3 = 4$ ✓

4 Min Binomial Heap: The Full Definition

Min Binomial Heap

A **Min Binomial Heap** of n elements is a Binomial Forest F_n such that:

1. **One element per node:** each node stores exactly one element.
2. **Min heap ordering:** in each tree, every node's key $\leq$ all keys in its subtree. (Root of each tree = smallest key in that tree.)

A Min Binomial Heap is a **collection of trees** (a forest). Each individual tree is a binomial tree satisfying the min heap property. Do not confuse this with a single binary heap.

5 Building a Binomial Heap: Worked Example

Build a Min Binomial Heap from $S = \{10, 13, 1, 3, 8, 18, 7\}$

$n = 7$, and $7 = \langle 111 \rangle_2$, so the final forest $= \langle B_2, B_1, B_0 \rangle$.
Insert 10: $n = 1 = \langle 1 \rangle_2 \Rightarrow \langle B_0 \rangle$

$$B_0\text{: } \boxed{10}$$

Insert 13: $n = 2 = \langle 10 \rangle_2 \Rightarrow \langle B_1 \rangle$
Two B_0's $\rightarrow$ merge. Compare 10 vs 13: $10 \leq 13$, so 10 is root.

$$B_1\text{: } 10 \rightarrow 13$$

Insert 1: $n = 3 = \langle 11 \rangle_2 \Rightarrow \langle B_1, B_0 \rangle$
1 sits as the new B_0, B_1 stays.

$$B_1\text{: } 10 \rightarrow 13 \qquad B_0\text{: } \boxed{1}$$

Insert 3: $n = 4 = \langle 100 \rangle_2 \Rightarrow \langle B_2 \rangle$
3 forms B_0. Two B_0's (1 and 3): $1 \leq 3 \Rightarrow B_1$(root 1, child 3). Two B_1's (root 1 vs root 10): $1 \leq 10 \Rightarrow B_2$(root 1). 2 comparisons.

$$B_2\text{: root 1, children: } B_1(10 \rightarrow 13) \text{ and } B_0(3)$$

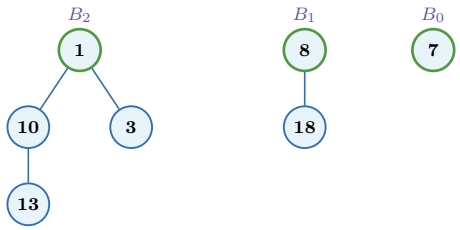
Insert 8: $n = 5 = \langle 101 \rangle_2 \Rightarrow \langle B_2, B_0 \rangle$

$$B_2(\text{root } 1) \qquad B_0\text{: } \boxed{8} \qquad 0 \text{ comparisons.}$$

Insert 18: $n = 6 = \langle 110 \rangle_2 \Rightarrow \langle B_2, B_1 \rangle$
Two B_0's (8 and 18): $8 \leq 18 \Rightarrow B_1$(root 8, child 18). 1 comparison.

$$B_2(\text{root } 1) \qquad B_1\text{: } 8 \rightarrow 18$$

Insert 7: $n = 7 = \langle 111 \rangle_2 \Rightarrow \langle B_2, B_1, B_0 \rangle$
7 is the new B_0. No merges needed. 0 comparisons.
Final heap: $\langle B_2(\text{root } 1),\ B_1(\text{root } 8),\ B_0(\text{root } 7) \rangle$



Total key comparisons: $0 + 1 + 0 + 2 + 0 + 1 + 0 = 4 = n - \alpha(n) = 7 - 3$

Important Facts

Each edge	corresponds to exactly **one key comparison**
Building from n elements	takes $O(n)$ key comparisons total

6 Memory Representation

6.1 Three Pointers Per Node

Pointer Structure

Each node stores **three pointers** plus two data fields:

Field	Points To	Color
PARENT	Parent node	Red
`key`	Element's key	(data)
`degree`	Number of children	(data)
CHILD	Leftmost child	Green
SIBLING	Right sibling	Blue

We need the PARENT pointer for Decrease Key (covered later).

6.2 Visual vs. Memory Representation

When we draw a binomial heap, the lines are **edges** (parent to child).
In memory, the actual **pointers** are:

- PARENT: child → its parent (upward)
- LEFT CHILD: parent → its **leftmost** child only
- RIGHT SIBLING: one child → the next sibling (across)

The visual edges are NOT pointers. Each visual edge is implemented through a combination of LEFT CHILD and RIGHT SIBLING pointers.

6.3 HEAD Pointer and Root List

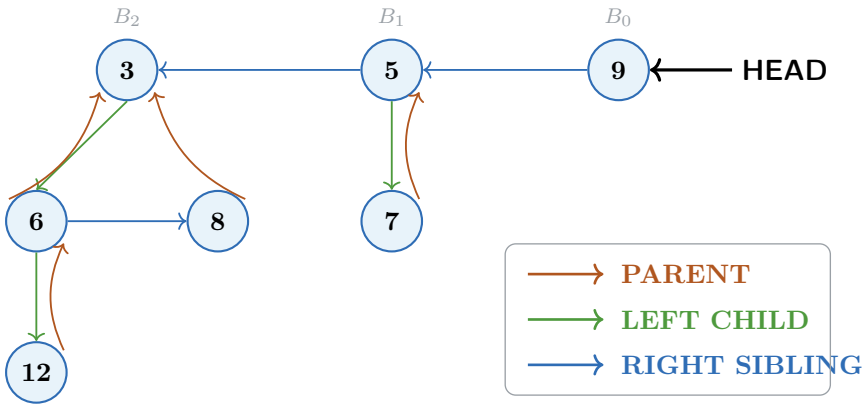
The roots of the B_k trees are connected via RIGHT SIBLING pointers, forming a **linked list of roots**. A **HEAD** pointer points to the root of the **smallest** tree (smallest k).

Memory Layout Example

Heap: $\langle B_3(\text{root } 8),\ B_2(\text{root } 1),\ B_0(\text{root } 7)\rangle$
Root linked list:

$$\texttt{HEAD} \rightarrow \boxed{7} \xrightarrow{\text{sib}} \boxed{1} \xrightarrow{\text{sib}} \boxed{8} \rightarrow \texttt{NIL}$$

HEAD points to 7 (B_0, smallest tree). Then 7's sibling pointer goes to 1 (B_2), then to 8 (B_3). Here is a full pointer diagram for a smaller heap $\langle B_2(\text{root } 3),\ B_1(\text{root } 5),\ B_0(\text{root } 9)\rangle$:



7 Two Key Lemmas

Before the operations, we need two building blocks. These lemmas are **the reason** all operations work efficiently.

7.1 Lemma 1: Merging Two B_k Trees

Lemma 1

We can merge two min heap ordered B_k trees into a single min heap ordered B_{k+1} tree using just one key comparison.

Why Lemma 1 Matters

This is the **carry operation**. Just like adding two 1 bits produces a carry in binary addition, merging two B_k's produces one B_{k+1}. This is the core mechanism behind Union, Insert, and Extract_Min.

Proof. Given two B_k trees with roots x and y:

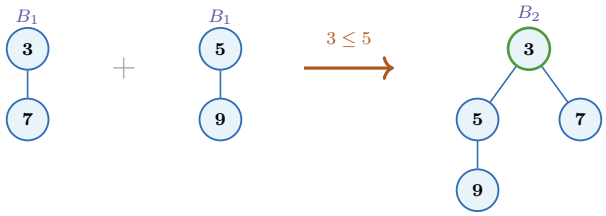
1. Compare x.key and y.key (one comparison).
2. Smaller root stays on top. Make the other tree the new leftmost subtree.

If $x \leq y$: x is root of B_{k+1}, with children $B_k(y)$, B_{k-1}, $\ldots$, B_0.

Min heap ordering preserved (smaller root on top). Structure is B_{k+1} by definition. □

Lemma 1 in Action: Merging Two B_1's into B_2

Two min heap ordered B_1 trees. One comparison decides the root.



$3 \leq 5$, so 3 stays as root. The other tree (root 5) becomes the **new leftmost child**. One comparison, one new edge.

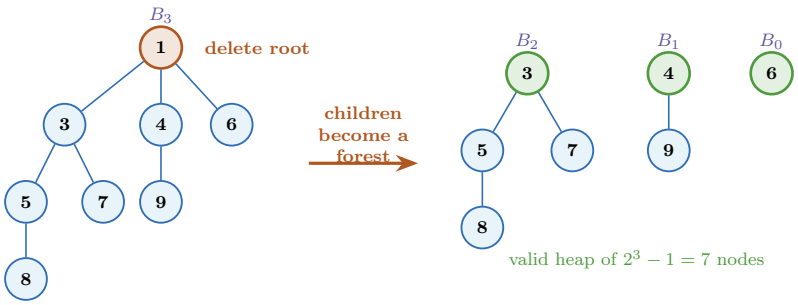
7.2 Lemma 2: Deleting the Root of B_k

Lemma 2

Deleting the root of a min heap ordered B_k gives a valid min binomial heap of $2^k - 1$ elements.

Lemma 2 in Action: Deleting the Root of B_3

Delete the root of a min heap ordered B_k and its children $(B_{k-1}, \ldots, B_0)$ fall out as a valid min binomial heap of $2^k - 1$ nodes.



The deleted root (1) was the overall minimum. Its children were already B_2, B_1, B_0 left to right, so they form a valid binomial forest, and each stays min heap ordered since their insides never changed.

Why Lemma 2 Matters

This is the key insight behind Extract_Min. When we extract the minimum (a root of some B_k tree), the remaining children form their own binomial heap. We then Union that new heap back with the rest.

Proof. Root of B_k has children $B_{k-1}, B_{k-2}, \ldots, B_1, B_0$.

Delete the root $\Rightarrow$ left with these k subtrees. Each is already min heap ordered (removing the root does not affect subtrees). The sequence $\langle B_{k-1}, \ldots, B_1, B_0 \rangle$ is a valid binomial forest (strictly decreasing k values), so it is a binomial heap of size $2^{k-1} + \cdots + 1 = 2^k - 1$. $\square$

8 Operations

The Big Picture

Everything reduces to Union.

- Insert = Union with a tiny heap (single B_0)
- Extract_Min = remove a root (Lemma 2 gives a new heap) + Union
- Union itself = binary addition using Lemma 1 as the carry

Lemma 1 ($O(1)$ merge) and Lemma 2 (delete root $\Rightarrow$ valid heap) are the two pillars.

8.1 Union(T, Q): Merging Two Binomial Heaps

Union(T, Q)

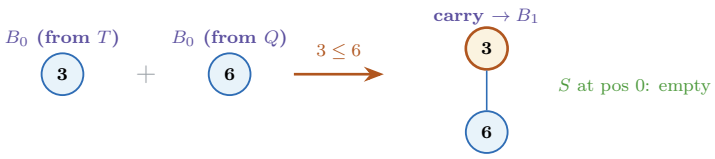
Input: two min binomial heaps T and Q.
Output: a single min binomial heap S containing all elements.
Algorithm: Binary addition!
Each column (bit position i) can have 0, 1, 2, or 3 copies of B_i (from T, from Q, plus possibly a carry from position $i-1$). Process right to left:

- **0 trees**: nothing in S, no carry
- **1 tree**: put it in S, no carry
- **2 trees**: merge via Lemma 1 into B_{i+1}, carry it. Nothing in S at position i.
- **3 trees**: put one in S, merge the other two into a carry

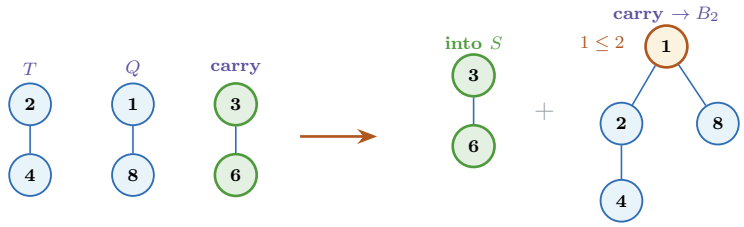
Full Worked Example: Union(T,Q) step by step

$T = \langle B_1, B_0\rangle$ (size $3 = 011_2$), $Q = \langle B_2, B_1, B_0\rangle$ (size $7 = 111_2$). Result S: $3 + 7 = 10 = 1010_2 \Rightarrow \langle B_3, B_1\rangle$.
Rule (binary addition, right to left): per column, 0/1 tree $\rightarrow$ drop into S; 2 trees $\rightarrow$ merge (1 comparison) and carry; 3 trees $\rightarrow$ one into S, merge the other two and carry.

Position 0 (B_0): two B_0's $\Rightarrow$ merge

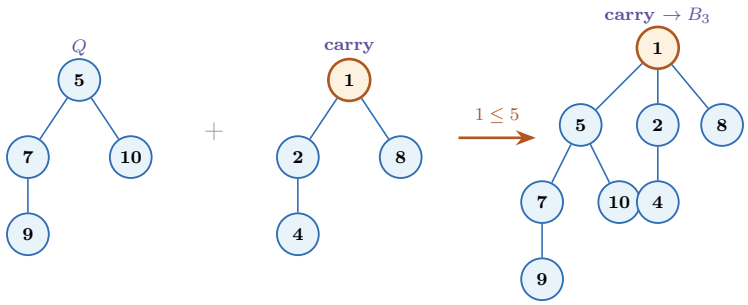


Position 1 (B_1): three B_1's $\Rightarrow$ one into S, merge two



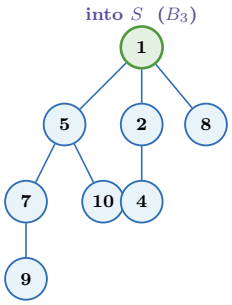
S at pos 1: $B_1(3 \to 6)$.

Position 2 (B_2): two B_2's $\Rightarrow$ merge

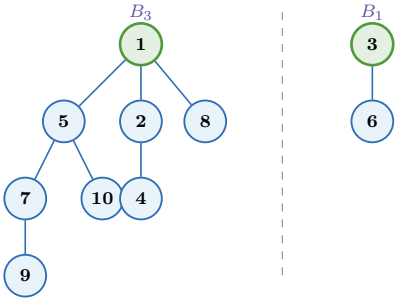


S at pos 2: empty.

Position 3 (B_3): one B_3 (the carry) $\Rightarrow$ drop into S, done



Result: $S = \langle B_3, B_1 \rangle$ $\quad (10 = 1010_2;\quad 3$ comparisons, 3 new edges)



Union Complexity

If $|T| \le n$ and $|Q| \le n$, each heap has $\le \lfloor \log_2 n \rfloor + 1$ trees.
$\Rightarrow$ Union takes $O(\log n)$ comparisons (one per column of the binary addition).

8.2 Insert(T, x): Adding a Single Element

Insert(T, x)

1. Create a new heap Q containing just x (a single B_0).
2. Return Union(T, Q).

Insert = Adding 1 in Binary

If the last bit is 0 (no B_0 exists): place the new B_0. Done in $O(1)$.
If the last bit is 1: merge into B_1 (carry). If B_1 already exists, merge again into B_2. This cascading carry can go up to $O(\log n)$ positions, but on average much less.

Insert Example

Heap $T = \langle B_2, B_0 \rangle$ (5 elements, binary 101). Insert element x.

1. x becomes B_0. Now two B_0's.
2. Merge them $\Rightarrow$ one B_1 (carry). No B_1 exists, so B_1 stays.
3. Result: $\langle B_2, B_1 \rangle$ (6 elements, binary 110). ✓

8.3 Min(T): Finding the Minimum

Min(T)

The minimum element is the root of one of the trees (by min heap property).
Walk through the root list (linked list via HEAD pointer). Compare each root's key. Return the smallest.
At most $\lfloor \log_2 n \rfloor + 1$ roots $\Rightarrow$ $O(\log n)$.

8.4 Extract_Min(T): Remove the Minimum

Extract_Min(T)

1. **Find the min:** walk the root list to find the tree B_k whose root has the smallest key. $O(\log n)$
2. **Remove that tree** from the heap T. $O(1)$
3. **Delete the root** of B_k. By **Lemma 2**, the children form a new binomial heap $Q = \langle B_{k-1}, \ldots, B_0 \rangle$. $O(\log n)$
4. **Union**(T, Q). $O(\log n)$

Total: $O(\log n)$.

Extract_Min Example

Heap: $\langle B_3$(root 1), B_1(root 3)$\rangle$ (10 elements)

1. Min = 1 (root of B_3).
2. Remove B_3 from heap. Remaining: $\langle B_1$(root 3)$\rangle$.
3. Delete root 1 from B_3. Children: B_2, B_1, B_0. New heap $Q = \langle B_2, B_1, B_0 \rangle$ (7 elements).
4. Union($\langle B_1 \rangle$, $\langle B_2, B_1, B_0 \rangle$):
   Position 0: one B_0 $\Rightarrow$ stays.
   Position 1: two B_1's $\Rightarrow$ merge into B_2 (carry).

Position 2: two B_2's $\Rightarrow$ merge into B_3 (carry).

Position 3: one B_3 $\Rightarrow$ stays.

Result: $\langle B_3, B_0 \rangle$ (9 elements, $9 = \langle 1001 \rangle_2$) ✓

Return 1.

9 Complexity Summary

Binomial Heap Operations

Operation	Worst Case	Key Insight
`Union(`T`, `Q`)`	$O(\log n)$	binary addition
`Insert(`T`, `x`)`	$O(\log n)$	Union with single B_0
`Min(`T`)`	$O(\log n)$	scan root list
`Extract_Min(`T`)`	$O(\log n)$	Lemma 2 + Union

10 Practice Problems

Problem 1: Build a Binomial Heap

Build a min binomial heap by inserting: $5, 12, 3, 9, 2, 15, 6, 1$.
(a) How many trees does the final heap have?
(b) What is the root of each tree?
(c) How many total key comparisons?

$n = 8 = \langle 1000 \rangle_2 \Rightarrow F_8 = \langle B_3 \rangle$ (one tree).
Step by step:

- Insert 5: $\langle B_0(5) \rangle$. 0 comps
- Insert 12: merge B_0's: $5 \leq 12$. $\langle B_1(5 \to 12) \rangle$. 1 comp
- Insert 3: $\langle B_1(5), B_0(3) \rangle$. 0 comps
- Insert 9: merge B_0's: $3 \leq 9$. Two B_1's: $3 \leq 5$. $\langle B_2(\text{root } 3) \rangle$. 2 comps
- Insert 2: $\langle B_2(3), B_0(2) \rangle$. 0 comps
- Insert 15: merge B_0's: $2 \leq 15$. $\langle B_2(3), B_1(2 \to 15) \rangle$. 1 comp
- Insert 6: $\langle B_2(3), B_1(2), B_0(6) \rangle$. 0 comps
- Insert 1: merge B_0's: $1 \leq 6$. Two B_1's: $1 \leq 2$. Two B_2's: $1 \leq 3$. $\langle B_3(\text{root } 1) \rangle$. 3 comps

(a) 1 tree. (b) Root is 1. (c) $1 + 0 + 2 + 0 + 1 + 0 + 3 = 7 = n - \alpha(n) = 8 - 1$

Problem 2: Extract_Min

Given: $\langle B_2(\text{root } 3, \text{children: } 7 \to 9 \text{ and } 5),\ B_0(\text{root } 10)\rangle$. Perform Extract_Min.

1. Min $= 3$ (root of B_2). Extract it.
2. Remove B_2. Remaining: $\langle B_0(10)\rangle$.
3. Root 3 had children: B_1(root 7, child 9) and B_0(root 5). New heap $Q = \langle B_1(7 \to 9), B_0(5)\rangle$.
4. Union($\langle B_0(10)\rangle$, $\langle B_1(7), B_0(5)\rangle$):

   Position 0: two B_0's (10, 5). $5 \le 10 \Rightarrow B_1$(root 5, child 10). Carry.

   Position 1: B_1(root 7) + carry B_1(root 5). $5 \le 7 \Rightarrow B_2$(root 5). Carry.

   Position 2: just carry. Done.

   Result: $\langle B_2(\text{root } 5)\rangle$ (4 elements, $4 = \langle 100\rangle_2$) ✓

Return 3.

Problem 3: Properties Quick Check

For B_5:

Nodes?	$2^5 = 32$
Height?	5
Degree of root?	5
Nodes at depth 2?	$\binom{5}{2} = 10$
Nodes at depth 5?	$\binom{5}{5} = 1$

For F_{13} ($13 = \langle 1101\rangle_2$):

Which trees?	$\langle B_3, B_2, B_0\rangle$
How many trees?	$\alpha(13) = 3$
How many edges?	$13 - 3 = 10$

11 Exam Relevance

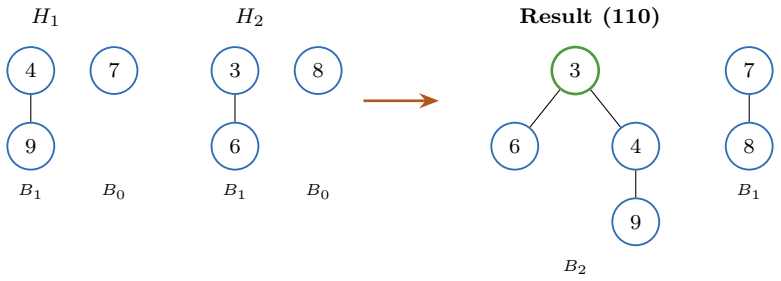
Based on past CSC263 exams (2012 through 2015), no specific binomial heap questions appeared. However, heap questions show up frequently. The kSet ADT problem from Fall 2014 (see Lecture 2 notes) used max heaps and min heaps together.

- "Draw the binomial heap after inserting $x_1, x_2, \ldots$" ⇒ binary addition method
- "Show the result of Extract_Min" ⇒ Lemma 2 + Union
- "How many key comparisons?" ⇒ count new edges = count merges
- "Why $O(\log n)$?" ⇒ at most $\lfloor \log_2 n \rfloor + 1$ trees, binary addition has that many columns

12 Midterm-Style Process Problems

Predicted: Union two binomial heaps (with carries)

H_1 has 3 nodes (binary 011) and H_2 has 3 nodes (binary 011). Union them. Since $3+3=6$ (binary 110), the result is a B_2 plus a B_1. Keys are shown below.



Add the forests like binary numbers, $011 + 011 = 110$, linking two B_k into one B_{k+1} (smaller root wins) whenever a column holds two trees:

1. **Order 0 $(1+1)$:** link $B_0\{7\}$ and $B_0\{8\}$ into a B_1 **carry** $\{7 \to 8\}$. Column 0 is now empty.
2. **Order 1 $(1+1+$ carry $= 11)$:** three B_1 trees now sit here: $\{4 \to 9\}$, $\{3 \to 6\}$, and the carry $\{7 \to 8\}$. Link two of them, $\{3 \to 6\}$ and $\{4 \to 9\}$ (root min $= 3$), into a B_2 **carry**; the third, $\{7 \to 8\}$, stays as the order-1 result bit.
3. **Order 2 (carry only):** the B_2 carry becomes the result.

Result: a B_2 rooted at 3 (children 6 and 4, and 4 keeps its child 9) plus a B_1 $\{7 \to 8\}$. Six nodes, min $= 3$. Each link is one key comparison, so UNION is $O(\log n)$.

Predicted: Extract-Min from a binomial heap

A heap holds one B_2 rooted at 3 with children 6 and $4 \to 9$. Perform EXTRACT-MIN.

1. **Find** the minimum root. Here there is one root, 3, so min $= 3$.
2. **Remove** root 3. Its children become separate binomial trees forming a new heap H': a $B_1\{4 \to 9\}$ and a $B_0\{6\}$.
3. **Union** H' back with the rest of the heap (empty here), giving the final heap $\{4 \to 9\} + \{6\}$, now with min $= 4$.

Finding the min scans $\leq \log n + 1$ roots, splitting is $O(\log n)$, and the union is $O(\log n)$, so EXTRACT-MIN is $O(\log n)$.

Quick Reference: Lecture 3

B_k	two B_{k-1}'s merged; has 2^k nodes, height k, $\binom{k}{d}$ at depth d
F_n	forest of B_k's matching 1 bits of n; has $\alpha(n)$ trees, $n-\alpha(n)$ edges
Min Binomial Heap	F_n + one element per node + min heap ordering
Memory	3 pointers: PARENT, LEFT CHILD, RIGHT SIBLING; HEAD → root list
Lemma 1	merge two $B_k \to B_{k+1}$ with 1 comparison (the "carry")
Lemma 2	delete root of $B_k \to$ valid heap of 2^k-1 elements
Union	binary addition $\to O(\log n)$
Insert	Union with $B_0 \to O(\log n)$
Min	scan root list $\to O(\log n)$
Extract_Min	Lemma 2 + Union $\to O(\log n)$

Lecture 4: Binomial Heaps (Part 2) + Binary Search Trees

1 Remaining Binomial Heap Operations

Recall from Lecture 3: a Min Binomial Heap T is a forest of binomial trees $B_{k_1}, B_{k_2}, \ldots$, each satisfying the min-heap property (parent key $\leq$ children keys). Every node stores three pointers: PARENT, LEFT CHILD, RIGHT SIBLING.

We already have Union, Insert, Min, Extract_Min — all $O(\log n)$. Now we add three more operations that **modify a specific node** we already have a pointer to.

1.1 Decrease_Key(T, x, k): Make a Key Smaller

Goal: change the key of node x to a **smaller** value k (where $k < x$.key).

Why would we want this? Shortest-path algorithms (Dijkstra's) update distances as they find shorter paths. A priority queue needs to let you say "this element is now more urgent."

Decrease_Key(T, x, k)

Precondition: we have a **direct pointer** to node x. No searching needed.
Algorithm:

1. Set x.key $\leftarrow k$
2. Bubble up: while x is not a root **and** x.key $<$ x.parent.key:
   - swap x.key with x.parent.key
   - move x pointer up to the parent
3. Stop when min-heap property is restored (parent's key $\leq x$.key) or x is a root

Why "Bubble Up" Works

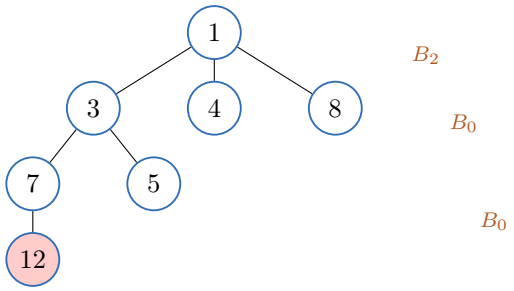
We decreased x's key, so the only possible violation is x.key $<$ x.parent.key (child smaller than parent). Swapping with parent fixes this violation but might create a new one further up. We keep going until we reach a parent whose key is $\leq$ ours, or we hit the root.
The PARENT pointer (from Lecture 3's memory representation) is what makes this efficient — we can walk straight up without searching.

Complexity: $O(\log n)$

At most we bubble up from the deepest node to the root. In a binomial tree B_k, the height is k. The largest tree in a heap of n elements is at most $B_{\lfloor \log_2 n \rfloor}$, so we do at most $\lfloor \log_2 n \rfloor$ swaps.

Worked Example: Decrease_Key

Suppose we have B_3 (8 nodes, height 3) with keys arranged as:



`Decrease_Key`$(T, x, 2)$ where x is the node with key 12:

1. Set x.key $= 2$
2. Compare with parent (key 7): $2 < 7 \Rightarrow$ swap. Now 2 is at depth 2, 7 is at depth 3.
3. Compare with parent (key 3): $2 < 3 \Rightarrow$ swap. Now 2 is at depth 1, 3 is at depth 2.
4. Compare with parent (key 1): $2 > 1 \Rightarrow$ STOP. Min-heap property restored.

Total swaps: 2 (out of max 3 = height of B_3). The tree is still a valid B_3 with min-heap ordering.

1.2 Delete(T, x): Remove a Specific Node

Goal: remove node x from T. Again, we have a direct pointer to x.

The trick: we already know how to remove a *root* (Extract_Min). So force x to become a root first!

Delete(T, x)

1. `Decrease_Key`$(T, x, -\infty)$: set x's key to $-\infty$. Since $-\infty$ is smaller than every other key, x bubbles **all the way up** to become the root of its binomial tree.
2. `Extract_Min`(T): the minimum element is now x (with key $-\infty$), so Extract_Min removes it. Internally this uses Lemma 2 (children of x's tree form a new heap) + Union.

Complexity: $O(\log n)$ (Decrease_Key $O(\log n)$ + Extract_Min $O(\log n)$)

1.3 Increase_Key(T, x, k): Make a Key Larger

Goal: change the key of node x to a **larger** value k (where $k > x$.key).

When we *decrease* a key, the only violation is with the parent (one node above), and we bubble up along a single path.
When we *increase* a key, the violation is with the **children**. In a binomial tree, a node in B_k can have up to k children. To bubble down:

1. We need to find the **minimum-key child** among all children (to maintain min-heap property after swap).
2. A node in B_k can have up to k children $\Rightarrow$ scanning children costs $O(k)$ per level.
3. There are up to k levels to descend.
4. Total: $O(k^2) = O(\log^2 n)$. That is **worse** than $O(\log n)$!

Better approach — Delete + re-Insert:

Increase_Key(T, x, k)

1. `Delete`(T, x): remove x entirely. $O(\log n)$
2. Create a new node x' with key k.
3. `Insert`(T, x'): add x' back via Union. $O(\log n)$

Complexity: $O(\log n)$ — beats the naive bubble-down approach of $O(\log^2 n)$.

Worked Example: Increase_Key

Suppose T contains node x with key 3. We want `Increase_Key`$(T, x, 15)$.
Step 1: Delete(T, x)

- `Decrease_Key`$(T, x, -\infty)$: bubble x (now key $-\infty$) to root of its binomial tree
- `Extract_Min`(T): removes x. Children form a new heap (Lemma 2), Union it back.

Step 2: Create node x' with key 15.
Step 3: `Insert`(T, x') = `Union`$(T, \{x'\})$. The new node 15 enters as a B_0.
Total: $O(\log n) + O(\log n) = O(\log n)$.

1.4 Complete Binomial Heap Operations Summary

All Binomial Heap Operations — Final Table

Operation	Method	Cost
Union(T, Q)	binary addition (Lemma 1 = carry)	$O(\log n)$
Insert(T, x)	Union$(T, \{x\})$	$O(\log n)$
Min(T)	scan root list	$O(\log n)$
Extract_Min(T)	remove min root, Lemma 2 + Union	$O(\log n)$
Decrease_Key(T, x, k)	set key, bubble up via PARENT	$O(\log n)$
Delete(T, x)	Decrease_Key$(-\infty)$ + Extract_Min	$O(\log n)$
Increase_Key(T, x, k)	Delete + Insert (not bubble-down!)	$O(\log n)$

2 Cost of k Successive Inserts

We now ask a more refined question: if we insert k elements into a heap T one by one, how many **total** key comparisons does it take?

Setup: Start with binomial heap T where $|T| = n$. Insert k elements one by one.

Naive analysis: Each insert costs at most $O(\log(n + k))$. So k inserts $\leq O(k \log(n + k))$. But this is a loose overestimate — most inserts are cheap!

2.1 Key Insight: Each Insert = Adding 1 in Binary

Recall from Lecture 3: `Insert`(T, x) = `Union`$(T, \{x\})$, which works exactly like adding 1 in binary. The "carries" in binary addition correspond to tree merges (Lemma 1):

- No B_0 in T? (rightmost bit is 0) Just place x as B_0. 0 carries.
- B_0 exists? (rightmost bit is 1) Merge two B_0's $\to B_1$. That's 1 carry.
- B_1 already exists? Merge two B_1's $\to B_2$. Another carry.
- Keep carrying until we hit a 0-bit (an empty slot for that tree size).

Carries = Cost

Each carry = one key comparison (to decide which root goes on top when merging via Lemma 1). So the total cost of an insert = the number of carries it produces.

2.2 Worked Example: 5 Successive Inserts into $|T| = 27$

5 Successive Inserts into $|T| = 27$

$27 = \langle 11011 \rangle_2$ means T currently has trees B_4, B_3, B_1, B_0.
Watch what happens to the binary representation as we add 1 repeatedly:

Insert #	Binary before	Binary after	Carries	What happens
1	**11011**	**11100**	2	bit 0: $1+1 \to$ carry; bit 1: $1+\text{carry} \to$ carry; bit
2	11100	11101	0	bit 0 was 0, just place B_0
3	11101	11110	1	bit 0: $1+1 \to$ carry; bit 1: $0+\text{carry} \to 1$
4	11110	11111	0	bit 0 was 0, just place B_0
5	11111	100000	5	all 1's $\to$ carry ripples through all 5 positions
		Total:	**8**	

Naive bound: $5 \times \lfloor \log_2 32 \rfloor = 5 \times 5 = 25$. Actual: only **8**!
Notice: Insert #5 is expensive (5 carries) but inserts #2 and #4 are free. The expensive ones "pay" for all the 0-bits they create, making future inserts cheap.

2.3 Exact Formula via Edge Counting

We can derive an **exact** formula for the total cost using a clever counting argument.

Edge Counting Argument

Fact (from Lecture 3): a binomial heap with m elements has exactly $m - \alpha(m)$ edges, where $\alpha(m)$ = number of 1-bits in binary representation of m.
Setup:

- Before k inserts: n elements $\Rightarrow n - \alpha(n)$ edges
- After k inserts: $n+k$ elements $\Rightarrow (n+k) - \alpha(n+k)$ edges
- **New edges created** $= \big[(n+k) - \alpha(n+k)\big] - \big[n - \alpha(n)\big] = k + \alpha(n) - \alpha(n+k)$

Where do new edges come from? Each of the k inserted nodes gets connected to the tree (contributes 1 edge just by being placed). Each **carry** (Lemma 1 merge) creates exactly 1 additional edge (the new link between two merged roots). So:

$$\text{new edges} = \underbrace{k}_{\text{placed nodes}} + \underbrace{\text{total carries}}_{\text{merges}}$$

Wait — that gives: total carries $= k + \alpha(n) - \alpha(n+k) - k = \alpha(n) - \alpha(n+k)$.
But the **total cost** includes both the "place the node" step and the carries:

$$\boxed{\text{Total cost} = k + \alpha(n) - \alpha(n+k)}$$

Verify with the example: $n = 27$, $k = 5$. $\alpha(27) = \alpha(\langle 11011 \rangle_2) = 4$. $\alpha(32) = \alpha(\langle 100000 \rangle_2) = 1$.
Total cost $= 5 + 4 - 1 = 8$. ✓ (matches our count above!)

Tight Bound: Total cost $\leq 2k$ when $k > \log_2 n$

Since $\alpha(n) \leq \lfloor \log_2 n \rfloor + 1$ (at most $\lfloor \log_2 n \rfloor + 1$ bits) and $\alpha(n+k) \geq 1$ (at least one 1-bit):

$$\text{Total cost} = k + \alpha(n) - \alpha(n+k) \leq k + \lfloor \log_2 n \rfloor + 1 - 1 = k + \lfloor \log_2 n \rfloor$$

When $k > \log_2 n$: total cost $< k + k = 2k$.

Average cost per insert $= \dfrac{\text{total cost}}{k} < \dfrac{2k}{k} =$ **2 key comparisons**
Even though a single insert can cost up to $O(\log n)$, on average each insert costs less than 2!

Practice Problem: Cost of Successive Inserts

Q: Start with $|T| = 13 = \langle 1101 \rangle_2$. Perform 3 successive inserts. What is the total cost?

Solution using binary addition:

Insert #	Before	After	Carries
1	1101	1110	1
2	1110	1111	0
3	1111	10000	4
		Total:	**5**

Verify with formula: cost $= k + \alpha(n) - \alpha(n+k) = 3 + \alpha(13) - \alpha(16) = 3 + 3 - 1 = 5$. ✓
Verify with edge counting: $13 - 3 = 10$ edges before. $16 - 1 = 15$ edges after. New edges $= 5$. ✓

— **New Topic** —

We now shift from priority queues to a different ADT. Priority queues are great when you always want the min (or max), but what if you want to **search for a specific key**? That requires a **Dictionary**.

3 The Dictionary ADT

Dictionary ADT

Object: A set S of elements, each identified by a key.
Operations:

- **Search**(S, k): find and return the element with key k (or report it doesn't exist)
- **Insert**(S, x): add element x to S
- **Delete**(S, x): remove element x from S (given a pointer to x)

Dictionaries are everywhere: symbol tables in compilers, database indices, contact lists, DNS lookups. Any time you need to look something up by a key.

3.1 First Attempt: Linked List

Insert	$\Theta(1)$	prepend to head
Search	$\Theta(n)$	must scan entire list (key could be anywhere)
Delete	$\Theta(n)$	need to search first, then remove

Search takes $\Theta(n)$ because a linked list has **no ordering** — there is no way to skip ahead. We need a structure that lets us **eliminate half the candidates** at each step.

Goal

A data structure where *all three* dictionary operations run in $O(\log n)$ time. The **Binary Search Tree** is our first attempt at this.

4 Binary Search Trees (BST)

4.1 The BST Property

The idea: store elements in a binary tree, arranged so that at each node, we know which direction to go. This is the **binary search** idea applied to a tree structure.

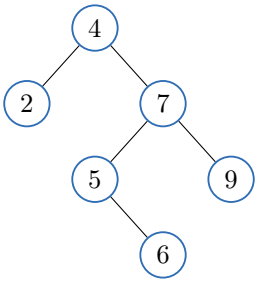
For **every** node x in the tree:

$$\text{keys in left subtree of } x \;\leq\; x.\text{key} \;\leq\; \text{keys in right subtree of } x$$

Note: Keys are **NOT necessarily unique**. The $\leq$ (not $<$) is critical — equal keys can go

either way.

Example: Valid BST with $S = \{2, 4, 5, 6, 7, 9\}$



Check the property at each node:

- **Node 4:** left subtree $\{2\}$ — all ≤ 4 ✓. Right subtree $\{5, 6, 7, 9\}$ — all ≥ 4 ✓.
- **Node 7:** left subtree $\{5, 6\}$ — all ≤ 7 ✓. Right subtree $\{9\}$ — $9 \geq 7$ ✓.
- **Node 5:** no left child. Right subtree $\{6\}$ — $6 \geq 5$ ✓.
- Leaves 2, 6, 9: no children, trivially satisfied.

The *shape* of the tree depends on the **insertion order**. The same set $S = \{2, 4, 5, 6, 7, 9\}$ can produce many different valid BSTs. This non-uniqueness is what leads to the worst-case problem we'll see later.

4.2 In-Order Traversal

A key consequence of the BST property: visiting nodes in **left-root-right** order gives all keys in ascending sorted order.

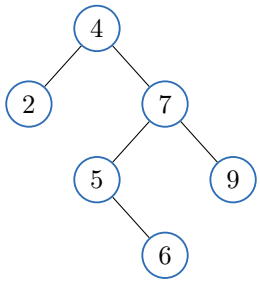
InOrderTraversal(x)

```
InOrderTraversal(x):
    if x != NIL:
        InOrderTraversal(x.left)          // recurse into left subtree
        visit(x)                          // process current node
        InOrderTraversal(x.right)         // recurse into right subtree
```

Why does this produce sorted order? The BST property guarantees: everything in the left subtree $\leq$ current node $\leq$ everything in the right subtree. Recursion applies this at every level.

Worked Example: In-Order Traversal

Using the BST above (root = 4):



Trace: go left from 4 → visit **2** → back up, visit **4** → go right → go left from 7 → visit **5** → go right from 5 → visit **6** → back up, visit **7** → go right from 7 → visit **9**
Result: $2, 4, 5, 6, 7, 9$ ✓ sorted!

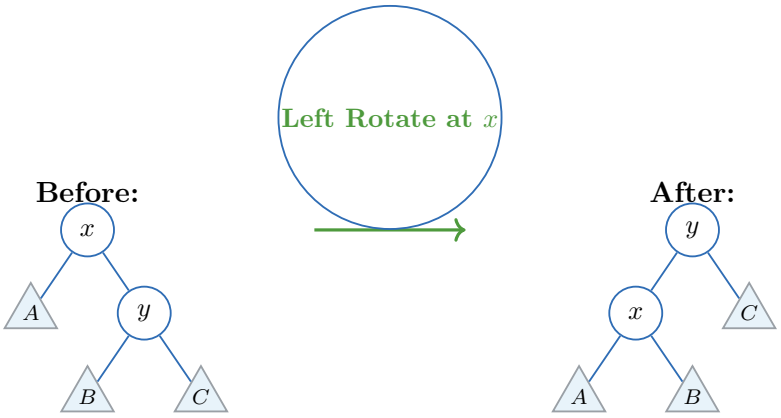
4.3 Left Rotation

A rotation restructures a BST without changing which keys it stores or violating the BST property. Rotations are the fundamental tool for **rebalancing** — we'll use them heavily when we get to AVL trees next lecture.

Left Rotation at node x

Let $y = x$.right (must exist). Left rotation at x:

1. y takes x's position (becomes new subtree root)
2. x becomes y's left child
3. y's old left subtree (B) becomes x's new right subtree

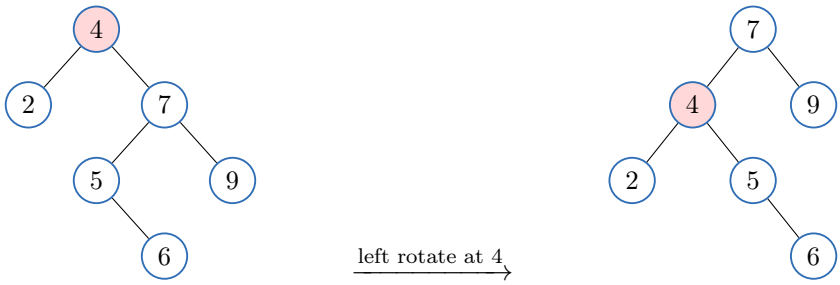


Why BST Property Is Preserved

Before the rotation, we know: keys in $A \leq x \leq$ keys in $B \leq y \leq$ keys in C.
After the rotation, B moves from y's left to x's right. But B's keys are between x and y, so B being x's right subtree ($\geq x$) and in y's left subtree ($\leq y$) is perfectly valid. Nothing else changes.
Cost: $O(1)$ — just rearranging 3 pointers.

Worked Example: Left Rotation at Node 4

$S = \{2, 4, 5, 6, 7, 9\}$, tree rooted at 4. Left rotate at node 4:
Before: root = 4, right child = 7, 7's left subtree = {5, 6}



7 becomes the new root. 4 becomes 7's left child. {5, 6} (was 7's left subtree) becomes 4's right subtree.
Verify: In-order traversal of the result: 2, 4, 5, 6, 7, 9. Same as before. BST property preserved. ✓

4.4 BST Operations

All BST operations follow one path from root toward a leaf, making comparisons at each node to decide left or right.

Search(T, k): Find a Key

Search(T, k)

```
x = T.root
while x != NIL and x.key != k:
    if k < x.key:
        x = x.left                    // k must be in left subtree
    else:
        x = x.right                   // k must be in right subtree
return x                              // NIL if not found
```

Why this works: The BST property tells us that if $k < x$.key, then k cannot be in the right subtree (all keys there are $\geq x$.key $> k$). So we can safely ignore the entire right subtree and go left. Symmetric for $k > x$.key. This is the same idea as binary search on a sorted array.

Complexity: $O(h)$ where h = height of tree (we visit at most one node per level).

Worked Example: Search$(T, 5)$

Tree rooted at 4 → {2, 7 → {5 → {∅, 6}, 9}}:

1. At root 4: $5 > 4 \Rightarrow$ go right to 7
2. At 7: $5 < 7 \Rightarrow$ go left to 5
3. At 5: $5 = 5 \Rightarrow$ Found! Return this node.

3 comparisons, 3 nodes visited. Height of tree = 3, so this is within $O(h)$. ✓

Insert(T, z): Add a New Node

Insert(T, z)

```
Follow the search path for z.key (comparing at each node)
When you reach a NIL position, place z there as a new leaf
```

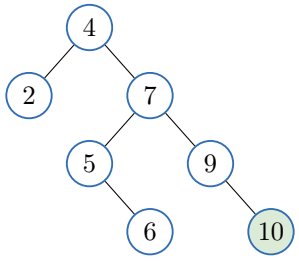
Why this works: The search path tells us exactly where z **should** be to maintain the BST property. The NIL position at the end is the only valid place for z's key. Since z becomes a leaf, it doesn't disturb any existing nodes.

Complexity: $O(h)$

Worked Example: Insert$(T, 10)$

Using the same tree. Search for where 10 belongs:

1. At root 4: $10 > 4 \Rightarrow$ go right
2. At 7: $10 > 7 \Rightarrow$ go right
3. At 9: $10 > 9 \Rightarrow$ go right — but right child of 9 is NIL!
4. Insert 10 as right child of 9.



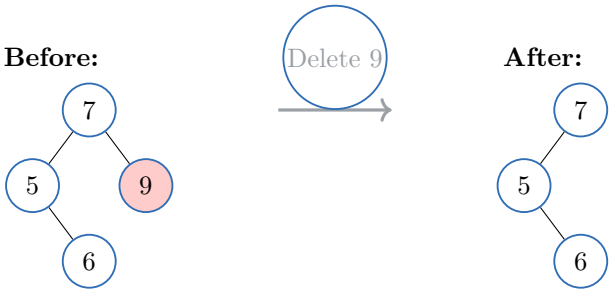
4.5 BST Delete(T, x): Three Cases

Deletion is the most complex BST operation. We have a pointer to node x and need to remove it while maintaining the BST property.

Removing a node from the middle of a tree can disconnect its children. We need to "repair" the tree so every node still has a parent and the BST property still holds.

Case 1: x is a leaf (no children)

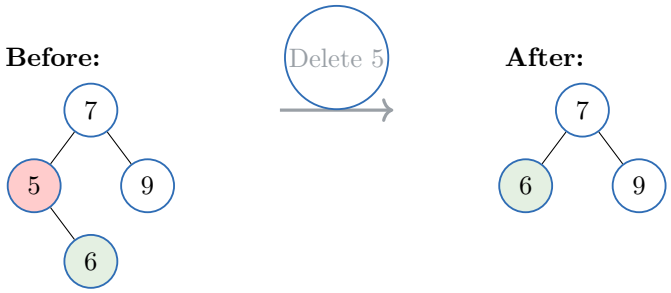
Simplest case. Just remove x — nothing to reconnect.



Case 2: x has exactly one child

Replace x with its only child (and the child's entire subtree). The child "moves up" to take x's position.

Why this is safe: if x had key k and its child's subtree contains keys that satisfied BST property relative to x, those keys also satisfy BST property relative to x's parent (they were already in the correct subtree of the parent).



Node 6 (the only child of 5) replaces 5. BST property: $6 \leq 7$ ✓.

Case 3: x has two children (hardest case)

We can't just remove x — both subtrees need a parent. The solution: replace x's key with its successor's key, then delete the successor (which is easier to remove).

The Successor

The successor of node x = the node with the **smallest key that is larger than** x.key. In a BST, the successor is in x's right subtree (since right subtree has all keys $\geq x$.key). To find it: go to x's **right child**, then follow **left child pointers all the way down** until you hit NIL. That leftmost node in the right subtree is the successor.

Why the Successor Has At Most One Child

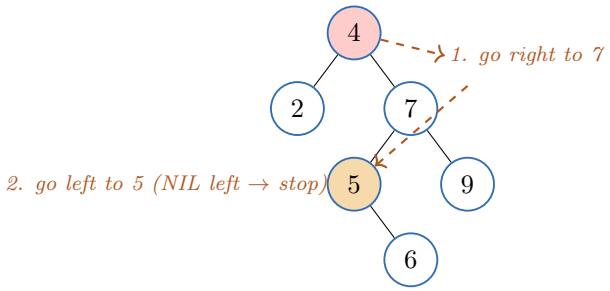
The successor is the **leftmost** node in some subtree. By definition, it has **no left child** (otherwise there would be a smaller key, contradicting "leftmost"). So the successor has at most one child (a right child). This means deleting the successor is either Case 1 or Case 2! This is the clever part: Case 3 **reduces to** Case 1 or 2.

1. **Find** the successor of x: go right once, then left all the way
2. **Copy** the successor's key into x's position (replacing x's key)
3. **Delete** the original successor node (Case 1 or 2)

Worked Example: Delete$(T, 4)$ — Two Children Case (Step by Step)

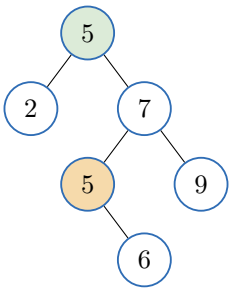
Tree: $4 \to \{2,\ 7 \to \{5 \to \{\emptyset, 6\},\ 9\}\}$
Sorted order of S: $2, \mathbf{4}, 5, 6, 7, 9$. The successor of 4 is 5.

Step 1: Find successor of 4

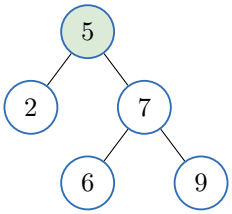


Go right to 7, then left to 5. Left child of 5 is NIL ⇒ successor = 5

Step 2: Copy successor's key into 4's position
Replace 4 with 5. Now there are temporarily two nodes with key 5.



Step 3: Delete original successor (the highlighted 5)
The successor node 5 has one child (6) ⇒ Case 2. Replace the successor 5 with its child 6:



Final result: $5 \to \{2,\ 7 \to \{6, 9\}\}$
Verify in-order: $2, 5, 6, 7, 9$ ✓ (4 removed, BST property intact)

Practice Problem: BST Delete

Q: Given BST: $8 \to \{3 \to \{1, 6 \to \{4, 7\}\},\ 10 \to \{\emptyset, 14 \to \{13, \emptyset\}\}\}$
Delete node 3 (two children).

Solution:

1. Successor of 3: go right to 6, then all-way left to 4. Successor = 4.
2. Copy 4 into 3's position.
3. Delete original 4: it is a leaf (Case 1), just remove it.
4. Result: $8 \to \{4 \to \{1, 6 \to \{\emptyset, 7\}\},\ 10 \to \{\emptyset, 14 \to \{13, \emptyset\}\}\}$

In-order: $1, 4, 6, 7, 8, 10, 13, 14$ ✓

BST Delete: Summary of All 3 Cases

Case	Condition	Action
1	x is a leaf (0 children)	Remove x
2	x has one child	Replace x with that child
3	x has two children	Copy successor's key into x, delete successor (Case 1 or 2)

All cases: $O(h)$. Finding successor walks down at most h nodes. Removing the successor is $O(1)$.

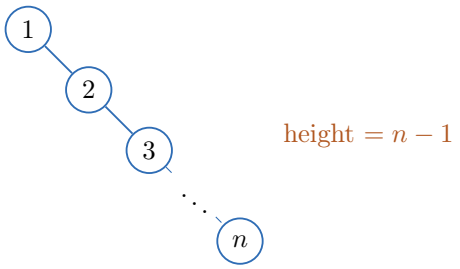
4.6 The Problem: Worst-Case Height

All BST operations run in $O(h)$ where h = height of the tree. So the real question is: **how tall can a BST get?**

Best case: A perfectly balanced BST with n nodes has height $\lfloor \log_2 n \rfloor$. All operations are $O(\log n)$.

Worst case: Consider inserting $1, 2, 3, \ldots, n$ **in sorted order**:

- Insert 1: root = 1
- Insert 2: $2 > 1$, goes right. Now: $1 \to 2$
- Insert 3: $3 > 1$, right. $3 > 2$, right. Now: $1 \to 2 \to 3$
- $\vdots$
- Insert n: goes right at every step. Chain: $1 \to 2 \to 3 \to \cdots \to n$



This is a degenerate chain — every node has exactly one child. It is a valid BST but it looks like a linked list!

Height $= n - 1$ edges. So all operations become $\Theta(n)$:

	Best-case BST	Worst-case BST
Search	$O(\log n)$	$\Theta(n)$
Insert	$O(\log n)$	$\Theta(n)$
Delete	$O(\log n)$	$\Theta(n)$

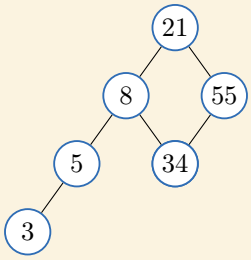
A plain BST gives us no guarantee on height. If the input happens to be sorted (or nearly sorted), we degrade to a linked list. We need a BST variant that **automatically rebalances** to keep the height small.

Next lecture: the AVL tree — a self-balancing BST that guarantees height $\leq O(\log n)$, making all dictionary operations $O(\log n)$ in the **worst case**. It uses **rotations** (like the left rotation we just learned) to rebalance after every insert and delete.

5 Past Exam Questions

★ **Seen on a past exam** (Fall 2014 Midterm, Q3b)

[4 marks] Draw the result when **55 is deleted** from the AVL tree below. Show your work; in particular, identify clearly any **rotation** performed.



Step 1: BST-Delete 55. Node 55 has one child (34) ⇒ Case 2: replace 55 with 34.
Result: $21 \to \{8 \to \{5 \to \{3, \emptyset\}, 13\}, 34\}$
Step 2: Check AVL balance. Node 21 has left subtree height 2, right subtree height 0. Balance factor $= |2 - 0| = 2 > 1$. Unbalanced!
Step 3: Rotate. Right-rotate at 21 (with 8 as pivot).
Final result: $8 \to \{5 \to \{3, \emptyset\}, 21 \to \{13, 34\}\}$

Key concepts tested: BST Delete (Case 2) + AVL balance check + Rotation.
(AVL trees are covered in Lecture 5. This exam question is included to show what's coming and how BST deletion connects to AVL rebalancing.)

★ **Seen on a past exam** (Fall 2014 Midterm, Q2)

[10 marks] Consider the "k-Set" ADT:
Objects: Collection C of rational numbers (duplicates allowed).
Operations:

- `Insert`(C, x): add x to C
- `k-Select`(C): return element at rank k in C, or NIL if $|C| < k$

Implement using data structures from class. `k-Select` must run in $O(1)$ worst case.

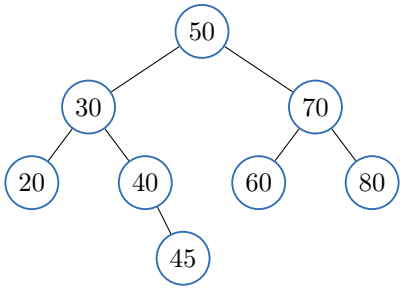
Solution: Store k smallest elements in a max-heap H_1, rest in min-heap H_2.
Idea: the k-th smallest element = the largest among the k smallest = root of a max-heap containing exactly the k smallest.
`k-Select`(C): if $|H_1| < k$: return NIL; else return $H_1[1]$ (root of max-heap). $O(1)$.
`Insert`(C, x): if $x < H_1[1]$ or $|H_1| < k$: insert into H_1, else insert into H_2. If $|H_1| > k$: extract max from H_1, insert into H_2. Cost: $O(\log n)$.

Key concepts tested: Heap operations, ADT implementation, runtime analysis

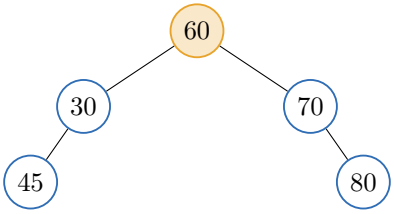
6 Midterm-Style Process Problems

Predicted: BST Delete, all three cases on one tree

Starting from this BST, delete 20, then 40, then 50, in that order. Name which case each is.



Delete 20 (Case 1: leaf). No children, so just remove it. Node 30 now has only the right child 40.
Delete 40 (Case 2: one child). Node 40 has a single child 45. Splice 40 out by linking its parent 30 straight to 45.
Delete 50 (Case 3: two children). Find the **successor** = min of the right subtree: go right to 70, then all the way left to **60**. Copy 60 into the root node, then delete the old successor 60 (a leaf, Case 1).



Final tree above. Each delete walks one root-to-leaf path, so BST Delete is $O(h)$.

Predicted: cost of k successive inserts

A min binomial heap currently holds $n = 13$ elements. Using cost $= k + \alpha(n) - \alpha(n + k)$ (where $\alpha(m)$ is the number of 1-bits in m), find the key comparisons for $k = 5$ inserts and the average per insert.

$n = 13 = 1101_2$, so $\alpha(13) = 3$. $n + k = 18 = 10010_2$, so $\alpha(18) = 2$.

$$\text{cost} = k + \alpha(n) - \alpha(n+k) = 5 + 3 - 2 = 6 \text{ comparisons.}$$

Average per insert $= 6/5 = 1.2$. This matches the rule that once $k > \log_2 n$ the average cost per insert is ≤ 2, i.e. inserts are **amortized** $O(1)$ even though a single insert can cost up to $\log n$.

Quick Reference: Lecture 4

Binomial Heaps (Part 2)

Decrease_Key	set key, bubble up via PARENT pointer $\to O(\log n)$
Delete	Decrease_Key$(-\infty)$ + Extract_Min $\to O(\log n)$
Increase_Key	Delete + Insert (not bubble-down: that's $O(\log^2 n)$) $\to O(\log n)$
k inserts total cost	$k + \alpha(n) - \alpha(n+k)$; $\leq 2k$ when $k > \log_2 n$
Average insert cost	< 2 key comparisons (when $k > \log_2 n$)

Dictionary ADT + BST

Dictionary ADT	Search + Insert + Delete on keyed elements
Linked list baseline	Insert $\Theta(1)$, Search/Delete $\Theta(n)$
BST property	left subtree keys $\leq$ node key $\leq$ right subtree keys
In-order traversal	left $\to$ visit $\to$ right = sorted order
Left rotation	x goes down, x.right goes up; preserves BST; $O(1)$
BST Search/Insert	follow comparisons down the tree; $O(h)$
BST Delete	3 cases: leaf / 1 child / 2 children (via successor); $O(h)$
Worst-case BST height	$n - 1$ (degenerate chain) $\Rightarrow$ all ops $\Theta(n)$
Next: AVL trees	guarantee height $O(\log n)$ via rotations

Lecture 5: Balanced BSTs and AVL Trees

1 Recall: the Problem with BSTs

A plain BST implements the **Dictionary ADT** (Search, Insert, Delete), each in $\Theta(h)$ where h is the height. The catch: if we insert keys in sorted order $1, 2, 3, \ldots, n$, the tree becomes a degenerate chain with height $\Theta(n)$, so every operation is $\Theta(n)$.

Goal

Keep the tree **balanced** so its height stays $\Theta(\log n)$. Then Search, Insert, Delete are all $\Theta(\log n)$.

Balanced BST data structures for the Dictionary ADT: 2-3 trees, Red-Black trees, B-Trees, and AVL trees (this lecture).

2 Height and Balance Factor

2.1 Height of a Node

Height

height(v) = number of edges on the longest path from v down to a leaf.

- height(v) $= 0$ if v is a **leaf** (a single node).
- height(empty tree) $= -1$.

height(T) = height of the root node of T.

The -1 for the empty tree is a convention that makes the recurrence height(v) $= 1+\max($height(v.left), height(v.ri
work even when a child is missing.

2.2 Balance Factor

Balance Factor of a node v

$$\text{BF}(v) \;=\; \text{height(right subtree of } v) \;-\; \text{height(left subtree of } v) \;=\; h_R - h_L$$

The bigger $|\text{BF}(v)|$, the more imbalanced the node. We want every BF(v) as close to 0 as possible.

Reading the Balance Factor

$\text{BF}(v) = +1$ v is **right-heavy** $(h_R = h_L + 1)$
$\text{BF}(v) = \;\;0$ v is **balanced** $(h_R = h_L)$
$\text{BF}(v) = -1$ v is **left-heavy** $(h_L = h_R + 1)$

3 AVL Trees

3.1 Definition (Adelson-Velsky and Landis)

An **AVL tree** T is a BST where **every** node $v \in T$ satisfies

$$-1 \leq \mathrm{BF}(v) \leq +1$$

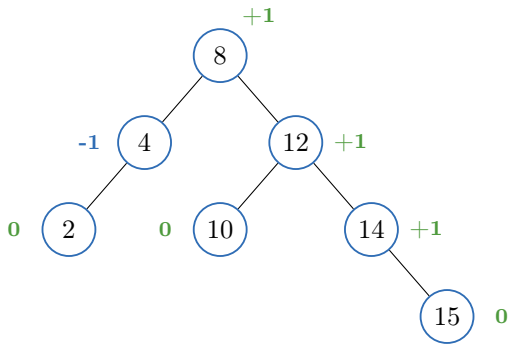
Every node is left-heavy, balanced, or right-heavy. **No node may reach** ± 2**.**

3.2 Is This an AVL Tree?

To check: compute $\mathrm{BF}(v)$ at *every* node. It is an AVL tree **iff** every $|\mathrm{BF}(v)| \leq 1$. A single node with $|\mathrm{BF}| \geq 2$ breaks it.

One tree, balance factor on every node

Compute $h_R - h_L$ at each node (heights shown are of the subtree rooted there).



Every $|\mathrm{BF}| \leq 1$, so this **is** an AVL tree. ✓

If any node reaches $\mathrm{BF} = +2$ or $\mathrm{BF} = -2$, the tree is not AVL. Example: a node whose left subtree has height 0 and right subtree has height 2 gives $\mathrm{BF} = 2 - 0 = +2$. Not AVL.

3.3 Properties

Why AVL trees are good

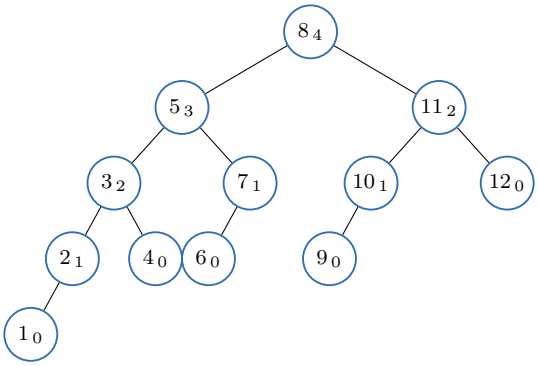
- An AVL tree of n nodes has height $\Theta(\log n)$ $\quad\big[\text{precisely, height} \leq 1.44 \log_2(n+2)\big]$.
- Insert and Delete can maintain the balance in $\Theta(\log n)$ time.
- Elegant, relatively clean and simple, works well in practice.

So Search, Insert, Delete are all $\Theta(\log n)$ worst case.

★ **Seen on a past exam** (Fall 2014 Midterm, Q3a)

[4 marks] Draw an AVL tree containing $\{1, 2, \ldots, 12\}$, but make it **as unbalanced as possible**. Next to each node, indicate the **height** of the subtree rooted at that node.

"As unbalanced as possible" = the **minimum** number of nodes are used to reach the height, so every node sits at its balance limit (BF = ± 1). One valid answer (height shown as a subscript):



Key idea: The subtree of height 4 (root 8) uses one child of height 3 and, by the AVL rule, the other child may be as short as height 2. That is the fewest nodes possible, which is why it looks so lopsided while still being AVL.

★ **Seen on a past exam** (Fall 2014 Midterm, Q3c)

[4 marks] Let $m(h)$ = the **minimum** number of nodes in any AVL tree of height h. Give a recurrence for $m(h)$ with base cases, and explain it briefly.

$$m(0) = 1, \qquad m(1) = 2, \qquad m(h) = 1 + m(h-1) + m(h-2) \text{ for } h \geq 2.$$

Why: height 0 needs 1 node; height 1 needs a root with one child (2 nodes). For height h, the root (1 node) needs one child of height $h-1$; by the AVL condition the *other* child can be as short as height $h-2$. Making both children minimal gives $m(h-1)+m(h-2)$ below the root.

Consequence: $m(h)$ grows like the Fibonacci numbers (exponentially in h), which forces $h = \Theta(\log n)$. This is the proof that AVL height is logarithmic.

4 AVL Operations

The three operations

Search(T, x)	identical to BST search $\to O(\log n)$ (height is $O(\log n)$)
Insert(T, x)	BST insert + rebalance $\to O(\log n)$
Delete(T, x)	BST delete + rebalance $\to O(\log n)$

Search is unchanged from a plain BST, so the only new work is keeping the tree balanced after Insert (and Delete). This lecture covers **Insert**.

5 Insert

5.1 General Idea

Insert(T, x): the plan

1. Insert x into T as in any BST. Now x is a **leaf**; set BF$(x) = 0$.
2. Walk **from x up to the root**. Only the balance factors of nodes *on this path* can change.
3. At each node v on the path, **adjust** BF(v). If some v becomes ± 2, **rebalance** at v with a rotation.

Why only the path changes: inserting a leaf only lengthens paths that pass through x's ancestors. No other node's subtree heights move.

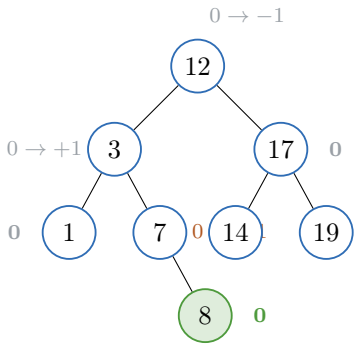
The stopping rule

Going up, at each ancestor v:

- If the subtree containing x got taller, BF(v) moves by ± 1.
- **If BF(v) becomes** 0, the subtree at v did *not* grow taller overall $\Rightarrow$ stop. Nothing above v can change.
- If BF(v) becomes ± 2, rotate to fix it (then the height is restored, so we also stop).

5.2 Worked Example 1: No Rebalancing

Start from this AVL tree of $\{1, 3, 7, 12, 14, 17, 19\}$ (all BF $= 0$), and **Insert 8**.



Trace: Insert 8

1. BST-insert 8: it lands as the **right child of 7**. Set BF$(8) = 0$.
2. Go up to 7: 8 is in 7's right subtree $\Rightarrow$ BF$(7) : 0 \to +1$. Still legal, and **the height of the subtree at 7 did not change** (it was 1 before, it is still 1). So we stop at 7.

Because the height of 7's subtree is unchanged, no ancestor above it is affected (3 and 12 keep their old balance factors, shown greyed). No rebalancing needed.

After inserting 8, node 3 would only change if 7's subtree grew. It did not, so we never even reach 3: we stop the moment a node keeps its height. This is the whole point of the stopping rule.

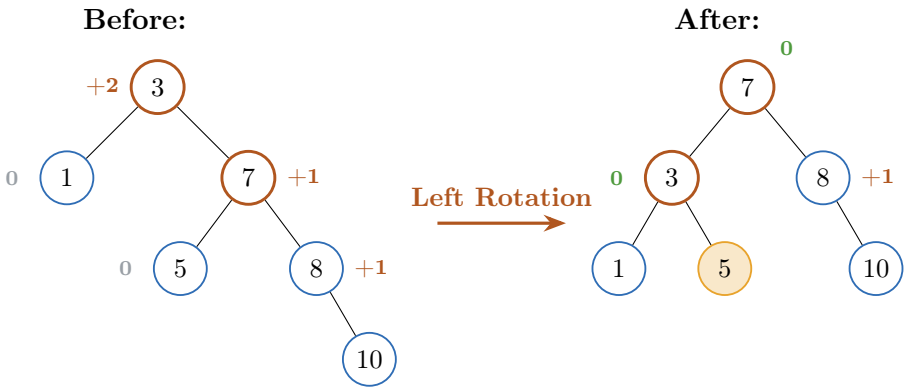
5.3 Worked Example 2: Single Rotation

Now the tree is the AVL of $\{1, 3, 5, 7, 8, 12, 14, 17, 19\}$. **Insert 10.**

10 BST-inserts as the **right child of 8**. Walking up: BF(8) : $0 \to +1$, then BF(7) : $0 \to +1$, then BF(3) : $+1 \to +2$. Node 3 is now unbalanced (right-heavy), and its right child 7 is also right-heavy (+1).

This is the "right-right" (RR) case ⇒ single Left Rotation

$v = 3$ has BF(v) $= +2$ **and** BF(v.right) $= +1$. Fix with one **Left Rotation** at v.



Reading the rotation

Left-rotating at 3: its right child 7 becomes the new subtree root, and 3 becomes 7's **left** child.

- **Where does 5 go?** 5 was 7's *left* subtree. In a left rotation, 7's old left subtree becomes 3's new **right** subtree. So 5 hangs under 3, not under 8, because $3 < 5 < 7$, so 5 belongs on 3's right. (BST property preserved.)
- After: BF(7) $= 0$, so 7's subtree is balanced again.

Bonus: nothing propagates upward

The subtree rooted at 3 had height 2 *before* inserting 10; after the rotation the subtree rooted at 7 has height 2 again. **Same height as before the insert**, so the balance factor of 12 (the parent) is unchanged and we stop. One rotation fixes everything.

5.4 Worked Example 3: Double Rotation

Tree is the AVL of $\{1, 3, 5, 7, 8, 12, 14, 17, 19\}$. **Insert 6.**

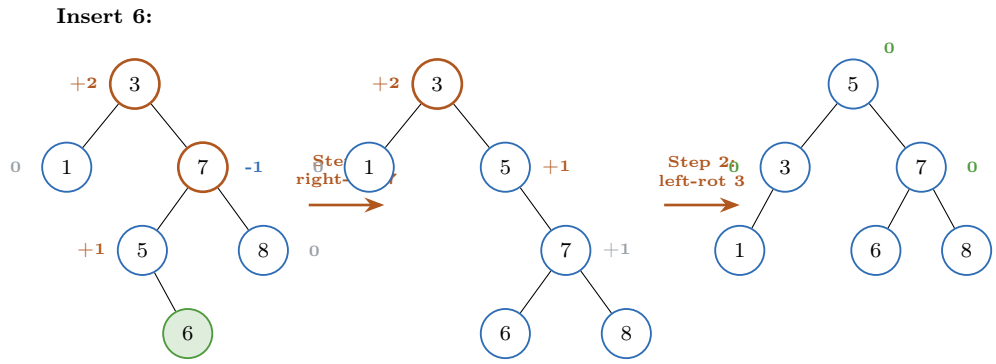
6 BST-inserts as the **right child of 5**. Walking up: BF(5) : $0 \to +1$, BF(7) : $0 \to -1$, BF(3) : $+1 \to +2$. Now $v = 3$ has BF $= +2$, but its right child 7 is left-heavy (-1): the "bend" zig-zags.

Because the heavy grandchild is on the *left* of the right child (a right-then-left zig-zag), one left rotation at 3 leaves the tree unbalanced. We need **two** rotations.

This is the "right-left" (RL) case ⇒ Double Right-Left Rotation

$v = 3$ has $\mathrm{BF}(v) = +2$ **and** $\mathrm{BF}(v.\mathrm{right}) = -1$.

1. **Right-rotate** the right subtree of v (rotate around 7).
2. **Left-rotate** at v (rotate around 3).



Result of the double rotation

The middle key 5 rises to the top of the subtree, with 3 on its left and 7 on its right. Every node is back to BF = 0, the tree is still a BST, and the subtree height matches what it was **before** inserting 6, so we stop: no rebalancing propagates to 12.

5.5 The Four Rotation Cases

At the lowest unbalanced node v (with $\mathrm{BF}(v) = \pm 2$), which fix to apply is decided by the balance factor of the child on the heavy side.

Rotation case table

$\mathrm{BF}(v)$	heavy child's BF	Shape	Fix
$+2$	$\mathrm{BF}(v.\mathrm{right}) = +1$	right-right (RR)	single Left rotation at v
$+2$	$\mathrm{BF}(v.\mathrm{right}) = -1$	right-left (RL)	Right rotation on v.right, then Left at v
-2	$\mathrm{BF}(v.\mathrm{left}) = -1$	left-left (LL)	single Right rotation at v
-2	$\mathrm{BF}(v.\mathrm{left}) = +1$	left-right (LR)	Left rotation on v.left, then Right at v

Rule of thumb

- Heavy side and grandchild's heavy side **agree** (both right, or both left) ⇒ single rotation.
- They **disagree** (a zig-zag) ⇒ double rotation (fix the child first, then v).

After the rotation the subtree's height returns to what it was before the insert, so **at most one rotation (single or double) is ever needed per insert**.

5.6 Insert Algorithm Sketch

Insert(T, x)

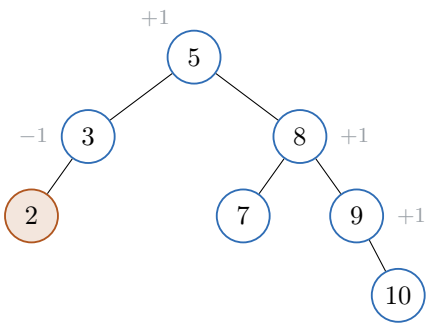
```
1.  Insert x into T as in any BST.
    x is now a leaf; set BF(x) = 0.
2.  Go up from x to the root.  For each node v on this path:
    Adjust BF(v):
       if x is in the RIGHT subtree of v:  increment BF(v)
       if x is in the LEFT subtree of v:  decrement BF(v)
       if BF(v) == 0:  (subtree height unchanged)  STOP
    Rebalance if necessary:
       if BF(v) == +2:
           if BF(v.right) == +1:  Left rotation; update BFs; STOP
           if BF(v.right) == -1:  Right-Left rotation; update BFs; STOP
       if BF(v) == -2:  (symmetric: Right or Left-Right rotation) STOP
```

Cost: the walk up is at most the height $= O(\log n)$, each step is $O(1)$, and at most one rotation ($O(1)$) happens. So **Insert is** $O(\log n)$.

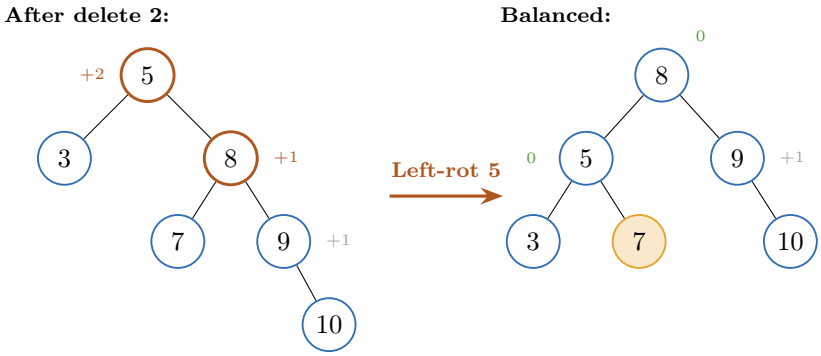
6 Midterm-Style Process Problems

Predicted: AVL Delete with rebalancing

Delete 2 from this AVL tree, then restore the AVL property.



Delete 2 (a leaf): the left subtree of 5 shrinks from height 1 to 0. Walk up: at 5, BF $= 2 - 0 = +2$ and its right child 8 is right-heavy ($+1$): **RR**, so a single **Left rotation at 5**.

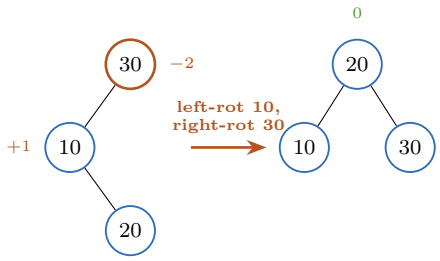


8 rises, and its old left child 7 moves to become the right child of 5 ($5 < 7 < 8$). Remember: unlike Insert, a Delete rotation can leave the subtree shorter, so you may need to keep rebalancing up toward the root.

Predicted: build an AVL by inserting a sequence

Insert 30, then 10, then 20 into an empty AVL tree. Show the rebalancing.

30 then 10 gives a legal tree (30 is left-heavy, BF $= -1$). Inserting 20 goes $30 \to 10 \to$ right, making BF(10) $= +1$ and BF(30) $= -2$ with left child 10 *right-heavy*: the **LR** zig-zag.



The middle key 20 rises to the root with 10 left and 30 right. One double rotation fixes it, and at most one rotation (single or double) is ever needed per insert.

Quick Reference: Lecture 5

Height	edges to deepest leaf; leaf $= 0$, empty $= -1$
BF(v)	$h_R - h_L$; $+1$ right-heavy, 0 balanced, -1 left-heavy
AVL tree	BST with $-1 \le \text{BF}(v) \le +1$ for *every* node
AVL height	$\Theta(\log n)$ $(\le 1.44 \log_2(n+2))$; $m(h) = 1 + m(h-1) + m(h-2)$
Search	same as BST $\to O(\log n)$
Insert	BST-insert leaf, walk up adjusting BF, rotate if $\pm 2 \to O(\log n)$
Stop rule	stop when a node's BF becomes 0 (its height did not change)
RR $(+2,+1)$	single Left rotation
LL $(-2,-1)$	single Right rotation
RL $(+2,-1)$	Right rotation on child, then Left at v
LR $(-2,+1)$	Left rotation on child, then Right at v
Rotations/insert	at most one (single or double)

Lecture 6: AVL Insert Correctness and Cost

1 What This Lecture Adds

Lecture 5 gave the **Insert** procedure by example and the **four rotation cases** (RR, LL, RL, LR). This lecture does *not* re-teach those. It proves the procedure restores the AVL property in **every** case, states the **node layout**, and shows each rotation is $\Theta(1)$.

Setup: the first unbalanced node

Insert x as a BST leaf, then walk up. Let A = the **first** node on the path from x to the root with $|\mathrm{BF}(A)| = 2$. Everything strictly below A is still AVL.
By symmetry assume $\mathrm{BF}(A) = +2$ (the -2 case is a mirror image). Then before the insert $\mathrm{BF}(A) = +1$, and the insert went into A's **right** subtree and **increased its height** from h to $h+1$. A's left subtree still has height h.

Let B = right child of A (root of the subtree that grew to height $h+1$).

2 Why the Insert Algorithm is Correct

2.1 The Skipped Step: $\mathrm{BF}(B) \neq 0$

Before rotating we must know B's shape. Since A is the **first** unbalanced node, B is still balanced: $\mathrm{BF}(B) \in \{-1, 0, +1\}$.

Claim: $\mathrm{BF}(B)$ cannot be 0

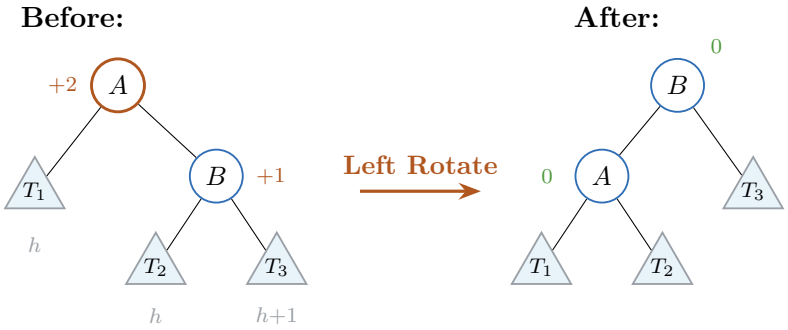
So $\mathrm{BF}(B) = +1$ (**Case 1a**) or $\mathrm{BF}(B) = -1$ (**Case 1b**). These are the only two cases.

Why $\mathrm{BF}(B) = 0$ is impossible. The insert of a single leaf increased B's height by exactly 1 (from h to $h+1$). One leaf can lengthen only **one** of B's two subtrees. If $\mathrm{BF}(B) = 0$ now (both subtrees height h), then before the insert one subtree was height h already, so B had height $h+1$ *before* the insert too. That contradicts "B's height increased." □

$\mathrm{BF}(B) = +1$: the growth is on B's **right** (right-right, RR).
$\mathrm{BF}(B) = -1$: the growth is on B's **left** (right-left, RL). Here name $C = B$.left.

2.2 Case 1(a): Single Left Rotation

x landed in B's right subtree. Fix with a single **Left Rotation at** A.



What the rotation does

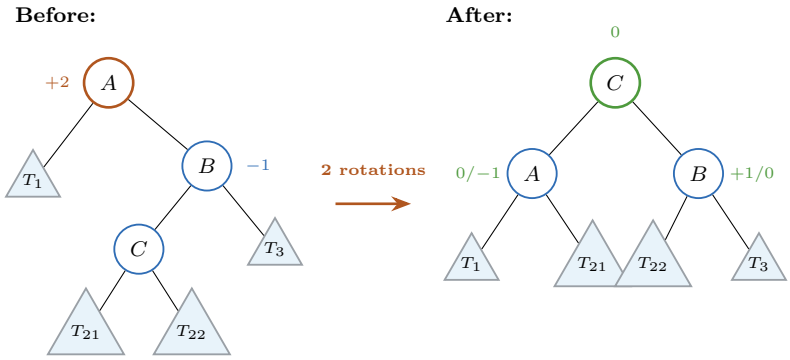
B rises, A becomes B's left child, and B's old left subtree T_2 (height h) becomes A's new right subtree. Order $T_1\,A\,T_2\,B\,T_3$ is unchanged, so the **BST property holds**. New factors: $\mathrm{BF}(A) = 0$, $\mathrm{BF}(B) = 0$.

Bonus: height is restored

Before inserting x, the subtree at A had height $h + 2$. After the rotation it is $h + 2$ again. **Same height as before the insert** $\Rightarrow$ no ancestor's BF changes $\Rightarrow$ stop. One rotation finishes the whole insert.

2.3 Case 1(b): Double Right-Left Rotation

x landed in B's **left** subtree, rooted at $C = B$.left. A single left rotation would **not** fix it. Do two: **right-rotate at** B, then **left-rotate at** A.



What the double rotation does

C (the middle key) rises to the top. Its two subtrees T_{21}, T_{22} split: T_{21} goes under A, T_{22} goes under B. Order $T_1\,A\,T_{21}\,C\,T_{22}\,B\,T_3$ is preserved, so the **BST property holds**. Final factors: $\mathrm{BF}(C) = 0$; A and B get $\{0, -1\}$ and $\{+1, 0\}$ depending on which of T_{21}, T_{22} received x. The subtree height returns to $h + 2$, so again we stop.

Special case $h = -1$

When $h = -1$, the subtrees are empty and C **is** the inserted leaf x. The double rotation collapses to: A, x, B become root x with children A and B, all BF $= 0$. Same rule, no special handling needed.

3 Cost, Node Layout, and Algorithm

3.1 Rotation Cost and Total Insert Cost

A rotation is $\Theta(1)$

A left (or right) rotation rewires only **3 pairs of pointers**. No subtree is copied or traversed $\Rightarrow \Theta(1)$.

Insert is $\Theta(\log n)$

BST insert of the leaf	$O(\log n)$
Walk up adjusting BF	$O(\log n)$, $O(1)$ per node
Rebalance	**at most one** rotation (single or double), $\Theta(1)$

The "height restored" bonus is what caps it at **one** rotation: after rotating, the subtree has its pre-insert height, so nothing above changes. Total: $O(\log n)$.

3.2 Contents of an AVL Node

AVL node layout

Each node stores **5 fields**:

PARENT	pointer to parent (needed to walk up)
`key`	the element's key (data)
`BF`	balance factor $\in \{-1, 0, +1\}$ (stored, not recomputed)
LEFT CHILD	pointer to left child
RIGHT CHILD	pointer to right child

Storing `BF` in the node lets the walk-up update it in $O(1)$ per step instead of recomputing subtree heights.

3.3 Final Insert Algorithm

Insert(T, x)

```
Insert x into T as in any BST; x is now a leaf; set BF(x) = 0.
Go up from x to the root.  For each node v on the path:
   Adjust BF(v):
      if x is in RIGHT subtree of v:  BF(v) += 1
      if x is in LEFT subtree of v:  BF(v) −= 1
      if BF(v) == 0:  STOP                                  (height unchanged)
   Rebalance:
      if BF(v) == +2:
          if BF(v.right)==+1:  Left Rotation; update BFs; STOP
          if BF(v.right)==−1:  Right-Left Rotation; update BFs; STOP
      if BF(v) == −2:  symmetric (Right or Left-Right rotation)
```

4 Exam-Style Practice

★ Practice: insert triggering a double rotation

Start from the AVL tree of $\{10, 20, 30, 40, 50\}$ built in order, then **Insert 25**. Show the rebalancing.

Inserting $10, 20, 30$ forces one left rotation, giving root 20. After $40, 50$ (another left rotation at 30) the tree is:

$$20 \to \{\, 10,\ 40 \to \{30, 50\}\,\}$$

Insert 25: search path $20 \to 40 \to 30$, land as **left child of 30**.
Walk up: $\mathrm{BF}(30) : 0 \to -1$, $\mathrm{BF}(40) : 0 \to -1$, $\mathrm{BF}(20) : +1 \to +2$.
$v = 20$ has $\mathrm{BF} = +2$ and $\mathrm{BF}(v.\mathrm{right}) = \mathrm{BF}(40) = -1 \Rightarrow$ **Right-Left rotation**: right-rotate at 40, then left-rotate at 20. Middle key 30 rises:

$$30 \to \{\, 20 \to \{10, 25\},\ 40 \to \{\emptyset, 50\}\,\}$$

All $|\mathrm{BF}| \le 1$. In-order: $10, 20, 25, 30, 40, 50$ ✓

AVL questions in this course ask you to **draw the tree after an insert/delete and name each rotation** (see the Fall 2014 boxes in Lecture 5). The skill tested is exactly the case decision above: $\mathrm{BF}(v)$ plus the heavy child's BF picks single vs double.

5 Midterm-Style Process Problems

Predicted: compute $m(h)$ and bound the height

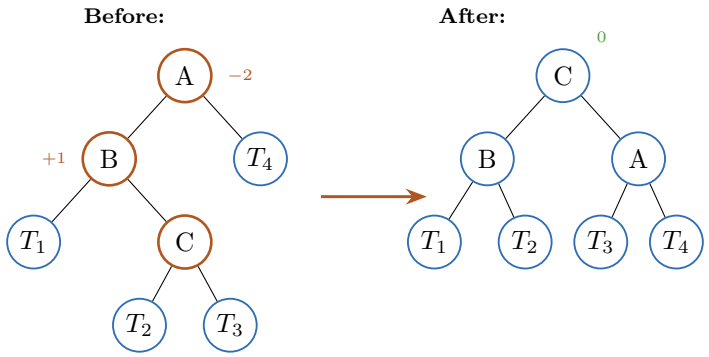
Let $m(h)$ be the minimum number of nodes in an AVL tree of height h. Using $m(0) = 1$, $m(1) = 2$, $m(h) = 1 + m(h-1) + m(h-2)$, fill the table through $h = 6$, and explain why this forces height $= O(\log n)$.

h	0	1	2	3	4	5	6
$m(h)$	1	2	4	7	12	20	33

Each value is 1 plus the previous two, so $m(h)$ grows like the **Fibonacci** numbers, that is *exponentially* in h. Since an AVL tree of height h has $n \ge m(h)$ nodes and $m(h)$ grows exponentially, h can only grow like $\log n$. Precisely, height $\le 1.44 \log_2(n+2)$, so every AVL operation is $O(\log n)$.

Predicted MC: identify the rotation

The AVL tree on the left became the one on the right after an insertion. Which rotation was applied at A?



(a) single Right at A (LL) (b) single Left at A (RR) (c) Left on B then Right at A (LR) (d) Right on B then Left at A (RL)

(c) LR double. A is left-heavy (-2) and its left child B is right-heavy ($+1$), a zig-zag, so left-rotate B then right-rotate A. The middle node C rises to the top with B and A as its children.

Quick Reference: Lecture 6

First unbalanced node A	lowest node with $\lvert\text{BF}\rvert = 2$ after insert; assume $+2$
$B = A$.right	grew to height $h+1$; $\text{BF}(B) \neq 0$ (proved)
Case 1a $(+2, +1$, RR)	single Left rotation; $A, B \to 0$
Case 1b $(+2, -1$, RL)	Right-Left double; C rises, $\text{BF}(C) = 0$
$h = -1$ special case	inserted leaf becomes the new subtree root
Rotation cost	3 pointer pairs $\Rightarrow \Theta(1)$
Height after fix	equals pre-insert height $\Rightarrow$ at most 1 rotation
Node fields	PARENT, key, BF, LEFT CHILD, RIGHT CHILD
Insert total	BST insert + walk-up + ≤ 1 rotation $\to \Theta(\log n)$

End of Lectures 1–6 notes.

Lecture 7: Augmenting Data Structures

1 Augmenting a Data Structure

Augmenting a data structure

Take an **existing** data structure and add **extra stored information** so it supports **new operations**, without breaking the old ones. Three steps:

1. Decide **what extra info** to store at each node.
2. Check the info can be **cheaply maintained** under every existing operation.
3. Use the info to **implement the new operations** efficiently.

Not automatic: choosing the right field takes **creativity**.

2 Dynamic Order Statistics

Dynamic Order Statistics ADT

A dynamic set S of elements with **distinct keys**, supporting:

- **Search**(x), **Insert**(x), **Delete**(x) (the Dictionary operations)
- **Select**(k): return the k-th smallest element (the element of **rank** k)
- **Rank**(x): given a pointer to x, return its rank

Rank of x = position of x in the sorted order of S (starting at 1).

2.1 Example

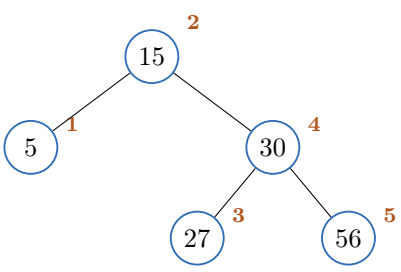
Select and Rank on $S = \{5, 15, 27, 30, 56\}$

$$\textbf{Select}(4) = 30 \quad \textbf{Select}(1) = 5 \quad \textbf{Rank}(15) = 2 \quad \textbf{Rank}(30) = 4$$

Rank(30) = 4 because **exactly 3** elements are < 30 (namely $5, 15, 27$), so 30 sits in position 4.

2.2 Naive Attempt: Store Each Node's Rank

Keep S in an **AVL tree** (Search/Insert/Delete already $O(\log n)$). To get Select/Rank, augment it. First idea: at each node store its **rank**.



Good, but...

Good: Rank(x) reads the stored field ($O(1)$ after finding x), and Select(k) searches for the node whose rank is k in $O(\log n)$.

Insert(2) (a new minimum) makes 2 rank 1 and **shifts every other rank up by 1**. One insert can rewrite **all** n rank fields $\Rightarrow$ Insert and Delete become $O(n)$. The rank field is **too global** to maintain cheaply.

3 Size-Augmented AVL Tree

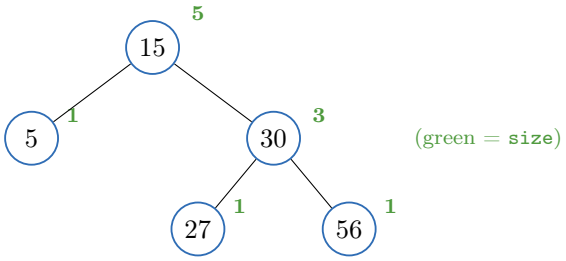
3.1 The `size` Field

Store subtree size

At each node x store $\texttt{size}(x)$ = number of nodes in the subtree rooted at x. Invariant, for every node:

$$\texttt{size}(x) = \texttt{size}(\text{left}(x)) + \texttt{size}(\text{right}(x)) + 1$$

(An empty child contributes $\texttt{size} = 0$.) This is a **local** field: it depends only on x's two children, so a single insert changes it on just **one root-to-leaf path**.



3.2 Select: the k-th Smallest

Relative rank RR(x)

RR(x) = the rank of x counting **only the nodes in x's own subtree**. In other words, if you sorted just x's subtree, RR(x) is the position x would land in.

$$\text{RR}(x) = \texttt{size}(\text{left}(x)) + 1$$

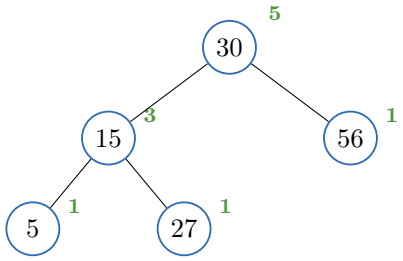
Why that formula works

In a BST, the **left subtree of x holds exactly the elements smaller than x** (within x's subtree). So:

- $\texttt{size}(\text{left}(x))$ = how many elements are smaller than x here.
- $+1$ = counts x itself.

Add them and you get x's position. No traversal needed: the `size` field already has the count.

Reading RR straight off the tree



Green numbers are `size`. Read RR off each node:

- RR(30) = `size`(left=15) + 1 = 3 + 1 = 4. So 30 is the **4th smallest** in this whole subtree. Check: 5, 15, 27, **30**, 56 ✓
- RR(15) = `size`(left=5) + 1 = 1 + 1 = 2. So 15 is the **2nd smallest** in the subtree under 15 ({5, 15, 27}). Check: 5, **15**, 27 ✓
- RR(5) = 0 + 1 = 1 (no left child). 5 is smallest in its subtree. ✓

Notice RR is **local**: RR(15) = 2 is its rank inside {5, 15, 27}, not its rank 2 in the full set. Select stitches these local ranks together as it walks down.

Select(x, k): the k-th smallest in the subtree at x

```
RR ← size(left(x)) + 1
if k = RR:  return x                                  // x sits exactly at position k
if k < RR:  return Select(left(x), k)                 // target is smaller, look left
if k > RR:  return Select(right(x), k − RR)           // skip x and all of left
```

Call `Select(T, k) = Select(root, k)`.

The one tricky line: why subtract when going right

Going right means the answer is bigger than x, so it is **not** x and not anyone in x's left subtree. That is RR elements we are leaving behind. Inside the right subtree those elements no longer exist, so the thing that was rank k overall becomes rank $k - \text{RR}$ *there*. That is why we recurse with $k - \text{RR}$, not k.

Trace: Select(3) on $S = \{5, 15, 27, 30, 56\}$

1. At root 15: RR = `size`(left = 5) + 1 = 1 + 1 = 2. $k = 3 > 2 \Rightarrow$ go right, $k \leftarrow 3 - 2 = 1$.
2. At 30: RR = `size`(left = 27) + 1 = 1 + 1 = 2. $k = 1 < 2 \Rightarrow$ go left.
3. At 27: RR = 0 + 1 = 1. $k = 1 = \text{RR} \Rightarrow$ return 27.

Select(3) = 27 ✓ (sorted: 5, 15, **27**, 30, 56).

Cost of Select

Each call drops one level (or returns), $O(1)$ per level, height $O(\log n) \Rightarrow O(\log n)$.

3.3 Rank of a Node

Given a pointer to x, walk **from x up to the root**, accumulating the elements that come before x.

Rank(T, x): rank of x in the whole tree

```
RR ← size(left(x)) + 1                          // rank of x within its own subtree
y ← x
while y ≠ root:
    if y is the RIGHT child of its parent p:
        RR ← RR + size(left(p)) + 1              // p and its left subtree precede x
    y ← p
return RR
```

Why

Climbing to parent p: if we came up from p's **right** subtree, then p itself and all of p's **left** subtree are smaller than x, so add $\texttt{size}(\text{left}(p)) + 1$. If we came up from the **left**, those nodes are all larger, so RR is **unchanged**.

Accumulating up the path

Suppose $\text{RR}(x) = 7$ inside x's subtree. Climbing through ancestors where x sits in the right subtree, with left-subtree sizes 3, then 6:

$$7 \;\to\; 7 + 3 + 1 = 11 \;\to\; 11 + 6 + 1 = 18.$$

Each right-turn adds "that ancestor + its left subtree." Left-turns add nothing.

Cost of Rank

One step per level, $O(1)$ each, height $O(\log n) \Rightarrow O(\log n)$.

3.4 Maintaining `size` on Insert and Delete

Insert(T, x): keep `size` correct

```
Phase 1 (BST insert):  insert x as a leaf; size(x) = 1;
    for each node y on the path from the root down to x:  size(y) += 1
Phase 2 (rebalance):  walk up adjusting BF; rotate if some BF hits ±2.
    After a rotation, recompute size() of the rotated nodes
    bottom-up:  size(z) = size(left(z)) + size(right(z)) + 1
```

Why it stays $O(\log n)$

Phase 1 touches one root-to-leaf path ($O(\log n)$ increments). A rotation only changes the children of **2 nodes**, so recomputing their `size` from the invariant is $O(1)$ extra. AVL insert already does at most one rotation $\Rightarrow$ total $O(\log n)$. **Delete** is symmetric (decrement down the path; fix `size` after each rotation).

Fixing `size` after a left rotation

A left rotation at A (right child B rises). Only A and B changed children, so recompute **bottom-up**:

- A (now lower): $\texttt{size}(A) = \texttt{size}(T_1) + \texttt{size}(T_2) + 1$.
- B (now on top): $\texttt{size}(B) = \texttt{size}(A) + \texttt{size}(T_3) + 1$.

Everything below T_1, T_2, T_3 is untouched, so two updates suffice.

4 Summary and Practice

Size-augmented AVL: all operations $O(\log n)$

Search / Insert / Delete	$O(\log n)$	AVL + fix `size` on the path
Select(k)	$O(\log n)$	descend using RR = `size`(left) + 1
Rank(x)	$O(\log n)$	climb, add `size`(left(p)) + 1 on right-turns

★ **Practice: Select and Rank**

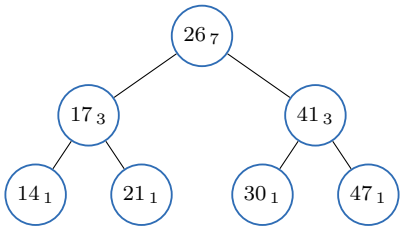
Augmented AVL for $S = \{5, 15, 27, 30, 56\}$ (root 15; 30 has children 27, 56). Do **Select**(5) and **Rank**(27).

Select(5)**:** root 15, RR = 2, $5 > 2 \Rightarrow$ right, $k = 3$. At 30, RR = 2, $3 > 2 \Rightarrow$ right, $k = 1$. At 56, RR = $1 = k \Rightarrow$ return 56.
Rank(27)**:** start RR = `size`(left(27)) + 1 = 0 + 1 = 1. Climb to 30: 27 is 30's **left** child $\Rightarrow$ unchanged. Climb to 15: 30 is 15's **right** child $\Rightarrow$ RR $\leftarrow 1 +$ `size`(left(15)=5) + 1 = 1 + 1 + 1 = 3. Rank(27) = 3 ✓

5 Midterm-Style Process Problems

Predicted: OS-Select on a size-augmented tree

Each node stores size = number of nodes in its subtree (shown as a subscript). Run OS-Select(5), the 5th smallest key.



At each node compute $r = \text{size(left)} + 1$, the rank of this node inside its own subtree.

1. At 26: $r = \text{size}(17) + 1 = 3 + 1 = 4$. Want $i = 5 > 4$, so go **right** and set $i = 5 - 4 = 1$.
2. At 41: $r = \text{size}(30) + 1 = 1 + 1 = 2$. Want $i = 1 < 2$, so go **left**.
3. At 30: $r = \text{size(nil)} + 1 = 1$. Want $i = 1 = r$: **return** 30.

In-order the keys are $14, 17, 21, 26, 30, 41, 47$, and indeed the 5th is **30**. One root-to-node path, so $O(\log n)$.

Predicted: design an augmentation

You have a set S of integers in an AVL tree. Design an augmentation so that SumLeq(t) = "sum of all keys $\leq t$" runs in $O(\log n)$, and Insert/Delete stay $O(\log n)$.

Augment each node u with size(u) and sum(u) = sum of all keys in u's subtree. Both satisfy a local rule from the children:

$$\text{sum}(u) = \text{sum(left)} + \text{key}(u) + \text{sum(right)},$$

so they are maintainable in $O(1)$ extra work per node during the usual Insert/Delete walk and rotations (a rotation touches a constant number of nodes).
Query. Walk down from the root: at node u, if $t < \text{key}(u)$ recurse left; else add sum(left)+key(u) to the running total and recurse right. This visits one path, so SumLeq is $O(\log n)$. (Dividing SumLeq(t) by a size count gives an $O(\log n)$ Average query, the same trick as past exams.)

Quick Reference: Lecture 7

Augmenting (3 steps)	pick field; keep it cheap to maintain; use it for new ops
Order statistics	Dictionary + Select(k) (k-th smallest) + Rank(x)
Naive (store rank)	Select/Rank easy but Insert/Delete $O(n)$ (too global)
Fix: store `size`(x)	`size`(x) = `size`(left) + `size`(right) + 1 (local)
RR(x)	`size`(left(x)) + 1 = rank of x within its subtree
Select(x, k)	=RR: return; <: go left; >: go right with $k -$ RR
Rank(T, x)	climb to root; right-child step adds `size`(left(p)) + 1
Maintain `size`	+1 down insert path; recompute 2 nodes per rotation ($O(1)$)
All operations	$O(\log n)$

Lecture 8: Hash Tables

6 Recall: the Dictionary ADT

Dictionary ADT

Object: a dynamic set S. Each element has a key $k \in U$, where U is the **universe** of possible keys.
Operations: **Search**, **Insert**, **Delete**.

We already saw BSTs and AVL trees give $O(\log n)$ per operation. Hashing aims for $\Theta(1)$ expected per operation.

7 If the Universe U is Small: Direct-Access Table

Direct-access table

If $U = \{0, 1, \ldots, u-1\}$ is small, use an array $T[0 \ldots u-1]$. Store the element with key k directly in slot $T[k]$. Empty slots hold `NIL`.

Direct-access operations: all $\Theta(1)$ worst case

Search(T, k)	return $T[k]$
Insert(T, x)	$T[k] \leftarrow x$ (where k is x's key)
Delete(T, x)	$T[k] \leftarrow$ `NIL`

Why it works: counting character frequencies

Read a text file, count how often each character appears. Each character has an ASCII code in $\{0, \ldots, 127\}$. Use $T[0 \ldots 127]$, all entries start at 0. For each character with code k, increment $T[k]$. Then $T[k]$ is the frequency of the character with code k.
Total time $\Theta(n)$ where n is the length of the text. Each character costs $\Theta(1)$.

8 When the Universe is Huge

Suppose keys are 64-bit integers: $|U| = 2^{64} \approx 1.84 \times 10^{19}$. A table of size 2^{64} is **infeasible**. Usually we only store $n \ll |U|$ keys, so almost every slot would be wasted.

Fix: use a table of size m that is close to n (not $|U|$), and a function that maps any key into $\{0, \ldots, m-1\}$. This is a hash table.

9 Hash Tables

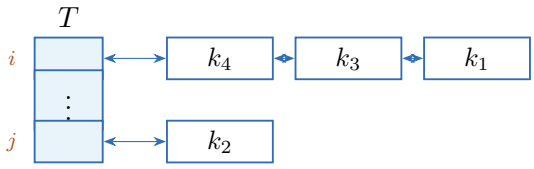
- m = size of the table $T[0 \ldots m-1]$.
- **Hash function** $h : U \to \{0, 1, \ldots, m-1\}$.
- For a key k, if $h(k) = i$ we say "k *hashes to slot* i".
- Store the element with key k in slot $T[h(k)]$.

A collision is when two different keys hash to the same slot: $h(k_1) = h(k_3)$. Since $|U| > m$, by pigeonhole **some pair of keys must collide**. We cannot avoid collisions, only resolve them.

10 Resolving Collisions: Chaining

Chaining

Keep a **linked list** at each table slot. To insert the element with key k, add it at the **head** of the list in slot $T[h(k)]$. All keys that hash to the same slot sit in the same list (chain).



Here $h(k_1) = h(k_3) = h(k_4) = i$ and $h(k_2) = j$. New keys go in at the head, so k_4 (inserted last) is first in slot i's chain.

11 Dictionary Operations with Chaining

Costs (assume doubly linked lists, and $h(k)$ computable in $\Theta(1)$)

Insert(x)	insert at head of $T[h(\text{key}(x))]$	$\Theta(1)$
Delete(x)	remove x, given a pointer to x	$\Theta(1)$
Search(k)	scan the chain at $T[h(k)]$	depends on chain length

All n keys could hash to the **same** slot. Then that one chain has length n, and Search takes $\Theta(n)$ in the worst case. So we analyse the **average** chain length instead.

12 Analysis: SUHA and the Load Factor

Simple Uniform Hashing Assumption (SUHA)

Each key k we insert is **equally likely** to hash into any of the m slots, **independently** of the other keys:

$$\Pr[h(k) = i] = \frac{1}{m}, \qquad 0 \le i \le m-1.$$

Expected chain length = load factor α

Insert n keys into an empty table $T[0 \dots m-1]$. Let n_i be the length of the chain at $T[i]$ (a random variable). The chains partition all keys, so

$$n_0 + n_1 + \cdots + n_{m-1} = n \implies E[n_0] + \cdots + E[n_{m-1}] = n.$$

By symmetry under SUHA all $E[n_i]$ are equal, so $m \cdot E[n_i] = n$, giving

$$\boxed{E[n_i] = \frac{n}{m} = \alpha} \qquad \text{the load factor of } T.$$

13 Expected Cost of Search (under SUHA)

Expected Search time is $\Theta(1+\alpha)$

Unsuccessful	inspect a whole chain: α comparisons on average
Successful	inspect about half a chain, then match: $\frac{\alpha}{2} + 1$ on average

The "+1" is the cost of computing $h(k)$ and indexing, which happens even when $\alpha = 0$. So expected Search time is $\Theta(1+\alpha)$.

Keeping Search fast: fix α

If n keeps growing while m is fixed, $\alpha = n/m$ grows and chains get longer. Keep α **bounded by a constant** by ensuring $m = \Theta(n)$. Then $\alpha = \Theta(1)$ and

$$\text{Expected Search time} = \Theta(1).$$

The table may have to grow (resizing)

To keep $\alpha \le c$ for a chosen constant c:

$$\frac{n}{m} \le c \iff m \ge \frac{n}{c}.$$

When n grows past $c\,m$: (1) allocate a new, larger table so $\alpha \le c$ holds again, then (2) **rehash** every current element into it.

14 Choosing a Hash Function

Division method

$$h(k) = k \bmod m$$

Common heuristic: choose m to be **prime**.

Picking m

We want to store $n = 2000$ keys with target $\alpha \leq 3$.

$$\alpha = \frac{2000}{m} \leq 3 \implies m \geq \frac{2000}{3} \approx 666.7.$$

Smallest qualifying prime: 673. Use $m = 673$ and $h(k) = k \bmod 673$.

If $m = 2^\ell$, then $k \bmod m$ uses **only the last ℓ bits** of k. Extreme case $m = 2$: even keys go to slot 0, odd keys to slot 1. If most keys are even, almost all land in slot 0: a very uneven distribution.
A good hash function uses **all the bits** of the key and spreads keys **evenly** across slots, even when the inserted keys follow some regular pattern.

15 Past Exam Questions

★ **Seen on a past exam** (Final, Dec 2013, Q1: Hash Tables)

[18 marks] Hash tables with chaining; m = table size, n = number of items.
(a) What issue with the *direct-access table* does the *hash table* solve?
(b) To store n items, what is a good choice for m?
(c) What is the purpose of chaining?
(d) What property should a good hash function have?
(e) Assume SUHA. Fill the running-time table (tight bounds, only n and m):

(a) A direct-access table needs size $|U|$, which is infeasible when U is huge (e.g. 64-bit keys). A hash table uses a small table of size $m = \Theta(n)$ and maps keys into it with h.
(b) $m = \Theta(n)$, chosen prime (keeps $\alpha = n/m$ constant).
(c) Chaining **resolves collisions**: all keys hashing to the same slot are kept in one linked list at that slot.
(d) It should spread keys **evenly** across all m slots and **use all the bits** of the key, so chains stay short even for patterned inputs.

	Search(k)	**Insert**(k)	**Delete**(x)
Worst case	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Expected	$\Theta(1+\alpha) = \Theta(1)$	$\Theta(1)$	$\Theta(1)$

(Delete is $\Theta(1)$ given a pointer to x and doubly linked chains. Expected Search uses $m = \Theta(n)$ so $\alpha = \Theta(1)$.)

★ **Seen on a past exam** (Final, Dec 2013, Q1e)

[4 marks] A hash table T of size 1000 uses chaining, starts empty, SUHA holds. What is the **maximum** number of keys we can insert so that, afterwards, the **expected** number of key comparisons to search for a key **not** in T is at most 15?

An unsuccessful search inspects a whole chain, so its expected number of comparisons is $\alpha = \frac{n}{m} = \frac{n}{1000}$. Require

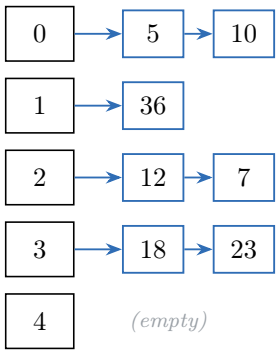
$$\frac{n}{1000} \leq 15 \implies n \leq 15000.$$

Maximum = 15000 keys.

16 Midterm-Style Process Problems

Predicted: build a chained table, then analyse

A chained hash table has $m = 5$ slots and $h(k) = k \bmod 5$. Insert, **at the head** of each chain, the keys $10, 7, 23, 5, 18, 12, 36$. Draw the table, then give the load factor and expected search cost under SUHA.



Homes: $10, 5 \to 0$; $36 \to 1$; $7, 12 \to 2$; $23, 18 \to 3$; nothing $\to 4$. Head insertion puts the later key in front, so slot 0 reads $5 \to 10$, slot 2 reads $12 \to 7$, slot 3 reads $18 \to 23$. **Load factor** $\alpha = n/m = 7/5 = 1.4$. Under SUHA the expected chain length is α, so expected search is $\Theta(1 + \alpha) = \Theta(1)$. Keep $m = \Theta(n)$ (resize and rehash when α passes a constant) and all three operations stay expected $\Theta(1)$. **Worst case:** all keys in one slot, a chain of length n, search $\Theta(n)$.

Predicted: "power of two choices" with chaining

Two independent hash functions h_1, h_2 both satisfy SUHA. When inserting x, place it in the **shorter** of the two chains $T[h_1(x)]$ and $T[h_2(x)]$ (ties go to h_1).

(a) If k of the m slots are empty just before inserting x, what is $\Pr[x \text{ lands in an empty slot}]$?

(b) With $m = 4$ and current chain lengths $4, 1, 7, 7$ at slots $0, 1, 2, 3$, what is the expected length of the chain x joins (not counting x)?

(a) x lands in an empty slot exactly when the *shorter* chain is empty, i.e. when **at least one** of the two hashed slots is empty. A single hash misses the empty slots with probability $\frac{m-k}{m}$, so

$$\Pr[\text{empty}] = 1 - \left(\tfrac{m-k}{m}\right)^2.$$

(b) For any pair of slots (i, j) the joined chain has length $\min(L_i, L_j)$ (a tie joins h_1, which is the same length). With $L = (4, 1, 7, 7)$ and all 16 ordered pairs equally likely,

$$\mathbb{E}[\text{length}] = \frac{1}{16}\sum_{i,j}\min(L_i, L_j) = \frac{13+4+19+19}{16} = \frac{55}{16} = 3.4375.$$

Picking the shorter of two random chains pulls the expected length well below the plain α.

Quick Reference: Lecture 8

Direct-access table	U small: store x in $T[k]$; all ops $\Theta(1)$; needs size $\lvert U\rvert$
Hash table	$h : U \to \{0, \ldots, m-1\}$; store x in $T[h(k)]$; $m = \Theta(n)$
Collision	$h(k_1) = h(k_2)$; unavoidable since $\lvert U\rvert > m$
Chaining	linked list per slot; Insert/Delete $\Theta(1)$, Search = chain length
Worst-case Search	all keys in one chain $\Rightarrow \Theta(n)$
SUHA	$\Pr[h(k) = i] = 1/m$, independent
Load factor	$\alpha = n/m$ = expected chain length $E[n_i]$
Expected Search	$\Theta(1+\alpha)$; unsuccessful α, successful $\alpha/2 + 1$
Keep fast	$m = \Theta(n) \Rightarrow \alpha = \Theta(1) \Rightarrow$ Search $\Theta(1)$
Resize	keep $\alpha \le c$: $m \ge n/c$; grow + rehash all
Division method	$h(k) = k \bmod m$, m prime; avoid $m = 2^\ell$
Good hash function	uses all bits, spreads keys evenly

Lecture 9: Bloom Filters

17 What is a Bloom Filter?

A space-efficient "**probabilistic dictionary**". It keeps the "fingerprints" of the elements of a set S (not the elements themselves).
Operations:

- **BF_Insert**(x): $S \leftarrow S \cup \{x\}$.
- **BF_Search**(x): returns one of two answers.

The two answers of BF_Search

"No" $\Rightarrow$ **definitely** $x \notin S$ (never wrong)

"Probably Yes" $\Rightarrow$ probably $x \in S$, but perhaps not

So a Bloom filter can have **false positives** (says "Probably Yes" for an $x \notin S$), but **never false negatives**.

Application: malicious URLs

A browser wants to check if a URL is malicious. The set S of malicious URLs is huge: say 10 million URLs at 50 bytes each, so ≈ 500 MB. We do not want to store that on the user's side. Store only a **Bloom filter** of S on the user's side (around 10 MB). To check a URL:

- "No" $\Rightarrow$ URL $\notin S$, safe to visit the website.
- "Probably Yes" $\Rightarrow$ probably malicious, issue a warning.

This works with a filter of ≈ 10 MB and a false positive rate of just 2%.

18 Structure

Bloom filter structure

- An array **BF**$[0 \ldots m-1]$ of m **bits**, initially all 0.
- t **independent** hash functions $h_1, h_2, \ldots, h_t$, each $h_i : U \to \{0, \ldots, m-1\}$, each satisfying SUHA.

SUHA here: every element is equally likely to hash to any of the m slots, independent of where the other elements hash.

Fingerprint

The "**fingerprint**" of x is the set of indices $h_1(x), h_2(x), \ldots, h_t(x)$.

BF_Insert(x) and BF_Search(x)

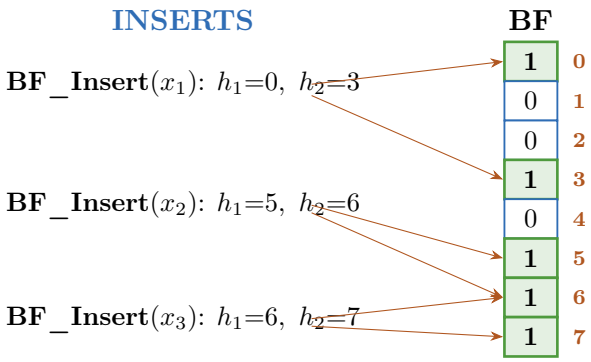
```
BF_Insert(x):                                   // write x's fingerprint into BF
   for i = 1 to t:
      BF[h_i(x)] = 1
BF_Search(x):                                   // look for x's fingerprint in BF
   for i = 1 to t:
      if BF[h_i(x)] = 0 then return "No"
   return "Probably Yes"
```

Why "No" is always correct

If x was ever inserted, *all* its fingerprint bits were set to 1. So if *any* fingerprint bit of x is still 0, then x was never inserted: "No" is certain. "Probably Yes" only means all of x's bits happen to be 1, which could be due to **other** elements.

19 Worked Example: BF$[0\ldots7]$ with $t = 2$

Insert x_1, x_2, x_3, then search x and x'. Each insert sets two bits; each search reads two bits.



Reading two searches

- **BF_Search**(x) with $h_1(x) = 3$, $h_2(x) = 1$: bit 3 is 1 but bit 1 is 0, so return "No". Certain: $x \notin \{x_1, x_2, x_3\}$.
- **BF_Search**(x') with $h_1(x') = 3$, $h_2(x') = 6$: both bits are 1, so return "Probably Yes". But x' was never inserted; bit 3 came from x_1 and bit 6 from x_2 and x_3. This is a **false positive**.

20 Probability of a False Positive

Setup. Insert $x_1, \ldots, x_n$ into an empty **BF**$[0\ldots m-1]$ with t independent hash functions satisfying SUHA. Then do **BF_Search**(x) for some $x \notin \{x_1, \ldots, x_n\}$. We want Pr[false positive] = Pr[BF_Search(x) = "Probably Yes"].

Step 1: probability a fixed bit is still 0

Fix an index i. Across the n inserts, each setting t bits, SUHA and independence of the h_j make the nt "did not hit i" events mutually independent:

$$\Pr[\mathbf{BF}[i]=0]=\prod_{k=1}^{n}\prod_{j=1}^{t}\Pr[h_j(x_k)\neq i]=\left(1-\tfrac{1}{m}\right)^{nt}\approx\left(e^{-1/m}\right)^{nt}=e^{-nt/m},$$

using $1-y\approx e^{-y}$ for small $y=1/m$. Therefore

$$\Pr[\mathbf{BF}[i]=1]=1-\Pr[\mathbf{BF}[i]=0]\approx\underbrace{1-e^{-nt/m}}_{q}.$$

Lemma 1

After inserting $x_1,\ldots,x_n$ into a **BF** of size m with t hash functions, for an arbitrary index i:

$$\Pr[\mathbf{BF}[i]=1]\approx 1-e^{-nt/m}\;=\;q.$$

Step 2: a false positive needs all t fingerprint bits set

BF_Search(x) = "Probably Yes" iff $\mathbf{BF}[h_1(x)]=1$ **and** $\cdots$ **and** $\mathbf{BF}[h_t(x)]=1$. Treating these t bits as independent and each $\approx q$ by Lemma 1:

$$\Pr[\text{false positive}]\approx q^{\,t}=\left(1-e^{-nt/m}\right)^{t}.$$

21 Choosing t: the Optimal Number of Hash Functions

Optimal t

Let $\frac{m}{n}$ = **number of bits per element** (fix this ratio). Minimise $\Pr[\text{false positive}]=\left(1-e^{-nt/m}\right)^{t}$ over t (take the derivative with respect to t, set to 0):

$$\text{Optimal } t=(\ln 2)\,\frac{m}{n}=0.69\,\frac{m}{n},\qquad\text{then }\Pr[\text{false positive}]\approx 0.62^{\,m/n}.$$

The malicious-URL numbers

Want a Bloom filter for $n=10$ million URLs. Allocate 8 bits per URL:

$$\text{Size } m=8n=8\times 10\text{M bits}\approx 10\text{ MB},\qquad\frac{m}{n}=8.$$

Much less than storing even one full URL per element. The best t:

$$t\approx 0.69\cdot\frac{m}{n}=0.69\times 8=5.52\text{ hash functions.}$$

With this t:

$$\Pr[\text{false positive}]\approx 0.62^{\,m/n}=0.62^{\,8}\approx 2\%.$$

22 Past Exam Question

★ **Seen on a past exam** (Final, Dec 2012, Q3: Hash / Bloom Filter)

[10 marks] Let H be the family of all hash functions $U \to \{0, \ldots, m-1\}$. For $i = 1, \ldots, k$ let $B_i[0 \ldots m-1]$ be a Boolean array, initially all 0. Let $S \subseteq U$ with $|S| = n$. Pick k hash functions $h_1, \ldots, h_k$ uniformly and independently from H. For each $x \in S$ and each i, set $B_i[h_i(x)] \leftarrow 1$. Define

$$Check(y) = \textbf{true} \iff \forall i \in \{1, \ldots, k\}\ B_i[h_i(y)] = 1.$$

(a) For $k = 2$, $m = 5$, $U = \{0, \ldots, 24\}$, $h_1(x) = x \bmod 5$, $h_2(x) = (2x + 3) \bmod 5$, $S = \{2, 4, 7\}$, draw B_1 and B_2.
(b) For $h \in H$ and $x, y \in U$ chosen uniformly and independently, what is $\Pr[h(x) = h(y)]$?
(c) If $y \in S$, explain why $Check(y) = \textbf{true}$ always.
(d) If $y \notin S$, explain why $\Pr[Check(y) = \textbf{true}] = \left(1 - (1 - 1/m)^n\right)^k$.

This is a Bloom filter with one array per hash function. *Check* is BF_Search, and (d) is the false positive probability.
(a) Compute each hash on $S = \{2, 4, 7\}$:

$$h_1 :\ 2 \mapsto 2,\ 4 \mapsto 4,\ 7 \mapsto 2 \qquad h_2 :\ 2 \mapsto 2,\ 4 \mapsto 1,\ 7 \mapsto 2$$

So B_1 has bits $2, 4$ set and B_2 has bits $1, 2$ set:

$$B_1 = [\,0, 0, 1, 0, 1\,], \qquad B_2 = [\,0, 1, 1, 0, 0\,] \quad \text{(indices 0..4)}.$$

(b) Fix $h(x)$; $h(y)$ is uniform over m slots and independent, so $\Pr[h(x) = h(y)] = \dfrac{1}{m} = \dfrac{1}{5}$.
(c) If $y \in S$, then when y was processed we set $B_i[h_i(y)] \leftarrow 1$ for *every* i. Those bits are never reset, so all are still 1 and $Check(y) = \textbf{true}$. (No false negatives.)
(d) Fix i. After inserting the n elements of S, by SUHA and independence

$$\Pr[B_i[h_i(y)] = 0] = \left(1 - \tfrac{1}{m}\right)^n \ \Rightarrow\ \Pr[B_i[h_i(y)] = 1] = 1 - \left(1 - \tfrac{1}{m}\right)^n.$$

$Check(y) = \textbf{true}$ needs this for all k independent arrays, so

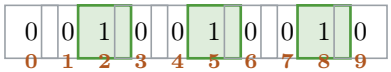
$$\Pr[Check(y) = \textbf{true}] = \left(1 - (1 - 1/m)^n\right)^k. \qquad \checkmark$$

(Same shape as our q^t with $t = k$.)

23 Midterm-Style Process Problems

Predicted: insert, query, and estimate the false-positive rate

A Bloom filter uses an $m = 10$ bit array and $t = 2$ hash functions. Insert x (hashes to bits $2, 5$) and y (hashes to bits $5, 8$). Then query z (hashes to bits $2, 8$).



Does the filter say z is present? Is that correct? Estimate the false-positive probability.

Inserting x, y sets bits $\{2, 5, 8\}$ (bit 5 is shared). Querying z checks bits 2 and 8, both 1, so the filter answers **"probably yes"**. But z was never inserted: this is a **false positive**. Bloom filters never give false negatives, but another element's bits can cover a query's bits exactly like this.

Probability. A given bit is still 0 with probability $\left(1 - \frac{1}{m}\right)^{nt} \approx e^{-nt/m}$, so it is 1 with probability $q = 1 - e^{-nt/m}$. A false positive needs all t queried bits to be 1:

$$\Pr[\text{false positive}] \approx q^t = \left(1 - e^{-nt/m}\right)^t.$$

Here $n = 2, t = 2, m = 10$: $q = 1 - e^{-0.4} \approx 0.33$, so $\Pr[\text{FP}] \approx 0.33^2 \approx 0.11$. (The exact finite form uses $1 - (1 - \frac{1}{m})^{nt}$ in place of q.)

Predicted: size the filter

You will store $n = 10^7$ URLs and want about 8 bits per URL, so $m = 8 \times 10^7$. What is the optimal number of hash functions t, and the resulting false-positive rate?

Optimal $t \approx 0.69\,\frac{m}{n} = 0.69 \times 8 \approx 5.5$ (use $t = 5$ or 6). The resulting rate is $\Pr[\text{FP}] \approx 0.62^{m/n} = 0.62^8 \approx 0.02$, i.e. about 2%. More bits per element (m/n) drives the false-positive rate down exponentially.

Quick Reference: Lecture 9

Bloom filter	probabilistic dictionary; stores fingerprints, not elements
Answers	"No" $\Rightarrow$ certainly $x \notin S$; "Probably Yes" $\Rightarrow$ maybe (false positives)
No false negatives	if x inserted, all its bits are 1
Structure	m-bit array **BF**, t independent hashes h_i under SUHA
BF_Insert(x)	set $\mathbf{BF}[h_i(x)] = 1$ for $i = 1..t$
BF_Search(x)	if any $\mathbf{BF}[h_i(x)] = 0$ return "No"; else "Probably Yes"
$\Pr[\mathbf{BF}[i] = 0]$	$(1 - 1/m)^{nt} \approx e^{-nt/m}$
$\Pr[\mathbf{BF}[i] = 1]$	$q = 1 - e^{-nt/m}$ (Lemma 1)
Pr[false positive]	$\approx q^t = (1 - e^{-nt/m})^t$
Optimal t	$(\ln 2)\,\frac{m}{n} = 0.69\,\frac{m}{n}$; then $\Pr[\text{FP}] \approx 0.62^{m/n}$
m/n	bits per element (fix it, then optimise t)

Lecture 10: Randomized Quicksort

24 The Algorithm

Randomized Quicksort $RQS(S)$

Input: a set S of n **distinct** keys. Output: the keys of S in increasing order.
Recursive, divide-and-conquer:

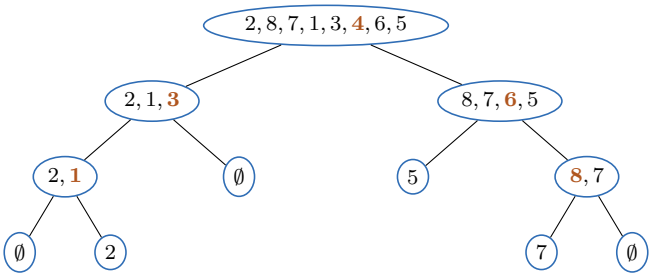
1. If S is empty, return. If $|S| = 1$, output the key and return.
2. Select a key p (the **pivot**) uniformly at random from S (each key equally likely).
3. Compare p to every other key, splitting S into two subsets:

$$S_< = \{\, s \in S \mid s < p \,\}, \qquad S_> = \{\, s \in S \mid s > p \,\}.$$

   (This split costs $n - 1$ key comparisons when $|S| = n$.)
4. Output $RQS(S_<)$; output p; output $RQS(S_>)$.

One execution on $S = \{2, 8, 7, 1, 3, 4, 6, 5\}$

Picking pivot 4 at the root splits into $S_< = \{2, 1, 3\}$ and $S_> = \{8, 7, 6, 5\}$, then each side recurses with its own random pivots. One possible recursion tree (pivots shown next to each node):



Output (in-order): $1, 2, 3, 4, 5, 6, 7, 8$.

Basic observations (in every $RQS(S)$ execution)

- Two keys are compared **only if** one of them is selected as a pivot.
- Two keys are compared **at most once**.
- If two keys are "**split apart**" into different subsets by a pivot (like 2 and 7 are split by pivot 4), they are **never** compared.

25 Worst-Case Number of Comparisons

Fix an input S with n distinct keys, run $RQS(S)$, and let C = number of key comparisons.

Worst case: $\Theta(n^2)$

If each pivot is the largest (or smallest) remaining key, the split is maximally uneven:

$$C = (n-1) + (n-2) + \cdots + 2 + 1 = \frac{n(n-1)}{2}, \quad \text{i.e. } \Theta(n^2).$$

But the pivot is *random*, so we care about the expected value $E(C)$, averaged over all random pivot choices.

26 Expected Number of Comparisons: Indicator Variables

Setup

Let $z_1 < z_2 < \cdots < z_i < \cdots < z_j < \cdots < z_n$ be the keys of S in **sorted** order. Define the indicator random variable

$$c_{ij} = \begin{cases} 1 & \text{if } RQS(S) \text{ compares } z_i \text{ and } z_j, \\ 0 & \text{otherwise.} \end{cases}$$

Since any pair is compared at most once, the total number of comparisons is

$$C = \sum_{1 \le i < j \le n} c_{ij}.$$

Expected value of an indicator = its probability

$$E[c_{ij}] = 1 \cdot \Pr[c_{ij} = 1] + 0 \cdot \Pr[c_{ij} = 0] = \Pr[c_{ij} = 1] = \Pr[z_i \text{ and } z_j \text{ are compared}].$$

By **linearity of expectation**:

$$E(C) = E\Big(\sum_{1 \le i < j \le n} c_{ij}\Big) = \sum_{1 \le i < j \le n} E[c_{ij}] = \sum_{1 \le i < j \le n} \Pr[z_i \text{ and } z_j \text{ are compared}].$$

27 The Key Lemma

Lemma: probability two keys are compared

For $i < j$:

$$\Pr[z_i \text{ and } z_j \text{ are compared}] = \frac{2}{j - i + 1}.$$

Intuition on the extremes

- z_1 and z_n (smallest and largest) are compared iff the **first** pivot chosen among all n keys is z_1 or z_n: probability $\frac{2}{n}$. Here $j - i + 1 = n$, so the lemma checks out. (In the example $n = 8$: $\Pr = \frac{2}{8} = \frac{1}{4}$.)
- z_i and z_{i+1} (adjacent in sorted order) are **always** compared: nothing lies between them, so whoever becomes a pivot first between the two must compare them. Here $j - i + 1 = 2$,

so $\frac{2}{2} = 1$. ✓

Proof of the Lemma

Consider the set of keys between z_i and z_j inclusive:

$$Z_{ij} = \{ z_i, z_{i+1}, \ldots, z_j \}, \qquad |Z_{ij}| = j - i + 1.$$

- As long as *RQS* picks pivots **not in** Z_{ij}, all of Z_{ij} stays together inside one subset, and z_i, z_j stay uncompared.
- At some point *RQS* **must** pick a pivot $p \in Z_{ij}$ (to split Z_{ij}). Consider the **first** such p. Two cases:
  1. If $z_i < p < z_j$ (so p is a *middle* element of Z_{ij}): p splits z_i and z_j apart $\Rightarrow$ they are **never** compared.
  2. If $p = z_i$ or $p = z_j$: then z_i and z_j **are** compared.

Since the first pivot in Z_{ij} is equally likely to be any of its $|Z_{ij}| = j - i + 1$ keys, and exactly 2 of them (z_i and z_j) cause a comparison:

$$\Pr[z_i \text{ and } z_j \text{ compared}] = \Pr[p = z_i \text{ or } p = z_j \mid p \in Z_{ij}] = \frac{2}{j - i + 1}. \qquad \square$$

28 Putting it Together: $E(C) = O(n \log n)$

The bound

$$E(C) = \sum_{1 \le i < j \le n} \frac{2}{j - i + 1} \le 2n\left(1 + \tfrac{1}{2} + \tfrac{1}{3} + \cdots + \tfrac{1}{n}\right) \quad \text{[CLRS p. 184].}$$

The harmonic number $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ is $O(\log n)$. Therefore

$$\boxed{E(C) = O(n \log n)}.$$

Worst case $C = \Theta(n^2)$, but the **expected** number of comparisons is only $O(n \log n)$. Randomizing the pivot makes the bad $\Theta(n^2)$ inputs vanishingly unlikely: no fixed input is consistently bad.

29 Past Exam Question

★ **Seen on a past exam** (Final, Aug 2014, Q6: Analysis of a Randomized Algorithm)

[20 marks] An array A of n distinct numbers is sorted ascending **except** that the first two elements $A[1], A[2]$ are swapped (e.g. $[20, 10, 30, 40, \ldots]$). `RANDOM-SORT` repeatedly picks a pair (i, j) uniformly at random from the set S of all pairs with $1 \le i \le n$, $1 \le j \le n$ (including $i = j$ and $i > j$), removes it, and if $i < j$ and $A[i] > A[j]$ swaps them; it stops once A is sorted.

(1) How many "while" iterations are needed to **guarantee** A is sorted? **(2)** Expected number of iterations before A is sorted (exact form)?

There are $|S| = n^2$ pairs. The array is sorted the moment the one **fixing pair** $(1, 2)$ is drawn (it is the only pair with $i < j$ and $A[i] > A[j]$); every other pair does nothing.
(1) Worst case: the fixing pair $(1, 2)$ is drawn **last**, after all $n^2 - 1$ useless pairs. So to guarantee sorted we may need all

$$n^2 \text{ iterations.}$$

(2) The fixing pair is equally likely to be at any position $1, 2, \ldots, n^2$ in the random removal order. Its expected position is

$$E[\text{iterations}] = \frac{1 + 2 + \cdots + n^2}{n^2} = \frac{n^2 + 1}{2}.$$

Expected $= \dfrac{n^2 + 1}{2}$.

Method tested: same as *RQS* analysis: an indicator / uniform-position argument, then expectation.

★ **Seen on a past exam** (Final, Dec 2012, Q8: Randomized Search)

[15 marks] A sorted linked list of length n is stored in an $n \times 2$ matrix A (value, next pointer) with a `head` index. *Search*(x) first jumps to a **uniformly random** row i, compares $A[i, 1]$ with x (resetting to `head` if too big, else following `next`), then walks forward. Find the worst-case **expected** number of comparisons.

Worst case (deterministic): $\Theta(n)$ (a random start near the front, then walk the whole list).
Expected: define indicators over which rows are visited, use linearity of expectation exactly as in the *RQS* analysis. The random starting row makes the expected walk length a constant fraction of n, so the expected number of comparisons is $\Theta(n)$ in the worst case, but the *constant* is smaller than the deterministic walk.

Why it is here: it drills the same toolkit: probability space + indicator random variables + linearity of expectation.

30 Midterm-Style Process Problems

Predicted: why RQS is $O(n \log n)$ expected

Let the sorted order be $z_1 < z_2 < \cdots < z_n$. Show the expected number of key comparisons in randomized quicksort is $O(n \log n)$.

Indicators. Any pair z_i, z_j $(i < j)$ is compared **at most once**, and only if one of them is the *first* pivot chosen from the block $Z_{ij} = \{z_i, \ldots, z_j\}$. Let $X_{ij} = 1$ if they are ever compared. Then $C = \sum_{i<j} X_{ij}$ and

$$\mathbb{E}[C] = \sum_{i<j} \Pr[z_i, z_j \text{ compared}].$$

Lemma. Each of the $j-i+1$ keys of Z_{ij} is equally likely to be the first pivot; a comparison happens iff that first pivot is z_i or z_j (an interior first pivot splits them apart forever). So

$$\Pr[z_i, z_j \text{ compared}] = \frac{2}{j-i+1}.$$

Sum. With $d = j - i$,

$$\mathbb{E}[C] = \sum_{i=1}^{n-1} \sum_{d=1}^{n-i} \frac{2}{d+1} < \sum_{i=1}^{n} 2H_n = 2nH_n = O(n \log n),$$

where $H_n = \Theta(\log n)$ is the harmonic number.

Predicted: worst case and a single pair

(a) What is the worst-case number of comparisons for RQS, and which pivots cause it? (b) For $n = 10$, what is $\Pr[z_2$ and z_9 are compared$]$?

(a) The worst case is when every pivot is the current smallest or largest, so each split is of size 0 and $n-1$. Then the comparison count is $\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$.

(b) By the lemma, $\Pr = \frac{2}{j-i+1} = \frac{2}{9-2+1} = \frac{2}{8} = \frac{1}{4}$. Far-apart ranks are compared rarely, which is exactly why the expected total collapses to $O(n \log n)$.

Quick Reference: Lecture 10

$RQS(S)$	random pivot p; split into $S_<, S_>$; recurse; split costs $n-1$
Compared when	two keys compared only if one is a pivot; at most once
Split apart	a pivot between them $\Rightarrow$ never compared
Worst case	$C = \frac{n(n-1)}{2} = \Theta(n^2)$
Indicator	$c_{ij} = 1$ iff z_i, z_j compared; $C = \sum_{i<j} c_{ij}$
$E[c_{ij}]$	$= \Pr[z_i, z_j \text{ compared}]$ (indicator)
Linearity	$E(C) = \sum_{i<j} \Pr[z_i, z_j \text{ compared}]$
Key lemma	$\Pr[z_i, z_j \text{ compared}] = \frac{2}{j-i+1}$ (Z_{ij}, first pivot argument)
Result	$E(C) \le 2nH_n = O(n \log n)$

Tutorial: Augmented Data Structures and Interval Trees

31 The Problem: Overlapping Intervals

Applications

- Book a conference room from 16:00 to 17:00: does it **conflict** with an existing booking?
- Does my lifeguard shift **overlap** with anyone else's? If so, who?

Closed interval and overlap

A closed interval is $i = [t_1, t_2]$ with $t_1 \leq t_2$. Write $\text{low}[i] = t_1$ and $\text{high}[i] = t_2$.
Two intervals i and i' **overlap** iff:

$$\text{low}[i] \leq \text{high}[i'] \quad \textbf{and} \quad \text{low}[i'] \leq \text{high}[i].$$

("Each one starts before the other one ends.")

Store a set of intervals and support, all in $O(\log n)$ time (n = number of intervals):

- **Insert**(x), **Delete**(x),
- **Search**(x): return *an* interval in the structure that overlaps with x (or `NIL`).

32 Designing an Augmented Data Structure (the Method)

The design steps (this method is the important part)

1. **What base data structure, and why?**
2. **What information should be stored?**
   - the **key** of the base structure,
   - the **augmentations** (extra fields).
3. **Maintain the augmentations efficiently**: each must be computable in $O(1)$ from a node's own fields and its children's fields.
4. Implement the new operation using the augmentations.

Step 1 and 2 for interval trees

Base structure: an AVL tree. Why? It gives **ordering** plus $O(\log n)$-height operations.
For each node x storing interval $i = (\text{low}[i], \text{high}[i])$:

- **Key** $= \text{low}[x]$ (the low endpoint). The AVL is ordered by low endpoints.
- **Augmentation** $\max[x]$ = the largest high endpoint among all intervals in x's subtree.

33 Maintaining the Augmentation $\max[x]$ Efficiently

$\max[x]$ is computable in $O(1)$ from the children

$$\max[x] = \max\big(\text{high}[x],\ \max[x.\text{left}],\ \max[x.\text{right}]\big).$$

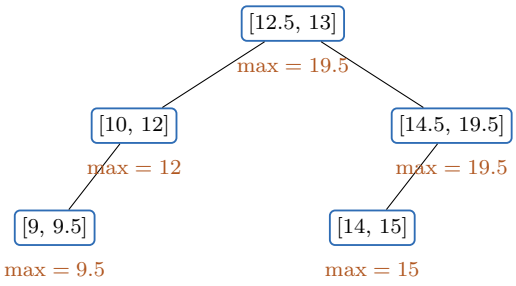
So after any structural change, max only needs updating on the $O(\log n)$ nodes on the path from the root to the modified leaf, plus the $O(1)$ rotated nodes.

Inserts and Deletes stay $O(\log n)$

Insert / Delete = ordinary AVL insert / delete + max updates. The updates apply only to nodes on the path from the root to the modified leaf, plus rotated nodes:

$$O(\log n) \text{ nodes} \times O(1) \text{ update each} = O(\log n).$$

A small interval tree (keyed on low, with max below each node)



Each max equals the largest high endpoint in that node's subtree, and each is the max of the node's own high and its children's max values.

34 Interval-Search(x)

Search(x): find any interval overlapping x

```
y = root
while y != NULL and (y and x do NOT overlap):
    if y.left = NULL or max[y.left] < low[x]:
        y = y.right
    else:
        y = y.left
return y
```

Cost: one root-to-leaf descent, $O(1)$ per node, so $O(\log n)$.

35 Why it is Correct

Loop invariant

If there is an interval that overlaps with the query x, **then** there always is one such interval in y's subtree.

Maintaining the invariant

Holds at the root (the whole tree is y's subtree).
After each loop iteration, when y and x do not overlap we move to a child:

(i) If y.left = `NULL`: the left subtree is empty, so any overlapping interval must be on the right. Go right. ✓

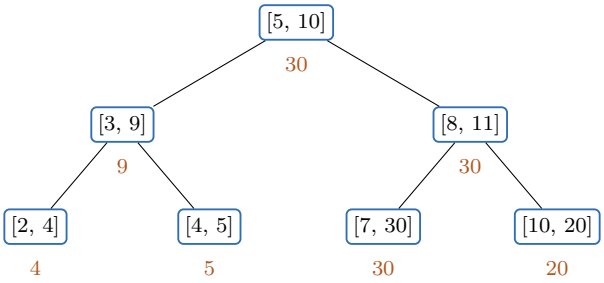
(ii) If $\max[y.\text{left}] < \text{low}[x]$: every interval in the left subtree ends before x starts, so none can overlap x. Go right. ✓

(iii) If $\max[y.\text{left}] \geq \text{low}[x]$: some interval in the left subtree ends at or after $\text{low}[x]$. One can show that if *any* interval in y's subtree overlaps x, then one in the **left** subtree does too, so going left is safe. Go left. ✓

When the loop ends, either $y =$ `NULL` (no overlap exists, by the invariant) or y overlaps x.

A worked search

Interval tree (key = low, max below each node):



Search $x = [12, 13]$**:**

- At $[5, 10]$: no overlap ($10 < 12$). $\max[\text{left}] = 9 < \text{low}[x] = 12$, so go **right**.
- At $[8, 11]$: no overlap ($11 < 12$). $\max[\text{left}] = 30 \geq 12$, so go **left**.
- At $[7, 30]$: overlap? $\text{low} = 7 \leq 13$ and $12 \leq 30$: yes. Return $[7, 30]$.

Search $x = [0, 1]$**:** at $[5, 10]$ no overlap, $\max[\text{left}] = 9 \geq 0$ so go left; at $[3, 9]$ no overlap, $\max[\text{left}] = 4 \geq 0$ so go left; at $[2, 4]$ no overlap, left is `NULL` so go right; right is `NULL`, loop ends with $y =$ `NULL`. Correctly reports **no overlapping interval**.

36 Past Exam Questions

★ Seen on a past exam (Final, Dec 2012, Q2: BST / Augmentation)

[10 marks] Define $Average(v)$: the average of all keys in the subtree rooted at node v. Augment an AVL tree to support $Average$ in $O(1)$ without hurting the $O(\log n)$ bound of Insert and Delete.
(a) What extra info at each node? **(b)** How to do $Average$ in $O(1)$? **(c)** Why do Insert/Delete stay $O(\log n)$?

(a) Store at each node v two augmentations:

$$\text{size}[v] = \text{number of nodes in } v\text{'s subtree}, \qquad \text{sum}[v] = \text{sum of all keys in } v\text{'s subtree}.$$

(b) $Average(v) = \dfrac{\text{sum}[v]}{\text{size}[v]}$, which is $O(1)$.
(c) Both augmentations are computable in $O(1)$ from the children:

$$\text{size}[v] = 1 + \text{size}[v.\text{left}] + \text{size}[v.\text{right}], \quad \text{sum}[v] = \text{key}[v] + \text{sum}[v.\text{left}] + \text{sum}[v.\text{right}].$$

They only change on the $O(\log n)$ nodes on the insert/delete path plus the $O(1)$ rotated nodes, each updated in $O(1)$: total still $O(\log n)$. This is the exact same augmentation method as $\max[x]$ for interval trees.

★ Seen on a past exam (Final, Dec 2013, Q10: Range Search)

[15 marks] Support on a set S of integers, each in $O(\log n)$: `Insert`(k), `Delete`(x), `Search`(k), and

$$IsEmptyRange(S, a, b) = \textbf{true} \iff S \text{ has no integer in } [a, b].$$

(a) Which structure (no modifications needed)? **(b)** Pseudocode for $IsEmptyRange$. **(c)** Why $O(\log n)$?

(a) A **balanced BST (AVL tree)** keyed on the integers. Insert, Delete, Search are already $O(\log n)$.
(b) Find the smallest key that is $\geq a$ (the *successor* of $a-1$), then check whether it falls in $[a, b]$:

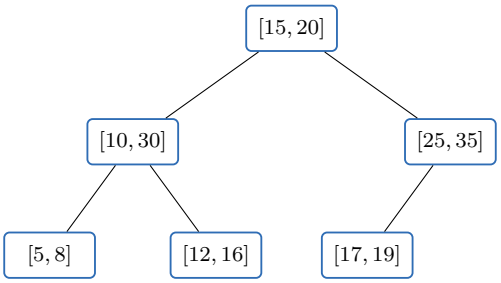
```
IsEmptyRange(S, a, b):
   k = smallest key in S with k ≥ a
      // walk down:  at node x, if x.key ≥ a go left (remember
x), else go right
   if k = NIL or k > b:  return true
   else:  return false
```

(c) Finding the smallest key $\geq a$ is a single root-to-leaf descent, $O(1)$ per node, on a tree of height $O(\log n)$: so $O(\log n)$. If that key exists and is $\leq b$, there is an integer of S in $[a, b]$; otherwise the range is empty.
Link to interval trees: same idea, a balanced BST answering a *range* question in $O(\log n)$ by using ordering (here no augmentation is even needed).

37 Midterm-Style Process Problems

Predicted: fill max fields, then run Interval-Search

Each node shows [low, high]. Fill max[x] at every node, then run Interval-Search for $i = [21, 23]$.



Max fields, bottom up, max = max of own high and children's max: $[5,8] \to 8$, $[12,16] \to 16$, $[17,19] \to 19$, $[10,30] \to \max(30,8,16) = 30$, $[25,35] \to \max(35,19) = 35$, root $[15,20] \to \max(20,30,35) = 35$.
Search $[21,23]$ (go left iff left child exists and max[left] $\geq$ low[i] $= 21$, else right):

1. At $[15,20]$: overlap needs $15 \leq 23$ and $21 \leq 20$; the second fails, **no overlap**. max[left] $= 30 \geq 21$, so go left.
2. At $[10,30]$: $10 \leq 23$ and $21 \leq 30$, both hold. **Overlap, return** $[10,30]$.

★ **Find-the-bug** (modern MC style)

```
Buggy-Interval-Search(T, i)
1  x = T.root
2  while x ≠ nil and i does not overlap x.int:
3     if x.left ≠ nil and x.left.max < i.low:
4        x = x.left
5     else x = x.right
6  return x
```

Which line contains the bug? (a) line 2 (b) line 3 (c) lines 4 and 5 (d) line 6

(b) line 3. The test is backwards. Go **left** when the left subtree *could* hold an overlap, i.e. when x.left.max $\geq$ i.low. As written it goes left exactly when the left subtree cannot help, so it misses overlaps on the left. Fix: `x.left.max ≥ i.low`.

Quick Reference: Interval Trees

Overlap test	$\text{low}[i] \le \text{high}[i']$ **and** $\text{low}[i'] \le \text{high}[i]$
Design steps	base DS (+ why) $\to$ key + augmentations $\to$ maintain in $O(1) \to$ use
Base structure	AVL (ordering + $O(\log n)$)
Key	$\text{low}[x]$ (low endpoint)
Augmentation	$\max[x]$ = largest high endpoint in x's subtree
Maintain max	$\max[x] = \max(\text{high}[x], \max[x.\text{left}], \max[x.\text{right}])$; $O(1)$ per node
Insert / Delete	AVL + max updates on path + rotated nodes = $O(\log n)$
Search(x)	go right if left empty or $\max[\text{left}] < \text{low}[x]$, else go left; $O(\log n)$
Invariant	if any interval overlaps x, one such interval is in y's subtree
General method	store size/sum/max etc. computable in $O(1)$ from children

---

End of Lectures 8, 9, 10 & Interval Trees tutorial notes.

Lecture 11: Disjoint Sets (Union–Find), Part 1

1 The Disjoint Sets ADT

Disjoint Sets (Union–Find) ADT

Object: n distinct elements named $1, 2, \ldots, n$. Initially each element is in its own singleton set:

$$S_1 = \{1\}, \quad S_2 = \{2\}, \quad \ldots, \quad S_n = \{n\}.$$

Each set has a representative element that uniquely identifies it. Write S_x for the set represented by x.

Operations:

- **Union**(x, y): here x, y are distinct set representatives. Replace S_x and S_y by $S = S_x \cup S_y$. Some element of S becomes the representative of S.
- **Find**(z): return the representative of the set that contains z.

The sequence σ we care about

Each **Union** merges two sets into one, so it reduces the number of sets by 1. Starting from n singletons, there can be at most $n - 1$ Unions.

$$\sigma \;=\; \text{a sequence of } (n-1) \textbf{ Unions} \text{ mixed with } m \geq n \textbf{ Finds}.$$

Goal: a data structure that minimizes the total cost of executing such a sequence σ.

Tracing the operations (S_j = set represented by j)

Initially	$S_1{=}\{\underline{1}\}$, $S_2{=}\{\underline{2}\}$, $S_3{=}\{\underline{3}\}$, $S_4{=}\{\underline{4}\}$, $S_5{=}\{\underline{5}\}$	
Union$(3,4)$	$S_3 = \{\underline{3}, 4\}$	(now **Find**$(4) = 3$)
Union$(1,5)$	$S_1 = \{\underline{1}, 5\}$	
Union$(1,3)$	$S_1 = \{\underline{1}, 5, 3, 4\}$	(now **Find**$(4) = 1$, **Find**$(2) = 2$)
Union$(1,2)$	$S_1 = \{\underline{1}, 5, 3, 4, 2\}$	

2 Implementation 1: Linked Lists

One linked list per set

Store each set as a doubly linked list, with a **head** and **tail** pointer. The first element of the list is the representative.



Costs

Union(x, y) splice tail of one list onto head of the other $\to O(1)$

Find(z) walk from z to the head to read the representative $\to O(n)$

$$\text{Worst-case cost of } \sigma = O(\underbrace{n \cdot 1}_{\text{per Union}} + \underbrace{m \cdot n}_{\text{per Find}}) = O(m\,n).$$

3 Implementation 2: Augmented Linked Lists

Augmented linked list

Same as before, but **each element also stores a pointer to its representative**. Now **Find** reads that pointer directly.

Costs flip

Find(z) follow the representative pointer $\to O(1)$

Union(x, y) concatenate, then redirect every element's pointer of one set $\to O(n)$

$$\text{Worst-case cost of } \sigma = O(\underbrace{n \cdot n}_{\text{per Union}} + \underbrace{m \cdot 1}_{\text{per Find}}) = O(n^2 + m).$$

3.1 Weighted Union (WU) on augmented lists

WU rule (by size)

When merging, **append the smaller list onto the bigger list** (keep a size counter for each list). This way we redirect the fewer pointers.

Each **Union** is still $O(n)$ in the worst case and each **Find** is $O(1)$, but WU improves the *total* cost of a whole sequence:

Claim (proved in tutorial)

With WU, the worst-case cost of executing σ is

$$O(\underbrace{m}_{m \text{ Finds}} + \underbrace{n \log n}_{(n-1) \text{ Unions}}).$$

4 Implementation 3: Disjoint-Set Forest

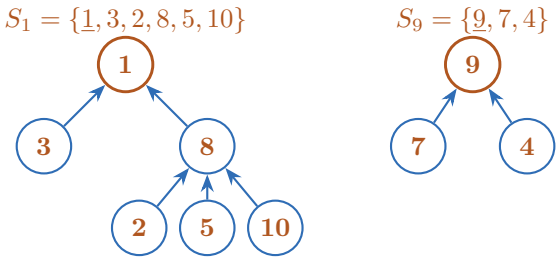
Forest representation

Each set is a tree:

- each node holds one element,
- each **non-root** node points to its **parent**,

- the **root** holds the set representative.

An array $A[1..n]$ stores, for each element x, a pointer $A[x]$ to x's node, so we can jump to any element in $O(1)$.



Costs

Find(x)	follow parent pointers up to the root	$\rightarrow$ $O(1 + \text{Find-path length})$
Union(x, y)	make root of S_x a child of root of S_y	$\rightarrow$ $O(1)$

Do **Union**$(1, 2)$, **Union**$(2, 3)$, ..., **Union**$(n-1, n)$ (each time attaching the old root under the new element). This builds a single chain of **height** $n - 1$.

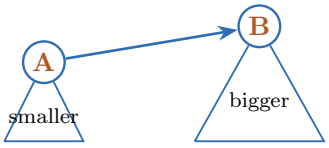
$$\text{Cost of } m \textbf{ Find}(1) = \Omega(m \cdot n) \quad \Rightarrow \quad \text{worst-case } \sigma = \Omega(m\, n).$$

To reduce the cost of σ, we must reduce the length of Find paths, i.e. reduce the **height** of the trees formed during σ.

5 Heuristic 1: Weighted Union (WU) by Size

WU rule (MUST KNOW)

In **Union**(A, B), make the root of the smaller tree a child of the root of the bigger tree. Here **size** = number of nodes in the tree. Store a size counter at each root.



Lemma (the key fact)

With WU, any tree T of height h created while running σ has at least 2^h nodes:

$$|T| \geq 2^h.$$

Proof sketch: induction on h

Base $h = 0$: a single node, $|T| = 1 = 2^0$. ✓
Step: assume the lemma for height h. A tree of height $h+1$ is formed by a **Union** where the smaller tree A (whose root sits at depth, giving height h) is hung under the bigger tree B:

- By I.H., $|A| \geq 2^h$ (the subtree that reached height h).
- By WU, B is the bigger tree, so $|B| \geq |A| \geq 2^h$.

Therefore $|T| = |A| + |B| \geq 2^h + 2^h = 2^{h+1}$. ✓

Corollary and Theorem

Since $|T| \leq n$, we get $2^h \leq |T| \leq n$, so

$$h \leq \log_2 n.$$

Theorem. With WU, executing any sequence σ of $n-1$ Unions and $m \geq n$ Finds takes

$$O(m + n \log n) = O(m \log n) \qquad \text{(since } m \geq n\text{)}.$$

6 Past Exam Questions

★ **Seen on a past exam** (Dec 2019 Final, MCQ)

Linked-list disjoint sets where every node stores a back-pointer to the representative.
Q12. Worst-case cost of a single `Union`(x, y) / `MakeSet`(x) / `FindSet`(x)?
Q13. Using union-by-weight, worst-case cost of a sequence of n operations (`MakeSet`, `FindSet`, `Union`)?

Q12. $O(n)$, $O(1)$, $O(1)$. A back-pointer makes `FindSet` and `MakeSet` $O(1)$, but `Union` must redirect every back-pointer of one set, so $O(n)$. (This is exactly our augmented-linked-list implementation.)
Q13. $O(n \log n)$. This is the weighted-union bound: a whole sequence costs $O(m + n \log n)$, which for n total operations is $O(n \log n)$.

★ **Seen on a past exam** (Dec 2019 Final, MCQ)

Q24. `UNKNOWN` operates on a disjoint-set forest. Identify it:

```
UNKNOWN(x):
    if x ≠ x.p:
        x.p = UNKNOWN(x.p)
    return x.p
```

Q23. Kruskal's algorithm employs which data structure?

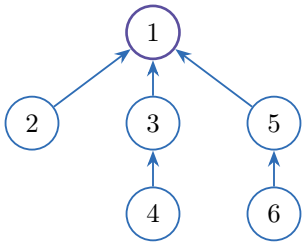
Q24. Find-Set with path compression. It walks up to the root and, on the way back, re-points every node on the path straight to the root (next lecture).
Q23. A disjoint set (Union–Find), used to detect whether adding an edge would form a cycle.

7 Midterm-Style Process Problems

Predicted: run a sequence of Weighted Union operations

Start with MakeSet$(1), \ldots,$ MakeSet(6). Apply, using **weighted union by size** (hang the smaller tree's root under the larger; on a tie the first argument becomes the root): Union$(1,2)$, Union$(3,4)$, Union$(5,6)$, Union$(1,3)$, Union$(1,5)$. Draw the final forest.

- Union$(1,2), (3,4), (5,6)$: three size-2 trees rooted at 1, 3, 5 (each tie makes the first arg the root).
- Union$(1,3)$: both size 2, tie, so root 1; 3 (with its child 4) hangs under 1.
- Union$(1,5)$: sizes 4 vs 2, so the smaller set (rooted 5) hangs under 1.



Root 1 holds all 6 elements. Weighted union keeps the height at $O(\log n)$ (a tree of height h needs $\geq 2^h$ nodes), so each FindSet is $O(\log n)$ and a sequence of m operations costs $O(m \log n)$.

Predicted: which implementation for which cost

Fill the worst-case single-operation cost for each disjoint-set implementation.

	MakeSet	FindSet	Union
Linked list (with rep pointer)	$O(1)$	$O(1)$	$O(n)$ (relabel the smaller list)
Forest, no heuristics	$O(1)$	$O(n)$ (tall chain)	$O(n)$
Forest, weighted union	$O(1)$	$O(\log n)$	$O(\log n)$

The forest with weighted union is the one worth memorizing: it turns the degenerate $O(n)$ chain into $O(\log n)$ height.

Quick Reference: Lecture 11

ADT	n elements, singletons; rep. per set; **Union**(distinct reps), **Find**(z)
σ	$n-1$ Unions mixed with $m \geq n$ Finds; minimize total cost
Linked list	Union $O(1)$, Find $O(n)$ $\rightarrow$ $\sigma = O(mn)$
Augmented list	Find $O(1)$, Union $O(n)$ $\rightarrow$ $\sigma = O(n^2 + m)$
WU on lists	append smaller onto bigger $\rightarrow$ $\sigma = O(m + n \log n)$
Forest	non-root $\rightarrow$ parent, root = rep; Find $O(1 + \text{path})$, Union $O(1)$
No heuristic	chain of height $n-1$ $\rightarrow$ $\sigma = \Omega(mn)$
WU by size	smaller tree becomes child of bigger
WU lemma	height h tree has $\geq 2^h$ nodes $\Rightarrow h \leq \log_2 n$
WU theorem	σ takes $O(m \log n)$

Lecture 12: Disjoint Sets (Union–Find), Part 2

1 Where We Are

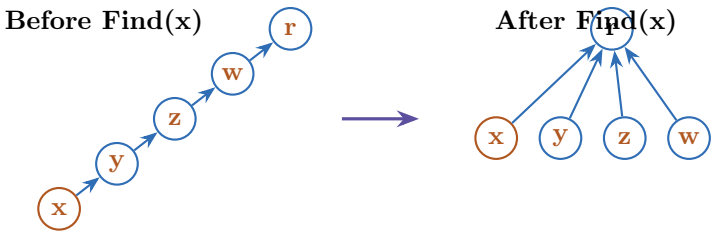
Recall

In the **forest** implementation, **Find**(x) costs $O(1+\text{length of the Find path})$ and **Union** costs $O(1)$. With **Weighted Union (WU)** every tree has height $\leq \log_2 n$, so σ costs $O(m \log n)$. We now add a second heuristic to push this down further.

2 Heuristic 2: Path Compression (PC)

PC rule (MUST KNOW)

During **Find**(x), after reaching the root, make **every vertex along the Find path a direct child of the root**.



Why PC helps: amortization

PC increases the cost of the current **Find**(x) (it rewires the whole path), but it makes *every future* **Find** on those nodes cheaper, since they now sit one step below the root.

$\Rightarrow$ the **average** (amortized) cost of each Find drops.

3 A Quick Detour: the Tower 2^{*n} and $\log^* n$

2^{*n} (a tower of 2's)

$$2^{*0} = 1, \qquad 2^{*(n+1)} = 2^{2^{*n}} \quad (n \geq 0).$$

Equivalently 2^{*n} is a stack of n twos: $2^{2^{\cdot^{\cdot^{2}}}}$.

$$\begin{aligned} 2^{*0} &= 1 & 2^{*3} &= 2^{2^{*2}} = 2^4 = 16 \\ 2^{*1} &= 2^{2^{*0}} = 2^1 = 2 & 2^{*4} &= 2^{2^{*3}} = 2^{16} = 65536 \\ 2^{*2} &= 2^{2^{*1}} = 2^2 = 4 & 2^{*5} &= 2^{2^{*4}} = 2^{65536} \approx 10^{19729} \quad \text{(REALLY big)} \end{aligned}$$

2^{*n} grows astronomically fast. (For scale, the number of atoms in the observable universe is about 10^{80}, already dwarfed by 2^{*5}.) Its inverse grows astronomically *slowly*:

$\log^* n$ (iterated logarithm)

$$\log^* n \;=\; \min\{\, k : 2^{*k} \geq n \,\}.$$

n	1	2	3, 4	$5, \ldots, 16$	$17, \ldots, 65536$	$65537, \ldots, 10^{19729}$
$\log^* n$	0	1	2	3	4	5

Takeaway

$\log^* n$ grows so slowly that for any input you will ever meet, $\log^* n \leq 5$. It is effectively a small constant, yet formally it still goes to infinity.

4 Time Complexity with WU and PC

Main Theorem (MUST KNOW)

With **both** Weighted Union and Path Compression, executing any sequence σ of $n-1$ Unions mixed with $m \geq n$ Finds takes

$$O(m \log^* n) \text{ time.}$$

That is, every such σ takes at most $m \log^* n$ time (for large m, n, within a constant factor).

Because $\log^* n \leq 5$ in practice, this is **essentially linear** in the number of operations, a huge improvement over $O(m \log n)$ from WU alone.

The theorem is an upper bound: *every* σ costs **at most** $O(m \log^* n)$. It does *not* claim $O(m)$. Whether some σ is forced to cost $\Omega(m \log^* n)$ or more is a separate (lower-bound) question, settled by later work below.

5 Analysis of the Disjoint-Set Forest: a Timeline

History of the bounds

Year	Result	Who
1964	forest implementation introduced	Galler & Fischer
1973	$O(m \log^* n)$ upper bound (WU + PC)	Hopcroft & Ullman
1975	$O(m\, \alpha^{-1}(m, n))$ upper bound	Tarjan
1979	$\Omega(m\, \alpha^{-1}(m, n))$ lower bound (restricted model)	Tarjan
1989	$\Omega(m\, \alpha^{-1}(m, n))$ lower bound (any structure)	Fredman & Saks

Here $\alpha(m, n)$ is the **inverse Ackermann function**, which grows even more slowly than $\log^* n$. The 1989 lower bound shows this near-constant per-operation cost is optimal: no disjoint-set data structure can do asymptotically better.

6 Past Exam Questions

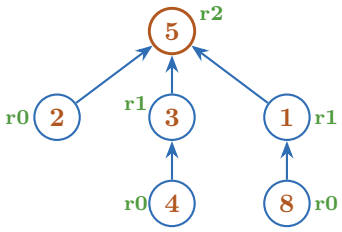
Exams (and CLRS) use **union-by-rank** and the operations `MakeSet`, `FindSet`, `Union`. Union-by-rank is the height-based cousin of our size-based WU: both hang the "lighter" tree under the "heavier" root and keep trees shallow. `FindSet` with path compression is our **Find** + PC. `Rank` starts at 0 for a singleton; merging two equal-rank roots bumps the surviving root's rank by 1.

★ **Seen on a past exam** (Apr 2022 Final, Q3a: draw the forest)

Run the following on a forest (union-by-rank + path compression), initially empty. Write each node's value and its **rank**. For each `Union`(x, y) with a rank tie, the new representative is the root of the set containing x.

```
MakeSet(5), MakeSet(2), Union(5,2), MakeSet(3), MakeSet(4),
Union(3,4),
Union(5,3), MakeSet(1), MakeSet(8), Union(1,8), FindSet(8),
Union(4,1)
```

Walking through it: `Union(5,2)` ties → 5 root (rank 1), 2 child. `Union(3,4)` ties → 3 root (rank 1), 4 child. `Union(5,3)` ties → 5 root (rank 2), 3 hangs under 5. `Union(1,8)` ties → 1 root (rank 1), 8 child. `FindSet(8)` returns 1 (path already length 1). `Union(4,1)`: root of 4 is 5 (rank 2), root of 1 is 1 (rank 1); lower rank hangs under higher, so 1 becomes a child of 5 (rank of 5 stays 2).



One tree: root 5 (rank 2) with children 2 (r0), 3 (r1, child 4), and 1 (r1, child 8).

★ **Seen on a past exam** (Apr 2022 Final, Q3b / Dec 2018 Final, Q2)

(2022, Q3b) L is a linked-list disjoint set with head pointers and union-by-weight; T is a tree disjoint set with union-by-rank and path compression. For a sequence S, let T_L, T_T be the total nodes accessed on L and T. Show there is a sequence with $T_T < T_L$.
(2018, Q2) Given a disjoint-set forest with union-by-rank + path compression, **(a)** draw the tree after a `FindSet`, and **(b)** draw the tree after a `Union` and state its rank.

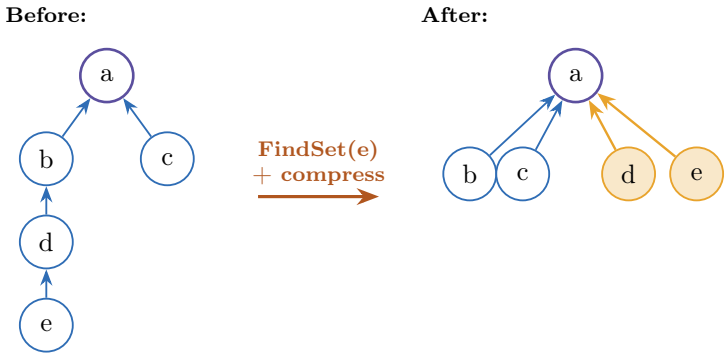
(2022, Q3b) idea. Build a tall tree, then do many Finds on its deepest node. On L each Find is $O(1)$ (head/back pointer), but on T the *first* Find triggers path compression, after which every later Find on that path is $O(1)$ too, while the list keeps paying for the long concatenations done by its Unions. Pick S as: a run of Unions that makes L redirect many pointers (large T_L) but that T handles with $O(1)$ rank-guided Unions and shallow trees. Then $T_T < T_L$.

(2018, Q2) method. `FindSet`(x): follow parents to the root, then re-point x and every node on the path straight to the root (path compression); ranks are unchanged. `Union`(x, y): find both roots, hang the smaller-rank root under the larger-rank root; on a tie, pick one root and increase its rank by 1. The new tree's rank is $\max(\text{rank}_x, \text{rank}_y)$, or that value $+1$ if the two ranks were equal.

7 Midterm-Style Process Problems

Predicted: FindSet with path compression

The forest below was built with union by rank. Run FindSet(e) with **path compression** and draw the result.



FindSet(e) walks $e \to d \to b \to a$ and returns the root a. Path compression then re-points *every node visited* straight to a, so e and d now point directly at a (b already did). Ranks are **unchanged** (rank only changes inside Union, on a tie). Flattening makes later finds cheaper: this is the amortized payoff behind the $O(m \log^* n)$ bound.

Predicted: $\log^*$ and the sequence cost

(a) Evaluate $\log^* n$ for $n = 2, 4, 16, 65536$. (b) State the cost of m operations under weighted union alone, versus weighted union plus path compression.

(a) $\log^* n$ is the number of times you apply $\log_2$ to bring n down to ≤ 1:

$$\log^* 2 = 1, \quad \log^* 4 = 2, \quad \log^* 16 = 3, \quad \log^* 65536 = 4.$$

It grows absurdly slowly: $\log^* n \leq 5$ for every n up to 2^{65536}, so in practice it is a small constant.
(b) Weighted union alone: a sequence of m operations costs $O(m \log n)$. Adding path compression brings it to $O(m \log^* n)$, essentially linear. (The tight bound is $O(m\,\alpha(m, n))$ with the inverse Ackermann function, proved optimal.)

Quick Reference: Lecture 12

Path Compression	in Find(x), re-point every node on the path to the root
PC effect	costs more now, cheaper later (amortized); average Find drops
2^{*n}	tower of n twos; $2^{*5} \approx 10^{19729}$; grows super fast
$\log^* n$	$\min\{k : 2^{*k} \geq n\}$; grows super slowly; ≤ 5 in practice
WU alone	$\sigma = O(m \log n)$
WU + PC	$\sigma = O(m \log^* n)$ (essentially linear)
Tarjan 1975	$O(m\,\alpha^{-1}(m, n))$, α = inverse Ackermann
Fredman–Saks 1989	$\Omega(m\,\alpha^{-1}(m, n))$ lower bound (optimal)
Exam terms	union-by-rank $\approx$ WU; FindSet + PC = Find + PC

End of Lectures 1–12 notes.